PROTOTIPO WEB PARA APOYAR LA ENSEÑANZA DE FUNDAMENTOS DE INTERPRETACIÓN Y COMPILACIÓN DE LENGUAJES DE PROGRAMACIÓN MEDIANTE LENGUAJES ORIENTADOS POR SINTAXIS

JOTA EMILIO LÓPEZ RAMÍREZ

Código: 2259394

jota.lopez@correounivalle.edu.co

ESMERALDA RIVAS GUZMÁN

Código: 2259580

esmeralda.rivas@correounivalle.edu.co

FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

PROGRAMA ACADÉMICO DE INGENIERÍA DE SISTEMAS

TULUÁ – VALLE DEL CAUCA

2026

PROTOTIPO WEB PARA APOYAR LA ENSEÑANZA DE FUNDAMENTOS DE INTERPRETACIÓN Y COMPILACIÓN DE LENGUAJES DE PROGRAMACIÓN MEDIANTE LENGUAJES ORIENTADOS POR SINTAXIS

JOTA EMILIO LÓPEZ RAMÍREZ

Código: 2259394

jota.lopez@correounivalle.edu.co

ESMERALDA RIVAS GUZMÁN

Código: 2259580

esmeralda.rivas@correounivalle.edu.co

Director

Carlos Andrés Delgado Saavedra, Ing.

carlos.andres.delgado@correounivalle.edu.co

FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

PROGRAMA ACADÉMICO DE INGENIERÍA DE SISTEMAS

TULUÁ – VALLE DEL CAUCA

2026

Tabla de Contenido

Resumen vii

Introducción viii

1. Contexto y objetivos 1
1.1. Problema de Investigación . . . 1
1.1.1. Descripción del problema . . . 1
1.1.2. Formulación del problema . . . 3
1.2. Objetivos . . . 3
1.2.1. Objetivo general . . . 3
1.2.2. Objetivos específicos y resultados esperados . . . 3
1.3. Alcance . . . 3
1.4. Estructura del documento . . . 4

2. Marco Referencial 6
2.1. Glosario . . . 6
2.2. Marco Teórico . . . 7
2.2.1. Fundamentos de la Interpretación de Lenguajes de Programación . . . 7
2.2.1.1. Gramáticas formales y notación BNF . . . 7
2.2.1.2. Árbol de Sintaxis Abstracta . . . 8
2.2.1.3. Proceso de interpretación y ambientes de ejecución . . . 9
2.2.1.4. Racket y la librería EOPL como marco pedagógico . . . 10
2.2.2. Teorías del Aprendizaje Aplicadas a la Enseñanza de Compiladores . . . 10
2.2.2.1. Teoría de la carga cognitiva . . . 10
2.2.2.2. Visualización de programas como apoyo al aprendizaje . . . 11
2.2.2.3. Evaluación por pasos como estrategia pedagógica . . . 11
2.2.3. Principios de Diseño de Herramientas Educativas Interactivas . . . 12
2.2.3.1. Usabilidad en tecnología educativa . . . 12
2.2.3.2. Organización visual y presentación simultánea de representaciones . . . 12
2.2.4. Relación conceptual entre gramática, AST y ambiente de ejecución . . . 12
2.3. Antecedentes . . . 13

3. Diagnóstico de dificultades conceptuales 17
3.1. Metodología . . . 17
3.1.1. Diseño de los instrumentos . . . 17

3.1.2. Selección de la población . . . 18
3.2. Análisis de la encuesta a estudiantes . . . 18
3.2.1. Resultados cuantitativos . . . 18
3.2.2. Resultados cualitativos . . . 20
3.2.3. Contraste con un diagnóstico previo de la asignatura . . . 20
3.3. Análisis de la encuesta a docentes . . . 21
3.3.1. Percepción general de los estudiantes . . . 22
3.3.2. Dificultades con paradigmas avanzados . . . 22
3.4. Revisión del microcurrículo . . . 23
3.5. Selección de conceptos base para el prototipo . . . 24

4. Diseño del prototipo FLP Viewer 26
4.1. Especificación de requisitos del sistema . . . 26
4.1.1. Enfoque metodológico . . . 26
4.1.2. Requisitos funcionales . . . 27
4.1.3. Requisitos no funcionales . . . 27
4.2. Diseño del prototipo . . . 28
4.2.1. Arquitectura del sistema . . . 28
4.2.1.1. Infraestructura de despliegue . . . 29
4.2.1.2. Capa de servidor . . . 29
4.2.1.3. Capa de cliente . . . 29
4.2.1.4. Pipeline de gramática . . . 30
4.2.2. Selección de tecnologías . . . 30
4.2.3. Prototipos de interfaz de usuario . . . 31
4.2.4. Estrategias de visualización . . . 34

5. Implementación del prototipo FLP Viewer 36
5.1. Repositorio de código fuente . . . 36
5.2. Implementación de los componentes . . . 37
5.2.1. Pipeline de gramática BNF . . . 37
5.2.2. Componente orquestador y paneles de la interfaz . . . 39
5.2.3. Biblioteca de ejemplos predefinidos . . . 43
5.2.4. Punto de acceso de ejecución . . . 44
5.2.5. Instrumentación en tiempo de ejecución . . . 45
5.3. Flujo de ejecución principal . . . 46
5.4. Despliegue en entorno de pruebas . . . 49
5.5. Verificación de requisitos funcionales . . . 50
5.6. Evaluación de la calidad del software . . . 51

6. Evaluación del prototipo FLP Viewer 54
6.1. Metodología . . . 54
6.1.1. Diseño del instrumento . . . 55
6.1.2. Selección de la población . . . 56
6.1.3. Procedimiento de aplicación . . . 56
6.1.4. Plan de análisis . . . 57
6.2. Limitaciones de aprendizaje documentadas en la literatura . . . 57

6.2.1. De la gramática a la estructura jerárquica 57
6.2.2. Ambientes de ejecución y estado 58
6.2.3. Ligadura, asignación y alcance 58
6.2.4. Herramientas de trabajo como fuente de carga adicional 58

7. Conclusiones **59**

Bibliografía **60**

A. Microcurrículo de FLP **64**

B. Encuesta a estudiantes **65**

C. Encuesta a docentes **66**

D. Protocolo de búsqueda sistemática de literatura **67**
D.1. Preguntas de búsqueda 67
D.2. Fuentes de información 67
D.2.1. Bases de datos primarias 67
D.2.2. Fuentes complementarias 68
D.3. Estrategia de búsqueda 68
D.3.1. Conceptos y términos 68
D.3.2. Cadenas de búsqueda por base de datos 68
D.4. Criterios de elegibilidad 70
D.4.1. Criterios de inclusión 70
D.4.2. Criterios de exclusión 70
D.5. Proceso de selección 70
D.6. Estudios incluidos 71

E. Requisitos del sistema **72**
E.1. Requisitos Funcionales 72
E.2. Requisitos No Funcionales 76

F. Instrumento de evaluación del prototipo **80**
F.1. Ficha técnica 80
F.2. Consentimiento informado 80
F.3. Sección A. Datos de contexto 81
F.4. Sección B. Escala de Usabilidad del Sistema (SUS) 81
F.5. Sección C. Cuestionario de utilidad pedagógica 81
F.6. Sección D. Preguntas abiertas 82

Lista de Figuras

1.1. Resultados de la encuesta aplicada a estudiantes de FLP (n=36) 2

2.1. AST de `(diff 3 (diff 1 2))`: cada producción BNF origina un constructor del tipo de datos. 8
2.2. Cadena de marcos tras `(extend-env 'x 3 (extend-env 'y 7 (empty-env)))`. `apply-env` recorre de izquierda a derecha. 9
2.3. Comparación entre el modelo sin IMPLICIT-REFS, donde el ambiente almacena directamente el valor, y el modelo con IMPLICIT-REFS, donde el ambiente almacena una referencia hacia una celda mutable en el *store*. 9
2.4. Relación conceptual entre los componentes del proceso de interpretación en el modelo EOPL. 13

4.1. Diagrama de arquitectura del sistema 30
4.2. Wireframe de la interfaz principal del prototipo FLP Viewer 33
4.3. Wireframe del modal de definición de gramática BNF 34

5.1. Pipeline de gramática BNF: de la especificación de entrada a los archivos Racket generados, pasando por el análisis léxico, el análisis sintáctico y la generación de código. 38
5.2. Modal de bienvenida del prototipo FLP Viewer 39
5.3. Modo de evaluación paso a paso con pasos pendientes de navegación 40
5.4. Modal de definición de gramática BNF con vista previa del archivo generado 41
5.5. Evaluación de una expresión con iteración y estado mutable: suma acumulada mediante bucle `for` 42
5.6. Evaluación de una función recursiva con clausura: la ligadura `f` en el panel de ambiente refleja una función como valor de primera clase 43
5.7. Diagrama de secuencia de la solicitud y evaluación en el servidor del prototipo FLP Viewer 47
5.8. Diagrama de secuencia de la distribución de resultados en la interfaz del prototipo FLP Viewer 48
5.9. Diagrama de despliegue del prototipo FLP Viewer en el entorno de pruebas 50
5.10. Panel de resultados del análisis de calidad del prototipo FLP Viewer en SonarCloud 52

D.1. Diagrama de flujo PRISMA del proceso de selección de antecedentes. 71

Lista de tablas

1.1. Objetivos Específicos 3

2.1. Síntesis comparativa de herramientas relacionadas con la enseñanza de compiladores e intérpretes (2021–2026) 14

3.1. Distribución de respuestas - encuesta a estudiantes de FLP ($n = 36$) 19
3.2. Percepción general de los docentes sobre habilidades de los estudiantes ($n = 5$) . . . 22
3.3. Percepción de los docentes de semestres avanzados sobre programación lógica y paradigmas avanzados ($n = 3$) 23
3.4. Indicadores de logro seleccionados como base para el prototipo 25

4.1. Resumen de requisitos funcionales del sistema 27
4.2. Resumen de requisitos no funcionales del sistema 28
4.3. Tecnologías seleccionadas y justificación 31

5.1. Biblioteca de ejemplos predefinidos y correspondencia con indicadores de logro . . . 43
5.2. Verificación del cumplimiento de los requisitos funcionales 50
5.3. Resultados del análisis de calidad del prototipo FLP Viewer mediante SonarCloud . 52

D.1. Bloques conceptuales de la estrategia de búsqueda (C1 `AND` C2 `AND` C3). 68
D.2. Estudios incluidos en la síntesis de antecedentes y su vía de identificación. 71

E.1. RF-01: Generación del AST 72
E.2. RF-02: Visualización gráfica e interactiva del AST 73
E.3. RF-03: Ejecución y evaluación del programa 73
E.4. RF-04: Representación del ambiente de ejecución 74
E.5. RF-05: Biblioteca de ejemplos 74
E.6. RF-06: Soporte y validación de sintaxis en Racket 75
E.7. RF-07: Visualización de los componentes internos del intérprete 75
E.8. RF-08: Plantilla base editable y exportación del entorno 76
E.9. RNF-01: Usabilidad 77
E.10. RNF-02: Rendimiento 77
E.11. RNF-03: Accesibilidad web 78
E.12. RNF-04: Mantenibilidad 78
E.13. RNF-05: Escalabilidad 79

F.1. Ficha técnica del instrumento de evaluación 80
F.2. Ítems del cuestionario SUS aplicado 81
F.3. Ítems del cuestionario de utilidad pedagógica 82
F.4. Relación entre los ítems del cuestionario de utilidad pedagógica y los indicadores de logro 82

Resumen

Palabras clave: interpretación de lenguajes de programación, visualización de programas, ambiente de ejecución, árbol de sintaxis abstracta, herramientas educativas interactivas, EOPL, Racket.

Introducción

La comprensión de los mecanismos internos mediante los cuales un programa escrito en un lenguaje de alto nivel se transforma en un cómputo ejecutable constituye uno de los pilares formativos de la Ingeniería de Sistemas. Los cursos de interpretación y compilación de lenguajes de programación introducen al estudiante en procesos como el análisis léxico y sintáctico, la construcción de árboles de sintaxis abstracta y la gestión de ambientes de ejecución, conceptos que sustentan áreas tan diversas como el desarrollo de compiladores, las herramientas de análisis estático y el diseño de lenguajes de dominio específico. Sin embargo, la naturaleza abstracta de estos contenidos y su distancia respecto a la experiencia de programación habitual del estudiante hacen de su enseñanza un reto reconocido en la literatura de educación en computación [1, 2].

En el programa de Ingeniería de Sistemas de la Universidad del Valle sede Tuluá, este reto se concentra en la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación* (FLP), que emplea el lenguaje Racket junto con los materiales del libro *Essentials of Programming Languages* (EOPL) como entorno principal de trabajo. Si bien esta combinación ofrece una base conceptual rigurosa, exige del estudiante una familiaridad previa con el paradigma funcional y con una sintaxis que, en ausencia de dicha familiaridad, tiende a desplazar su atención desde los conceptos de interpretación hacia la resolución de errores sintácticos [3]. A esta fricción se suma la carencia de recursos interactivos que permitan explorar de forma autónoma los procesos internos estudiados, pese a que la evidencia disponible señala que las representaciones visuales e interactivas favorecen la comprensión de estos procesos al hacer tangibles las transformaciones que ocurren en cada etapa del procesamiento de un programa [4–6].

Frente a este panorama, el presente trabajo de grado propone el diseño, la implementación y la evaluación de FLP Viewer, un prototipo web orientado a apoyar la comprensión de los contenidos de la asignatura FLP mediante la visualización de las estructuras internas que un intérprete construye y manipula durante la evaluación de un programa. A diferencia de las herramientas de visualización revisadas en el estado del arte, que operan sobre lenguajes predefinidos, el prototipo permite al estudiante definir su propia gramática mediante notación BNF y observar, de forma simultánea, el árbol de sintaxis abstracta generado y la evolución del ambiente de ejecución asociado.

El Capítulo 1 formula el problema de investigación con base en el diagnóstico realizado, plantea los objetivos del trabajo y delimita su alcance. Los capítulos posteriores documentan el marco referencial, el diagnóstico de dificultades conceptuales, el diseño y la implementación del prototipo, y los resultados de su evaluación con estudiantes de la asignatura.

Capítulo 1

Contexto y objetivos

1.1. Problema de Investigación

1.1.1. Descripción del problema

La interpretación y compilación de lenguajes de programación constituyen uno de los ejes centrales en la formación de ingenieros de sistemas. Comprender cómo un programa escrito en un lenguaje de alto nivel es procesado, analizado y ejecutado por un sistema computacional permite sustentar el trabajo en múltiples áreas de la disciplina, desde el desarrollo de compiladores y herramientas de análisis estático, hasta el diseño de lenguajes de dominio específico. Sin embargo, la enseñanza de estos contenidos enfrenta dificultades por sus conceptos abstractos, densos y difíciles de conectar con la práctica [1, 2].

En el programa de Ingeniería de Sistemas de la Universidad del Valle sede Tuluá, la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación* (FLP) demanda una comprensión teórica sólida y una capacidad de visualización de procesos internos que no resultan evidentes a partir del código fuente [6]. A lo largo de su desarrollo académico, los estudiantes deben asimilar conceptos como el análisis léxico y sintáctico, la construcción y recorrido de árboles de sintaxis abstracta (AST), la representación de ambientes de ejecución, la gestión del alcance léxico y el modelado de clausuras.

La principal herramienta de trabajo de la asignatura es el lenguaje Racket junto con los materiales del libro Essentials of Programming Languages (EOPL) [7]. Si bien esta combinación ofrece una base conceptual rigurosa y permite implementar intérpretes funcionales, su aprovechamiento exige una familiaridad previa con el entorno, así como un dominio sólido del paradigma funcional, en el que los estudiantes pueden presentar ciertos vacíos conceptuales. En particular, la sintaxis de prefijo de Racket, su estructura basada en paréntesis y los identificadores propios del entorno tienden a desplazar la atención cognitiva del estudiante desde los conceptos fundamentales de interpretación hacia la resolución de errores sintácticos del lenguaje utilizado [3].

Además de la carga impuesta por el entorno de programación, la asignatura carece de recursos interactivos que permitan a los estudiantes visualizar de manera dinámica y progresiva los procesos internos que se estudian. La literatura en tecnología educativa ha demostrado consistentemente que las representaciones visuales e interactivas favorecen la comprensión de procesos de compilación e

interpretación al hacer tangibles las transformaciones que ocurren en cada etapa del procesamiento de un programa [4, 5]. En ausencia de estos recursos, el aprendizaje depende en gran medida de las explicaciones del docente en clase, lo que reduce la autonomía del estudiante y limita las posibilidades de exploración fuera del aula.

Con el propósito de caracterizar estas dificultades desde la experiencia directa de quienes han cursado la asignatura, se aplicaron dos encuestas: una dirigida a 36 estudiantes que cursaron la asignatura en distintos períodos académicos, y otra a cinco docentes del programa. Los resultados de ambos instrumentos se analizan en detalle en el Capítulo 3; no obstante, los datos de la encuesta estudiantil permiten fundamentar el planteamiento del problema con evidencia cuantitativa directa.

Como se aprecia en la Figura 1.1, tres hallazgos resultan especialmente significativos. Primero, el 77.8 % de los estudiantes reportó haber tenido dificultades para comprender conceptos abstractos como entornos, estado o secuenciación, lo que confirma el carácter problemático de los contenidos de mayor abstracción. Segundo, apenas el 19.4 % se siente preparado para aplicar los conceptos aprendidos en otros cursos o proyectos, lo que revela una brecha importante entre la comprensión teórica y la transferencia práctica del conocimiento. Tercero, el 94.5 % de los encuestados consideró que recursos adicionales, como laboratorios interactivos, herramientas de visualización o ejemplos prácticos progresivos, habrían mejorado su aprendizaje de manera significativa.

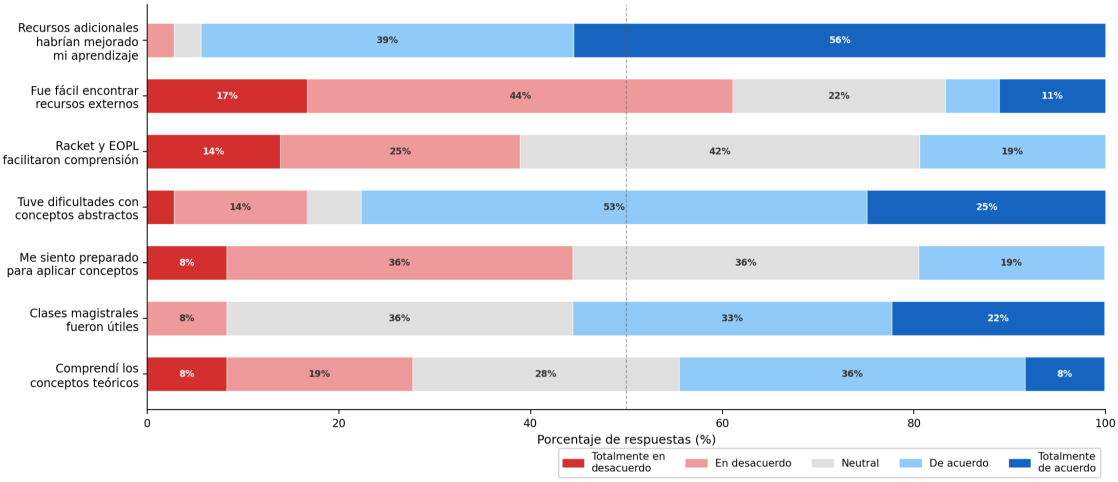


Figura 1.1: Resultados de la encuesta aplicada a estudiantes de FLP (n=36)
Fuente: elaboración propia

En síntesis, las dificultades observadas en la asignatura FLP responden a una combinación de factores: el alto nivel de abstracción inherente a los contenidos, la fricción cognitiva que genera el entorno de programación utilizado, la escasez de recursos externos de apoyo, y la ausencia de herramientas interactivas que permitan a los estudiantes explorar y visualizar los procesos que estudian. Esta situación genera dependencia de las clases presenciales, limita la autonomía en el aprendizaje y se traduce en una comprensión superficial de conceptos fundamentales para asignaturas posteriores y para el ejercicio profesional.

1.1.2. Formulación del problema

¿Cómo desarrollar un prototipo web que apoye la comprensión de los contenidos de la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación* (FLP) de la Universidad del Valle mediante la visualización de estructuras internas y el uso de lenguajes orientados por sintaxis?

1.2. Objetivos

1.2.1. Objetivo general

Desarrollar un prototipo web que apoye la comprensión de los contenidos de la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación* (FLP) de la Universidad del Valle mediante la visualización de estructuras internas y el uso de lenguajes orientados por sintaxis.

1.2.2. Objetivos específicos y resultados esperados

Objetivo específico	**Sección del documento**
Identificar las dificultades conceptuales que enfrentan los estudiantes de la asignatura, con el propósito de orientar la selección de ejemplos, casos de uso y mecanismos de visualización.	Capítulo 3
Caracterizar los fundamentos teóricos y técnicos relacionados con las gramáticas formales, el análisis sintáctico y la generación de árboles de sintaxis abstracta, que servirán como base conceptual para el diseño del prototipo.	Capítulo 2
Diseñar la arquitectura web y las interfaces de usuario, integrando estrategias de visualización que favorezcan la comprensión de las estructuras sintácticas.	Capítulo 4
Implementar un prototipo funcional que permita definir gramáticas, visualizar los árboles de sintaxis abstracta y los cambios en el estado durante la evaluación de los programas escritos por el usuario.	Capítulo 5
Evaluar la usabilidad y la utilidad pedagógica percibida del prototipo mediante pruebas con estudiantes inscritos en la asignatura, contrastando los resultados con las limitaciones de aprendizaje de estos conceptos documentadas en la literatura.	Capítulo 6

Tabla 1.1: Objetivos Específicos

1.3. Alcance

El presente trabajo se centra en el diseño e implementación de un prototipo web educativo que apoya la comprensión de los conceptos fundamentales de interpretación y compilación de lenguajes de programación en la asignatura FLP del programa de Ingeniería de Sistemas de la Universidad del Valle sede Tuluá.

Desde el punto de vista técnico, el prototipo cubre tres funcionalidades principales: (1) la definición y edición de gramáticas con sintaxis configurable por el usuario; (2) la visualización de árboles de sintaxis abstracta (AST) generados a partir de dichas gramáticas; y (3) la representación de los cambios en el ambiente durante la evaluación de programas escritos bajo la gramática definida. Estas funcionalidades están orientadas específicamente a los conceptos que, conforme al diagnóstico presentado en el Capítulo 3, presentan mayor dificultad de comprensión en la asignatura.

El alcance del prototipo se limita a funcionalidades demostrativas con fines pedagógicos. No se pretende desarrollar un lenguaje de propósito general ni implementar mecanismos avanzados de ejecución tales como depuración paso a paso, compilación a código máquina, optimización de rendimiento o autenticación y persistencia de usuarios. El sistema tampoco contempla despliegue en infraestructura institucional de uso permanente, puesto que su operación se restringe a un entorno de pruebas durante la fase de evaluación.

La evaluación del prototipo se llevó a cabo con un grupo de entre 10 y 20 estudiantes matriculados en la asignatura durante el período de desarrollo del trabajo. Dada la naturaleza y el tiempo disponible del proyecto, la evaluación se restringió a aspectos de funcionalidad técnica, usabilidad y utilidad pedagógica percibida, mediante la aplicación del cuestionario estandarizado System Usability Scale (SUS) y un cuestionario de utilidad pedagógica de elaboración propia, contrastados con las limitaciones de aprendizaje de estos conceptos documentadas en la literatura. Quedaron fuera del alcance las evaluaciones de impacto en el aprendizaje a largo plazo, pues estas requieren condiciones experimentales que exceden lo propio de un trabajo de grado.

Finalmente, el prototipo no pretende reemplazar los recursos actuales de la asignatura, en particular lo referente a Racket y EOPL, sino complementarlos, ofreciendo un punto de entrada más accesible y visual a los conceptos que los estudiantes identifican como los de mayor dificultad.

1.4. Estructura del documento

El presente documento está dividido en siete capítulos, además de los anexos que complementan la información presentada.

El Capítulo 1, *Contexto y objetivos*, plantea el problema de investigación, formula el objetivo general y los objetivos específicos del trabajo, y delimita el alcance técnico y evaluativo del prototipo a desarrollar.

El Capítulo 2, *Marco Referencial*, presenta el glosario de términos empleados a lo largo del documento, el marco teórico relacionado con los fundamentos de la interpretación de lenguajes de programación, las teorías del aprendizaje aplicadas a la enseñanza de compiladores y la relación conceptual entre gramática, AST y ambiente de ejecución, así como los antecedentes identificados mediante una búsqueda sistemática de literatura.

El Capítulo 3, *Diagnóstico de dificultades conceptuales*, describe la metodología empleada para identificar las dificultades de aprendizaje en la asignatura FLP, presenta el análisis de las encuestas aplicadas a estudiantes y docentes, y expone la selección de los conceptos base que orientan el diseño del prototipo a partir de la revisión del microcurrículo de la asignatura.

El Capítulo 4, *Diseño del prototipo FLP Viewer*, especifica los requisitos funcionales y no funcionales del sistema y describe las decisiones de diseño adoptadas, incluyendo la arquitectura del sistema, la

selección de tecnologías, los prototipos de interfaz de usuario y las estrategias de visualización.

El Capítulo 5, *Implementación del prototipo FLP Viewer*, documenta la implementación de los componentes del sistema, el flujo de ejecución principal, el proceso de despliegue en el entorno de pruebas y la verificación de los requisitos funcionales definidos en el Capítulo 4.

El Capítulo 6, *Evaluación del prototipo FLP Viewer*, describe la metodología empleada para evaluar el prototipo con estudiantes de la asignatura, y presenta los resultados obtenidos en términos de usabilidad, utilidad pedagógica y retroalimentación cualitativa, contrastados con las limitaciones de aprendizaje de estos conceptos documentadas en la literatura.

El Capítulo 7, *Conclusiones*, sintetiza los resultados alcanzados en contraste con los objetivos planteados, presenta las conclusiones generales del trabajo de grado y plantea las líneas de trabajo futuro identificadas a partir de sus limitaciones.

Finalmente, los anexos recogen el microcurrículo de la asignatura FLP, los instrumentos y resultados de las encuestas aplicadas a estudiantes y docentes, el protocolo de búsqueda sistemática de literatura, las especificaciones de requisitos del sistema y el instrumento empleado en la evaluación del prototipo.

Capítulo 2

Marco Referencial

2.1. Glosario

- **Ambiente de ejecución (*environment*):** Estructura de datos que asocia identificadores con sus valores durante la evaluación de un programa. Se organiza como una cadena de marcos (*frames*), donde cada llamada a `extend-env` agrega un nuevo marco con las ligaduras introducidas por la expresión en evaluación [7].
- **Árbol de sintaxis abstracta (AST):** Representación jerárquica de la estructura semántica de un programa, producida por el analizador sintáctico a partir del código fuente. Elimina los detalles superficiales de la notación concreta y conserva únicamente la información relevante para la evaluación del programa. En el modelo de EOPL, los nodos del AST corresponden a los constructores del tipo de datos definido mediante `define-datatype` [7].
- **BNF (notación de Backus-Naur):** Metalenguaje estándar para la especificación de gramáticas libres de contexto mediante reglas de producción de la forma $\langle \text{no-terminal} \rangle ::= \alpha_1 \mid \alpha_2 \mid \ldots$. En el prototipo propuesto en este trabajo, el estudiante utiliza notación BNF para definir la gramática de su intérprete [8].
- **Carga cognitiva:** Cantidad de esfuerzo mental requerido para procesar información en la memoria de trabajo. La teoría distingue entre carga intrínseca (complejidad inherente del contenido), extrínseca (generada por la forma de presentación) y germana (asociada a la formación de esquemas de conocimiento) [9].
- **Clausura (*closure*):** Valor que resulta de evaluar una expresión de función y que encapsula el cuerpo de la función junto con el ambiente léxico en el que fue definida. Las clausuras son la representación concreta de los procedimientos como valores de primera clase en el modelo de EOPL [7].
- **EOPL (*Essentials of Programming Languages*):** Libro de texto de Friedman y Wand que presenta una metodología para la construcción de intérpretes y compiladores en Racket usando la librería SLLGEN [7]. Es el texto base de la asignatura FLP.
- **FLP:** Abreviatura empleada en este documento para referirse a la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación* (código 750017C) del programa

de Ingeniería de Sistemas de la Universidad del Valle.

- **Indicador de logro (IL):** Criterio de evaluación del microcurrículo de la asignatura FLP que especifica el desempeño observable esperado en relación con un resultado de aprendizaje particular.
- **Ligadura (*binding*):** Asociación entre un identificador y un valor dentro de un marco del ambiente de ejecución. En el modelo funcional de EOPL, las ligaduras son inmutables: una vez creadas, no pueden modificarse; únicamente pueden ocultarse mediante la creación de un nuevo marco con el mismo identificador en el ambiente extendido [7].
- **Referencia implícita (*implicit reference*):** Modelo de ambiente en el que los valores se almacenan en celdas mutables representadas mediante referencias, y el ambiente contiene punteros a dichas celdas. Este modelo habilita la asignación destructiva mediante `set` sin crear un nuevo marco en el ambiente [7].
- **Resultado de aprendizaje (R.A.):** Componente del microcurrículo de la asignatura FLP que describe una competencia general que el estudiante debe alcanzar al finalizar el curso. Cada R.A. se desglosa en varios indicadores de logro (IL), con pesos evaluativos diferenciados sobre la nota total de la asignatura.
- **SLLGEN:** Biblioteca de análisis sintáctico incluida en Racket que genera automáticamente un escáner y un analizador a partir de una especificación léxica y una gramática BNF. Es la herramienta utilizada en EOPL para automatizar las etapas de análisis léxico y sintáctico en la construcción de intérpretes [7].

2.2. Marco Teórico

El presente proyecto se apoya en tres áreas de conocimiento: los fundamentos técnicos de la interpretación de lenguajes de programación, que constituyen el objeto de estudio que el prototipo busca hacer visible, las teorías del aprendizaje aplicadas a la educación en computación, que orientan las decisiones pedagógicas del diseño, y los principios de usabilidad y visualización en herramientas educativas interactivas, que guían las decisiones de interfaz. Los apartados siguientes desarrollan cada una de estas áreas.

2.2.1. Fundamentos de la Interpretación de Lenguajes de Programación

2.2.1.1. Gramáticas formales y notación BNF

Los lenguajes de programación se definen formalmente mediante gramáticas libres de contexto, clasificadas por Chomsky en el nivel dos de su jerarquía de lenguajes formales [10]. Una gramática libre de contexto se compone de un conjunto de símbolos terminales, un conjunto de símbolos no terminales, un símbolo inicial y un conjunto de reglas de producción que especifican cómo cada símbolo no terminal puede reemplazarse por una secuencia de símbolos terminales y no terminales. Esta estructura proporciona una descripción precisa y no ambigua de la sintaxis del lenguaje, que puede ser procesada de forma automática por un analizador sintáctico.

La notación de Backus-Naur (BNF) es un metalenguaje estándar para expresar gramáticas libres de contexto de forma legible para el ser humano [8]. En BNF, cada regla de producción adopta la

forma ⟨no-terminal⟩ ::= $\alpha_1 \mid \alpha_2 \mid \ldots$, donde las alternativas están separadas por el símbolo "|". Esta notación es el punto de entrada con el que el estudiante define la gramática de su intérprete en el prototipo propuesto: a partir de las reglas escritas en el área de definición de gramática, el pipeline genera automáticamente los archivos de código Racket necesarios para que SLLGEN construya el analizador correspondiente.

En el contexto de la librería EOPL, la gramática BNF se utiliza directamente como especificación de entrada para SLLGEN, la biblioteca de análisis sintáctico incluida en Racket [7]. A partir de dicha especificación, SLLGEN genera de forma automática un escáner y un analizador sintáctico, de modo que el estudiante puede concentrarse en la implementación de la semántica de su lenguaje sin necesidad de desarrollar manualmente las etapas de análisis léxico y sintáctico.

2.2.1.2. Árbol de Sintaxis Abstracta

El análisis sintáctico transforma la secuencia de tokens producida por el escáner en un árbol de sintaxis abstracta (AST), que representa la estructura jerárquica y semántica del programa con independencia de los detalles superficiales de la notación concreta [8]. A diferencia del árbol de derivación[1], el AST elimina elementos como paréntesis, palabras clave auxiliares y otros tokens cuya función es únicamente desambiguar la notación, pero que no contribuyen al significado del programa.

En el modelo de EOPL, el AST se representa mediante un tipo de datos algebraico definido con la macro `define-datatype` de Racket [7]. Cada regla de producción de la gramática se corresponde con un constructor del tipo de datos, de modo que existe una relación directa y verificable entre la estructura de la gramática BNF y la estructura del árbol. Esta correspondencia es pedagógicamente relevante porque permite al estudiante observar en el visualizador de AST del prototipo cómo cada construcción del programa origina un nodo específico del árbol, haciendo visible la relación entre la especificación formal del lenguaje y la representación interna que el intérprete utiliza para evaluar expresiones. La figura 2.1 ilustra esta correspondencia para la expresión `(diff 3 (diff 1 2))`.

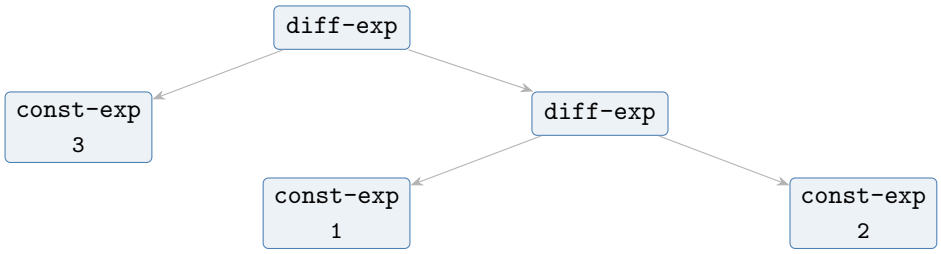


Figura 2.1: AST de `(diff 3 (diff 1 2))`: cada producción BNF origina un constructor del tipo de datos.

Fuente: Elaboración propia

[1]El *árbol de derivación* (o árbol de análisis sintáctico) representa de forma completa la secuencia de reglas de producción aplicadas para derivar una cadena a partir del símbolo inicial de la gramática, incluyendo paréntesis, palabras clave auxiliares y demás tokens de la notación concreta. El AST, en cambio, conserva únicamente la información necesaria para representar el significado del programa.

2.2.1.3. Proceso de interpretación y ambientes de ejecución

La interpretación de un programa consiste en recorrer el AST de forma recursiva y calcular el valor de cada nodo de acuerdo con las reglas semánticas del lenguaje [7]. En el modelo de EOPL, esta operación está encapsulada en la función `eval-expression`, que despacha sobre el tipo de cada nodo del árbol mediante la forma `cases`. Para cada constructor del tipo de datos, existe un caso que especifica cómo calcular el valor correspondiente, mayoritariamente invocando de forma recursiva a `eval-expression` sobre los subárboles del nodo.

Durante la evaluación, el intérprete mantiene un ambiente de ejecución que asocia identificadores con sus valores. Friedman y Wand definen el ambiente como una función parcial de identificadores a valores denotados, y la operación fundamental sobre él es la búsqueda mediante `apply-env`, que recorre la cadena de marcos desde el más reciente hasta el ambiente inicial [7]. La extensión del ambiente mediante `extend-env` agrega un nuevo marco con las ligaduras introducidas por la expresión en evaluación, sin modificar los marcos preexistentes en el modelo funcional, como se ilustra en la figura 2.2.

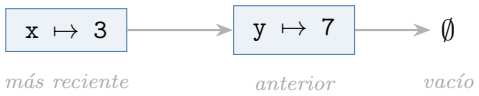


Figura 2.2: Cadena de marcos tras `(extend-env 'x 3 (extend-env 'y 7 (empty-env)))`. `apply-env` recorre de izquierda a derecha.

Fuente: Elaboración propia

A partir del Capítulo 4 de EOPL, el modelo introduce las referencias implícitas (IMPLICIT-REFS): los valores dejan de almacenarse directamente en el ambiente y pasan a residir en celdas mutables representadas por referencias. Esta modificación habilita la asignación destructiva mediante `set`, que modifica el contenido de una celda sin agregar un nuevo marco al ambiente. La diferencia entre ligadura y asignación que emerge de este cambio arquitectónico es uno de los conceptos de mayor dificultad en cursos que siguen el modelo de EOPL, dado que requiere distinguir entre dos operaciones que en la superficie parecen equivalentes pero tienen efectos estructuralmente distintos sobre el ambiente [11], tal como se compara en la figura 2.3.



Figura 2.3: Comparación entre el modelo sin IMPLICIT-REFS, donde el ambiente almacena directamente el valor, y el modelo con IMPLICIT-REFS, donde el ambiente almacena una referencia hacia una celda mutable en el *store*.

Fuente: Elaboración propia

2.2.1.4. Racket y la librería EOPL como marco pedagógico

Racket es un lenguaje de programación diseñado con propósitos docentes y de investigación en lenguajes de programación [12]. Su característica distintiva es la capacidad de modificar y extender el propio lenguaje mediante macros, lo que lo convierte en una plataforma especialmente adecuada para la implementación de intérpretes. En el contexto de la asignatura FLP, Racket se utiliza como plataforma para la construcción de los intérpretes descritos en EOPL, aprovechando su sistema de módulos y las herramientas de la librería para la generación automática de analizadores.

La librería EOPL proporciona, además de SLLGEN, las macros `define-datatype` y `cases`, que permiten definir tipos de datos algebraicos y despachar sobre ellos de forma idiomática [7]. Al declarar explícitamente los constructores del AST como constructores de un tipo de datos algebraico, el estudiante mantiene una correspondencia directa entre la gramática del lenguaje y la estructura del evaluador. En conjunto, estas herramientas permiten que la atención del estudiante se concentre en la semántica del lenguaje más que en los aspectos mecánicos del análisis sintáctico o la gestión de estructuras de datos, lo cual es coherente con el principio de reducción de la carga cognitiva extrínseca descrito en la sección siguiente [9].

2.2.2. Teorías del Aprendizaje Aplicadas a la Enseñanza de Compiladores

El diseño pedagógico del prototipo se fundamenta en tres corrientes teóricas que la literatura especializada ha identificado como relevantes para la enseñanza de contenidos de alta abstracción en ciencias de la computación: la teoría de la carga cognitiva, la visualización de programas como estrategia de apoyo al aprendizaje, y la evaluación por pasos como mecanismo para hacer observable el proceso de interpretación.

2.2.2.1. Teoría de la carga cognitiva

La teoría de la carga cognitiva, desarrollada por Sweller, establece que la memoria de trabajo posee una capacidad limitada para procesar información nueva de forma simultánea [9]. La teoría distingue tres tipos de carga: la intrínseca, determinada por la complejidad inherente del contenido y la interacción entre sus elementos; la extrínseca, generada por la forma en que la información se presenta; y la germana, asociada a los procesos mentales que contribuyen a la formación de esquemas de conocimiento duraderos. El diseño instruccional efectivo busca mantener la carga intrínseca en un nivel manejable, minimizar la carga extrínseca y maximizar la germana.

En el contexto de la enseñanza de compiladores e intérpretes, la carga intrínseca es elevada porque los conceptos involucrados, como gramáticas, AST, ambientes y clausuras, son abstractos y están interrelacionados de forma no trivial [11]. Esta complejidad hace especialmente necesario controlar la carga extrínseca mediante decisiones de diseño que faciliten la interpretación de la información presentada al estudiante. Cuando el estudiante debe coordinar fuentes dispersas de información durante su trabajo, como una especificación de gramática en un archivo, el código del evaluador en otro y la salida del intérprete en la consola del sistema operativo, la presión sobre la memoria de trabajo aumenta y los recursos cognitivos disponibles para la comprensión del contenido disminuyen [9]. El diseño de herramientas de apoyo para este tipo de cursos debe buscar reducir esa carga extrínseca integrando en un único entorno las representaciones relevantes del proceso de interpretación.

2.2.2.2. Visualización de programas como apoyo al aprendizaje

La visualización de programas es un área de investigación que estudia el uso de representaciones gráficas para apoyar la comprensión de conceptos abstractos de programación. Sorva et al. realizaron una revisión sistemática de los sistemas de visualización de programas para la enseñanza introductoria y concluyeron que la efectividad de estas herramientas está directamente relacionada con el grado en que la representación visual se corresponde con el modelo conceptual que el estudiante debe construir [13]. Cuando la visualización refleja con fidelidad las estructuras que el docente describe en clase, el estudiante puede establecer correspondencias directas entre la teoría y la herramienta, favoreciendo la comprensión y reduciendo el riesgo de formarse concepciones erróneas.

Python Tutor, desarrollado por Guo [14], es uno de los sistemas de visualización de ejecución más utilizados en educación en computación. La herramienta representa paso a paso el estado del ambiente de ejecución, incluyendo los marcos de activación, el montículo de objetos y los punteros entre ellos, para lenguajes como Python, Java y JavaScript. Su diseño demostró que la representación explícita del ambiente de ejecución facilita la comprensión de conceptos como el alcance de variables, la recursión y las referencias entre objetos. Sin embargo, Python Tutor no admite Racket ni visualiza la estructura interna de los intérpretes definidos por el usuario, lo que lo hace insuficiente para las necesidades específicas de la asignatura FLP.

La herramienta presentada por Spigariol et al. [3] constituye el antecedente más cercano al prototipo desarrollado en este trabajo en cuanto a objetivo pedagógico: combina un intérprete de un lenguaje funcional con componentes gráficos que visualizan las estructuras intermedias del proceso de interpretación. No obstante, dicha herramienta está diseñada para un lenguaje predefinido y no permite al estudiante definir su propia gramática ni trabajar con el modelo de EOPL y SLLGEN.

2.2.2.3. Evaluación por pasos como estrategia pedagógica

Clements, Flatt y Felleisen propusieron el modelo del *stepper* algebraico como herramienta para hacer observable el proceso de reducción de expresiones en lenguajes funcionales [15]. En este modelo, la evaluación de una expresión se presenta como una secuencia de pasos de reducción, donde cada paso muestra el estado del programa antes y después de la aplicación de una regla semántica. Este mecanismo permite al estudiante observar la correspondencia entre las reglas del evaluador y el comportamiento concreto del intérprete expresión por expresión.

La presentación de la evaluación como una secuencia de pasos discretos tiene fundamento en la teoría de la carga cognitiva: al exponer el resultado de una única expresión evaluada a la vez, la herramienta controla la cantidad de información nueva que el estudiante debe procesar de forma simultánea, reduciendo la presión sobre la memoria de trabajo y favoreciendo la comprensión progresiva del comportamiento del intérprete [9]. El prototipo desarrollado en este trabajo adopta este principio en el modo de evaluación paso a paso descrito en el Capítulo 4, aunque con una granularidad distinta a la del *stepper* algebraico de Clements et al.: en lugar de descomponer la evaluación de una expresión en sus reglas de reducción, el modo paso a paso presenta secuencialmente el resultado de cada expresión de nivel superior que el estudiante ingresa al intérprete, mostrando primero el resultado de la primera y permitiendo avanzar hacia las siguientes de forma controlada.

2.2.3. Principios de Diseño de Herramientas Educativas Interactivas

2.2.3.1. Usabilidad en tecnología educativa

La usabilidad se define como el grado en que un sistema puede ser utilizado por usuarios específicos para alcanzar objetivos concretos con efectividad, eficiencia y satisfacción en un contexto de uso determinado. En el ámbito de las herramientas educativas digitales, investigaciones recientes señalan que la usabilidad no incide únicamente en la eficiencia técnica sino también en la motivación y la retención del aprendizaje: una interfaz que genera fricción desplaza los recursos cognitivos del usuario desde la comprensión del contenido hacia la resolución de problemas de navegación o interacción [5]. Esta relación es especialmente relevante en herramientas para la enseñanza de compiladores, donde la carga cognitiva intrínseca ya es elevada.

En el ámbito de los cursos de compiladores, investigaciones previas han documentado que los estudiantes perciben las herramientas de desarrollo convencionales como una fuente adicional de dificultad, independientemente de la solidez técnica de los contenidos [11]. Por este motivo, el diseño de herramientas educativas para compiladores debe priorizar la minimización de los cambios de contexto y la integración de retroalimentación inmediata sobre los errores del estudiante.

2.2.3.2. Organización visual y presentación simultánea de representaciones

Norman señala que el diseño de interfaces debe respetar los límites de la atención humana, presentando la información de forma que la percepción y la interpretación requieran el mínimo esfuerzo posible [16]. En el diseño de herramientas educativas para compiladores, este principio se traduce en la presentación simultánea de representaciones complementarias del proceso de interpretación, de modo que el estudiante pueda relacionar el código fuente con el AST y el ambiente sin necesidad de cambiar de ventana ni de contexto mental.

La literatura sobre herramientas educativas para compiladores confirma que la presentación simultánea de representaciones sincronizadas favorece la comprensión [4, 6]. Además, la diferenciación visual mediante color y estructura tipográfica permite al estudiante identificar la naturaleza de cada elemento de la interfaz sin necesidad de leer su contenido completo, lo que reduce el esfuerzo de interpretación y contribuye a mantener los recursos cognitivos disponibles para el análisis del comportamiento del intérprete [17].

2.2.4. Relación conceptual entre gramática, AST y ambiente de ejecución

Los apartados anteriores describen por separado las estructuras y operaciones que conforman el proceso de interpretación en el modelo EOPL: la gramática BNF como especificación formal del lenguaje, el AST como representación interna que el analizador produce a partir de esa especificación, y el ambiente de ejecución como estructura dinámica que el evaluador consulta y extiende al recorrer el árbol. La figura 2.4 sintetiza estas relaciones y explicita la cadena que constituye el núcleo conceptual del curso: desde que el estudiante escribe una gramática hasta que el intérprete produce un valor y deja un rastro en el ambiente.

La flecha discontinua entre la gramática y el AST refleja la correspondencia estática descrita en la sección 2.2.1.2: cada producción BNF se convierte en un constructor del tipo de datos algebraico, de modo que la estructura del árbol es un reflejo directo de las decisiones que el estudiante toma al definir la gramática. Las flechas sólidas, en cambio, representan el flujo dinámico de la interpretación:

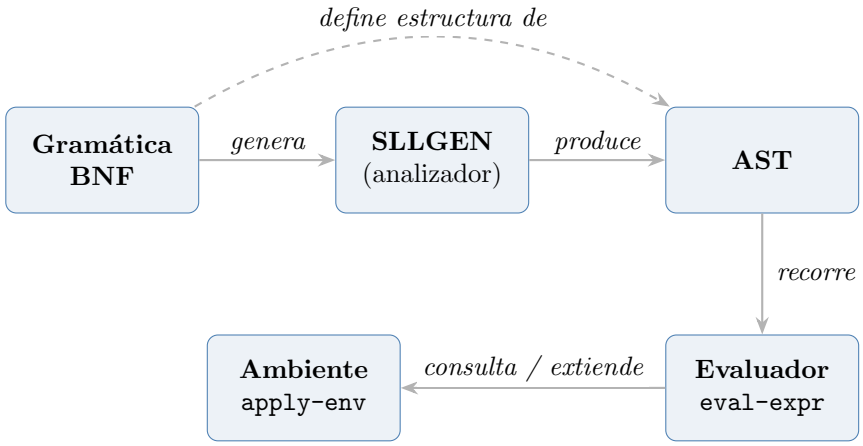


Figura 2.4: Relación conceptual entre los componentes del proceso de interpretación en el modelo EOPL.

Fuente: Elaboración propia

SLLGEN transforma el código fuente en un AST, el evaluador lo recorre de forma recursiva y, a medida que lo hace, consulta y extiende el ambiente. Esta distinción entre estructura estática y comportamiento dinámico es una de las fuentes de dificultad identificadas en la literatura para cursos de lenguajes de programación [11], y es precisamente lo que el prototipo propuesto busca hacer visible al presentar de forma simultánea el AST y el estado del ambiente tras cada paso de evaluación.

2.3. Antecedentes

La identificación de trabajos previos se realizó mediante una búsqueda sistemática en IEEE Xplore, ACM Digital Library y Scopus, acotada al período 2021–2026, utilizando términos relacionados con visualización de intérpretes, ambientes de ejecución, enseñanza de compiladores y herramientas educativas para cursos de lenguajes de programación. Los criterios de inclusión exigieron que los trabajos presentaran una herramienta o prototipo con componente de visualización o interactividad orientado a la enseñanza de alguna fase del proceso de interpretación o compilación, y que reportaran al menos una experiencia de uso o evaluación. El protocolo completo de búsqueda, incluyendo las cadenas booleanas utilizadas y el conteo de resultados por base, se presenta en el Anexo D.

Las propuestas encontradas pueden clasificarse en dos grupos: las que visualizan el pipeline de compilación clásico (análisis léxico, autómatas, tablas de análisis sintáctico) y las que apoyan específicamente la visualización del ambiente de ejecución y la evaluación de expresiones en lenguajes funcionales. La Tabla 2.1 presenta una síntesis comparativa de las herramientas más relevantes identificadas.

Tabla 2.1: Síntesis comparativa de herramientas relacionadas con la enseñanza de compiladores e intérpretes (2021–2026)

Autor / Año	**Herramienta**	**Aporte principal**	**Limitación / Vacío**
Jovanović et al. (2021) [4]	ComVIS	Entorno web interactivo para visualizar análisis léxico, construcción de autómatas y análisis sintáctico LL/LR con retroalimentación paso a paso. Reporta mejoras en comprensión y desempeño académico evaluadas mediante cuestionarios pre/post.	Centrado en el pipeline de compilación clásico. No aborda la evaluación de intérpretes ni los ambientes de ejecución funcionales.
Cai et al. (2023) [18]	Source Academy	Herramienta web que visualiza los ambientes de ejecución mediante representaciones gráficas de marcos y clausuras, argumentando que los modelos basados en pila no capturan correctamente la estructura de ambientes léxicos. Utilizada en un curso introductorio de CS en la Universidad Nacional de Singapur.	Diseñada para lenguajes predefinidos del proyecto Source Academy. No permite al estudiante definir su propia gramática ni visualizar el AST generado por un intérprete propio.
del Vado Vírseda (2023) [19]	ITT	Sistema de tutoría interactiva que visualiza el proceso de diseño de compiladores desde la implementación hacia la teoría, con ejercicios guiados y retroalimentación inmediata sobre análisis léxico, sintáctico y generación de código. Reporta experiencias de uso en cursos de la Universidad Complutense de Madrid.	Orientado al pipeline completo en lenguajes imperativos. No visualiza el ambiente de ejecución ni la evaluación de expresiones en intérpretes definidos por el propio estudiante.
Spigariol et al. (2023) [3]	Intérprete pedagógico funcional	Combina un intérprete de un lenguaje funcional con componentes gráficos que visualizan las estructuras intermedias del proceso de interpretación, permitiendo seguir el comportamiento del evaluador paso a paso. Es el antecedente más cercano, desde el punto de vista pedagógico, al prototipo propuesto en este trabajo.	Específico para un lenguaje predefinido. No permite al estudiante definir su propia gramática ni trabajar con el modelo EOPL y SLLGEN en Racket.
Abad y Henz (2024) [20]	CSE Machine	Extiende el modelo de ambientes de SICP a una máquina nocional completa para Scheme que visualiza paso a paso los ambientes, las clausuras y el control de flujo. Utilizada en CS1101S de la Universidad Nacional de Singapur con retroalimentación positiva de estudiantes y tutores.	Integrada en el ecosistema cerrado de Source Academy para un lenguaje predefinido. No está diseñada para que el estudiante construya e inspeccione su propio intérprete EOPL con gramática definida en SLLGEN.

(continuación)

Autor / Año	Herramienta	Aporte principal	Limitación / Vacío
Stamenković y Jovanović (2024) [6]	AWE	Sistema web educativo integral que combina visualización dinámica de estructuras de compilación, ejercicios guiados y experimentación directa para el aprendizaje del pipeline completo. Incluye módulos sobre análisis léxico y sintáctico, representación de árboles y generación de código. Reporta evidencia empírica de mejoras en comprensión con grupos de control y cuestionarios validados en instituciones de educación superior en Serbia.	Abarca el pipeline de compilación completo, pero no aborda la construcción interactiva de intérpretes EOPL ni la visualización del ambiente de ejecución desde la perspectiva del estudiante que implementa su propio evaluador.

Fuente: elaboración propia a partir de la revisión de la literatura.

De manera complementaria a la búsqueda sistemática, se consideró un antecedente desarrollado en el contexto institucional de la asignatura, que no fue identificado mediante el protocolo descrito por tratarse de un trabajo de grado no publicado en las bases de datos consultadas. Henao Gallego desarrolló un sistema de recomendación de materiales didácticos para FLP en la Universidad del Valle sede Tuluá, basado en generación aumentada por recuperación (RAG) y modelos de lenguaje de gran escala, que integra un asistente conversacional, una biblioteca centralizada de contenidos del curso y recomendaciones personalizadas según el perfil del estudiante [21]. Su diagnóstico identificó la dispersión del material didáctico y la complejidad de los temas como las principales causas de dificultad en la asignatura, y su evaluación con 22 estudiantes del período 2026-1 mostró una valoración favorable de la experiencia de uso, con un 86.4 % de calificaciones máximas en la interfaz y la totalidad de los participantes considerando útil la centralización de los recursos. Este trabajo aborda la problemática desde el acceso a los materiales de estudio, pero no ofrece una representación del proceso de interpretación que permita al estudiante observar la construcción del AST o la evolución del ambiente de ejecución. En consecuencia, ambas propuestas resultan complementarias, ya que atienden dimensiones distintas de una misma necesidad de apoyo identificada en la asignatura.

A partir del análisis comparativo de los trabajos identificados se reconocen tres vacíos que fundamentan el desarrollo de una propuesta específica para el contexto de la asignatura FLP. En primer lugar, las herramientas que más se aproximan al objetivo aquí planteado [3, 18, 20] operan sobre lenguajes predefinidos integrados en ecosistemas cerrados, sin posibilidad de que el estudiante defina su propia gramática ni intervenga en la estructura del evaluador. En segundo lugar, ninguna de las soluciones revisadas ofrece un flujo integrado que combine la edición de gramáticas BNF en notación SLLGEN, la visualización del AST resultante y la inspección del ambiente de ejecución dentro de un único entorno diseñado para acompañar el trabajo con la librería EOPL en Racket. En tercer lugar, la evidencia empírica en cursos de lenguajes de programación dictados en programas de ingeniería de universidades latinoamericanas es escasa, lo que demanda adaptación y validación contextualizada [4, 5].

Con el propósito de atender estos vacíos, este trabajo desarrolla un prototipo web diseñado para acompañar el aprendizaje de la asignatura FLP de la Universidad del Valle. A diferencia de las herramientas revisadas, el prototipo propuesto permite al estudiante definir su propia gramática en

notación BNF, genera automáticamente el código Racket necesario para que SLLGEN construya el analizador correspondiente y presenta de forma simultánea la estructura del AST producido junto con el estado del ambiente de ejecución tras cada evaluación. Esta integración busca reducir los cambios de contexto que el flujo de trabajo convencional con Racket y EOPL impone sobre el estudiante, concentrando en un único entorno las representaciones que se deben relacionar para comprender el proceso de interpretación [9].

Capítulo 3

Diagnóstico de dificultades conceptuales

Para identificar las dificultades conceptuales que enfrentan los estudiantes de la asignatura FLP, se realizó un diagnóstico basado en tres fuentes complementarias: una encuesta dirigida a estudiantes que han cursado la asignatura en distintos períodos académicos, una encuesta aplicada a docentes del programa de Ingeniería de Sistemas y una revisión del microcurrículo de la asignatura.

Estas fuentes permitieron caracterizar las dificultades desde tres perspectivas: la de los estudiantes que las experimentan directamente, la de los docentes que enseñan el contenido y la del diseño curricular que estructura el aprendizaje. Esta triangulación permitió una selección justificada de los conceptos que sirvieron como base para la biblioteca de casos predefinidos del prototipo. Adicionalmente, los resultados de la encuesta a estudiantes se contrastaron con un diagnóstico previo realizado en la misma asignatura [21], con el fin de verificar su consistencia frente a una fuente externa al presente trabajo.

3.1. Metodología

El diagnóstico se apoyó en un enfoque mixto, combinando métodos cuantitativos provenientes de encuestas con escala Likert [22] y métodos cualitativos a través de preguntas abiertas y análisis documental del microcurrículo.

3.1.1. Diseño de los instrumentos

El diseño de ambas encuestas se fundamentó en las dimensiones de evaluación identificadas en la literatura acerca de las dificultades en la enseñanza de compiladores e intérpretes [4,6] y sobrecarga cognitiva en programación [11], que orientaron la selección de los ítems a evaluar: comprensión teórica, capacidad de aplicación, percepción de las herramientas, disponibilidad de recursos y valoración de alternativas interactivas. El microcurrículo oficial de la asignatura, por su parte, permitió anclar las preguntas a los resultados de aprendizaje e indicadores de logro concretos de FLP, evitando que el instrumento midiera percepciones genéricas desvinculadas del contexto específico de la asignatura.

La encuesta a estudiantes fue diseñada con siete ítems de escala Likert de cinco niveles (1: Totalmente en desacuerdo, 5: Totalmente de acuerdo) para evaluar la percepción de las dificultades conceptuales en FLP, junto con una pregunta abierta orientada a identificar las recomendaciones de mejora. Esta estructura permitió distinguir entre dificultades atribuibles al contenido, al entorno de programación y a la escasez de recursos complementarios.

El diseño de la encuesta a docentes incluyó una pregunta de segmentación según el semestre en el que cada docente dicta programación principalmente. A esta le siguieron bloques temáticos con ítems Likert sobre la percepción de las dificultades estudiantiles en distintos paradigmas de programación, y una pregunta abierta final para comentarios o sugerencias sobre el proceso de enseñanza de programación.

Finalmente, la revisión del microcurrículo se realizó sobre el documento oficial de la asignatura FLP (código 750017C), el cual se encuentra en el Anexo A. Este documento estructura los contenidos a través de competencias específicas, resultados de aprendizaje (R.A.) e indicadores de logro (IL), lo que permitió contrastar las dificultades reportadas por estudiantes y docentes con la estructura formal de los contenidos e identificar cuáles indicadores concentran mayor complejidad conceptual.

3.1.2. Selección de la población

La población objetivo de la encuesta estudiantil fueron estudiantes que hubieran cursado FLP en la Universidad del Valle sede Tuluá durante los últimos tres años académicos. La convocatoria no se restringió al período en curso, con el objetivo de ampliar el tamaño de la muestra y capturar perspectivas tanto recientes como retrospectivas, lo que permitió que el diagnóstico reflejara una experiencia acumulada con la asignatura. La encuesta fue aplicada de forma voluntaria, obteniendo 36 respuestas, las cuales se encuentran detalladas en el Anexo B.

La población objetivo de la encuesta docente fueron los profesores del programa de Ingeniería de Sistemas de la Universidad del Valle sede Tuluá que dictan asignaturas de programación, independientemente del semestre. Esta decisión buscó obtener una perspectiva longitudinal del programa, considerando que mientras los docentes de semestres avanzados informan sobre las dificultades que observan directamente en FLP y asignaturas afines, los docentes de primeros semestres permiten caracterizar las bases con las que los estudiantes ingresan. La encuesta fue aplicada de forma voluntaria, obteniendo 5 respuestas, las cuales se encuentran detalladas en el Anexo C.

3.2. Análisis de la encuesta a estudiantes

3.2.1. Resultados cuantitativos

La Tabla 3.1 presenta la distribución de respuestas para los siete ítems Likert de la encuesta aplicada a los 36 estudiantes.

Tabla 3.1: Distribución de respuestas - encuesta a estudiantes de FLP ($n = 36$)

Ítem	Tot. desac.	En desac.	Neutral	De acuerdo	Tot. acuerdo	% Acuerdo	% Desacuerdo
P1. Comprendí adecuadamente los conceptos teóricos	8.3 %	19.4 %	27.8 %	36.1 %	8.3 %	**44.4 %**	27.8 %
P2. Las clases magistrales fueron útiles	0.0 %	8.3 %	36.1 %	33.3 %	22.2 %	**55.6 %**	8.3 %
P3. Me siento preparado para aplicar los conceptos	8.3 %	36.1 %	36.1 %	19.4 %	0.0 %	**19.4 %**	44.4 %
P4. Tuve dificultades con conceptos abstractos	2.8 %	13.9 %	5.6 %	52.8 %	25.0 %	**77.8 %**	16.7 %
P5. Racket y EOPL facilitaron la comprensión	13.9 %	25.0 %	41.7 %	19.4 %	0.0 %	**19.4 %**	38.9 %
P6. Fue fácil encontrar recursos externos	16.7 %	44.4 %	22.2 %	5.6 %	11.1 %	**16.7 %**	61.1 %
P7. Recursos adicionales habrían mejorado mi aprendizaje	0.0 %	2.8 %	2.8 %	38.9 %	55.6 %	**94.5 %**	2.8 %

Fuente: elaboración propia.

Los resultados permiten identificar cuatro hallazgos clave:

Brecha entre comprensión teórica y aplicación práctica : Aunque el 44.4 % de los estudiantes se mostraron de acuerdo en haber comprendido adecuadamente los conceptos teóricos (P1), solo el 19.4 % se sintió preparado para aplicarlos en otros cursos o proyectos (P3). Esta brecha revela que la comprensión teórica de los contenidos no se traduce necesariamente en una comprensión funcional que permita su uso autónomo. Adicionalmente, el porcentaje de respuestas neutrales en P3 (36.1 %) es significativamente mayor que en P1 (27.8 %), lo que sugiere que muchos estudiantes no tienen una opinión clara sobre su preparación para aplicar los conceptos, lo que refuerza la idea de una brecha entre teoría y práctica.

Alta incidencia de dificultades con conceptos abstractos : El ítem P4 concentra el mayor porcentaje de acuerdo de toda la encuesta: el 77.8 % de los participantes reportó haber tenido dificultades para comprender conceptos como entornos, estado o secuenciación. Este dato es especialmente significativo porque estos conceptos corresponden directamente al segundo resultado de aprendizaje (R.A.2) del microcurrículo de FLP, el cual concentra el 50.0 % del total de la nota de la asignatura.

Percepción negativa o neutra de Racket y EOPL como herramientas de aprendizaje : El ítem P5 muestra que solo el 19.4 % de los estudiantes consideró que Racket y EOPL facilitaron su comprensión, mientras que el 38.9 % estuvo en desacuerdo y el 41.7 % restante se mantuvo neutral. Este resultado sugiere que, para la mayoría de los participantes, las herramientas actuales no representan un apoyo claro al aprendizaje. Este hallazgo se alinea con la teoría de la carga cognitiva de Sweller, según la cual la sintaxis de Racket actúa como una carga extrínseca que satura la memoria de trabajo [9] y obstaculiza la comprensión semántica al obligar al estudiante a agotar sus recursos mentales en el descifrado visual del código [10].

Escasez de recursos externos y necesidad de herramientas adicionales : El 61.1 % de los estudiantes indicó que no fue fácil encontrar recursos externos de apoyo (P6), lo que apunta a un vacío en la disponibilidad de material complementario específico para los contenidos de la asignatura. Asimismo, el 94.5 % de los encuestados consideró que recursos adicionales habrían mejorado su aprendizaje de manera significativa (P7), constituyendo el dato de mayor consenso en toda la encuesta.

3.2.2. Resultados cualitativos

Del análisis temático de las recomendaciones proporcionadas por los estudiantes emergen cuatro categorías recurrentes:

Necesidad de experimentación con retroalimentación inmediata. Varios participantes señalaron que el principal obstáculo no es la complejidad conceptual en sí, sino la imposibilidad de explorar los conceptos en tiempo real con acompañamiento. Un estudiante sintetizó esta situación con precisión: *"me gustaría poder tener al profesor en vivo para una pregunta, porque las dudas surgen a medida que uno va haciendo y practicando [...] me quedó el vacío de un diagrama de ambientes"*. Otro participante expresó la misma necesidad de forma más directa: *"crear espacios donde los estudiantes puedan experimentar con Racket y la librería EOPL en tiempo real, con retroalimentación inmediata"*.

Carga cognitiva de Racket como obstáculo al aprendizaje semántico. La fricción generada por la sintaxis de Racket fue identificada de forma explícita por varios estudiantes como un factor que desplaza el foco del aprendizaje. El testimonio más detallado al respecto señala: *"el principal obstáculo del curso es la carga cognitiva que impone Racket. [...] Los nombres de funciones internas (como `car`, `cdr` o `map`) y la estructura visual plana terminan desplazando el foco de la semántica de lenguajes hacia la resolución de errores sintácticos"*.

Limitaciones de la metodología de aula invertida. Cuatro estudiantes mencionaron explícitamente la metodología de aula invertida como un factor de dificultad. Sus comentarios señalan que este modelo, si bien puede ser efectivo en otros contextos, resulta especialmente problemático para una asignatura con alto nivel de abstracción, donde la orientación directa del docente durante la exposición inicial de los conceptos es percibida como necesaria: *"con aula invertida es muy difícil entender"*; *"mayor explicación, resolver ejercicios desde cero con estudiantes"*.

Complejidad acumulada en los temas del último corte. Dos participantes señalaron que los temas de mayor dificultad se concentran en el tramo final del semestre, los cuales están relacionados con estado, paso de parámetros y tipos. Además, el ritmo de avance no permite profundizar en ellos adecuadamente: *"tomar más tiempo para la enseñanza de los temas más complejos que se ven en el último corte"*.

3.2.3. Contraste con un diagnóstico previo de la asignatura

Los hallazgos anteriores se contrastaron con el diagnóstico realizado por Henao Gallego en la misma asignatura y sede, mediante una encuesta aplicada a 21 estudiantes que cursaron FLP durante el período 2025-1 [21]. Dado que dicho instrumento se diseñó con un propósito distinto, orientado a

la recomendación de materiales, la comparación se limita a los aspectos que ambos diagnósticos abordan en común.

Ambos diagnósticos coinciden en la alta dificultad percibida de la asignatura. En el estudio de Henao Gallego, el 71.4 % de los estudiantes calificó el curso como difícil o muy difícil, y el principal factor de dificultad reportado fue la complejidad de los temas, por encima de la falta de tiempo o de motivación. Este resultado es consistente con el 77.8 % de estudiantes que, en el presente diagnóstico, reportó dificultades con conceptos abstractos (P4). Aunque en aquel estudio solo un participante mencionó de forma explícita el lenguaje Racket como factor de dificultad, su testimonio es coherente con los señalamientos cualitativos sobre la carga cognitiva de Racket recogidos en el presente diagnóstico. Ambos estudios coinciden, además, en la valoración de recursos complementarios, dado que el 76.2 % de los participantes de aquel estudio manifestó disposición a utilizar una herramienta especializada para la asignatura, lo que concuerda con el 94.5 % de acuerdo obtenido en el ítem P7.

En cuanto a los temas específicos, entre los más señalados como difíciles en aquel diagnóstico figuran la construcción de un intérprete a partir de una gramática y la construcción recursiva de datos con gramáticas BNF (7 menciones cada uno), así como las nociones de ligadura y alcance de variables y el estado y la secuenciación en lenguajes imperativos (6 menciones cada uno), contenidos que corresponden a los indicadores de logro seleccionados como base del prototipo. No obstante, los temas con mayor número de menciones fueron los lenguajes orientados a objetos (11) y el paso de parámetros (9), mientras que los ambientes, como tema independiente, recibieron una sola mención. Esta diferencia puede atribuirse en parte al formato de las preguntas, ya que aquel instrumento presentaba los ambientes como una opción separada de la ligadura, el estado y la construcción de intérpretes, contenidos que en la práctica se implementan a través de ellos. Aun así, el resultado indica que las dificultades de la asignatura no se limitan a los conceptos representados en el prototipo.

Finalmente, en aquel estudio solo el 14.3 % de los estudiantes señaló los recursos interactivos, como plataformas o simuladores, entre los más útiles para comprender los contenidos, frente al 85.7 % que señaló los videos. Este dato no permite concluir una baja valoración de este tipo de recursos, dado que, como se evidenció en el ítem P6, el 61.1 % de los estudiantes encuestados en el presente diagnóstico manifestó dificultades para encontrar recursos externos de apoyo, lo que sugiere una escasa disponibilidad de herramientas interactivas específicas para los contenidos de FLP.

3.3. Análisis de la encuesta a docentes

La encuesta fue respondida por cinco docentes del programa, identificados como D1 a D5. El instrumento segmentó los bloques temáticos, permitiendo que los cinco diligenciaran el bloque de percepción general; D1 y D5 respondieron lo relacionado con la programación imperativa y la programación orientada a objetos, teniendo en cuenta que dictan en primero y segundo semestre. Por otro lado, D2, D3 y D4 respondieron el bloque de programación lógica y paradigmas avanzados, lo que corresponde a quinto semestre en adelante. Si bien el perfil directamente relacionado con FLP es el de los docentes de semestres avanzados, las respuestas de D1 y D5 permiten caracterizar las bases con las que los estudiantes llegan a la asignatura, lo que es un factor determinante en su capacidad de asimilar contenidos de alta abstracción [11].

3.3.1. Percepción general de los estudiantes

Este bloque obtuvo la respuesta de los cinco docentes y permitió construir un diagnóstico transversal del programa. La Tabla 3.2 presenta los resultados, agrupando primero a los docentes de semestres avanzados (D2, D3, D4) y luego a los de primeros semestres (D1, D5), para facilitar la comparación por grupo.

Tabla 3.2: Percepción general de los docentes sobre habilidades de los estudiantes ($n = 5$)

Afirmación	**D2 (5°+)**	**D3 (5°+)**	**D4 (5°+)**	**D1 (1°-2°)**	**D5 (1°-2°)**
Demuestran habilidades básicas de pensamiento lógico	De acuerdo	Tot. acuerdo	Tot. acuerdo	Neutral	En desacuerdo
Tienen dificultades para analizar un problema antes de programar	En desacuerdo	De acuerdo	De acuerdo	En desacuerdo	Neutral
Logran depurar errores de manera efectiva	De acuerdo	De acuerdo	Neutral	En desacuerdo	Neutral
Pueden dividir problemas complejos en partes más pequeñas	Tot. acuerdo	Neutral	En desacuerdo	En desacuerdo	Neutral
Comprenden la importancia de la abstracción	Tot. acuerdo	Tot. acuerdo	En desacuerdo	En desacuerdo	Neutral

Fuente: elaboración propia.

La lectura de la Tabla 3.2 revela una brecha de percepción entre grupos. Los docentes de semestres avanzados tienden a valorar positivamente las habilidades de sus estudiantes más que los de primeros semestres, lo cual es un resultado esperable dado que los estudiantes de semestres superiores han superado los filtros académicos iniciales. D1 se muestra consistentemente en desacuerdo en cuatro de los cinco ítems, mientras que D5 se mantiene neutral, ambos describiendo un perfil de estudiante de primeros semestres con habilidades lógicas y de abstracción mínimas.

Además, se observa una variedad de respuestas dentro del grupo avanzado: mientras que D2 y D3 coinciden en que los estudiantes de semestres superiores comprenden la importancia de la abstracción, D4 está en desacuerdo con esta afirmación. Lo mismo ocurre con la capacidad de dividir problemas complejos, donde D2 responde “Totalmente de acuerdo” y D4 “En desacuerdo”. Estas diferencias dentro del mismo grupo de docentes indican que las habilidades de abstracción y descomposición de problemas no están consolidadas de manera uniforme en los estudiantes.

Esta observación es coherente con lo expuesto por Zagami, quien señala que los estudiantes enfrentan una alta demanda cognitiva al intentar procesar conceptos abstractos simultáneamente con habilidades prácticas; cuando la capacidad de la memoria de trabajo se ve excedida por esta multiplicidad de elementos, se generan bloqueos y sentimientos de frustración en el aprendiz [11]. Asimismo, se reconoce que la naturaleza intrínsecamente abstracta de los algoritmos y construcciones teóricas en esta disciplina representa un desafío que dificulta la comprensión intuitiva [6].

3.3.2. Dificultades con paradigmas avanzados

Los docentes D2, D3 y D4 respondieron el bloque sobre los paradigmas directamente relacionados con los contenidos de FLP. La Tabla 3.3 muestra sus respuestas, evidenciando una percepción más positiva en D2 y D3, mientras que D4 se muestra más crítico, especialmente en relación con la capacidad de los estudiantes para seleccionar el paradigma adecuado para resolver un problema.

Tabla 3.3: Percepción de los docentes de semestres avanzados sobre programación lógica y paradigmas avanzados ($n = 3$)

Afirmación	**D2 (5°+)**	**D3 (5°+)**	**D4 (5°+)**
Comprenden los conceptos fundamentales de programación lógica	Tot. acuerdo	Tot. acuerdo	Tot. acuerdo
Tienen dificultades con el razonamiento lógico aplicado a la programación	De acuerdo	De acuerdo	En desacuerdo
Logran modelar problemas usando reglas lógicas	Tot. acuerdo	Neutral	De acuerdo
Comprenden la diferencia entre los distintos paradigmas	Tot. acuerdo	De acuerdo	En desacuerdo
Pueden seleccionar el paradigma adecuado para resolver un problema	De acuerdo	De acuerdo	Tot. en desacuerdo

Fuente: elaboración propia.

El único ítem en el que los tres docentes coinciden es la comprensión de los conceptos fundamentales de programación lógica, donde todos respondieron "Totalmente de acuerdo". Sin embargo, esta percepción contrasta con el resto de las respuestas, lo que sugiere que saber qué es la programación lógica no implica saber aplicarla para resolver problemas.

Las opiniones también difieren respecto al razonamiento lógico aplicado a la programación. D2 y D3 consideran que los estudiantes presentan dificultades en este aspecto, mientras que D4 no lo percibe así. Estas diferencias pueden estar relacionadas con el contexto y el tipo de actividades realizadas, ya que en cursos más guiados algunas dificultades pueden pasar desapercibidas, pero en asignaturas como FLP, donde los estudiantes deben construir soluciones con mayor autonomía, estas limitaciones resultan más evidentes.

Asimismo, la capacidad de comprender las diferencias entre paradigmas y seleccionar el adecuado para un problema concentra la mayor dispersión del bloque, debido a que D2 y D3 mantienen una percepción positiva, mientras que D4 responde, respectivamente, "En desacuerdo" y "Totalmente en desacuerdo". Esta percepción negativa demuestra el salto cognitivo que representa abandonar el paradigma imperativo para adoptar el paradigma funcional [23], dificultad que se agrava cuando el lenguaje utilizado impone una carga sintáctica adicional.

3.4. Revisión del microcurrículo

El microcurrículo de la asignatura FLP organiza sus contenidos en torno a tres competencias específicas y una competencia genérica, articuladas a través de tres resultados de aprendizaje y veinte indicadores de logro distribuidos con pesos diferenciados en las actividades de evaluación.

El **Resultado de Aprendizaje 1 (R.A.1)**, con un peso del 40.0 % sobre la nota total, abarca la construcción de datos recursivos mediante gramáticas BNF, el manejo del cálculo lambda, alcance de variables, implementación de tipos abstractos de datos y la construcción de árboles de sintaxis abstracta. Este R.A. sienta las bases formales sobre las que se construye todo el trabajo posterior de la asignatura.

El **Resultado de Aprendizaje 2 (R.A.2)**, con el mayor peso de la asignatura (50.0 %), se centra en la implementación de lenguajes con ambientes de variables, el recorrido del AST, la construcción de intérpretes básicos y la implementación de conceptos como condicionales, clausuras, estado y

paso de parámetros. La profundidad de este resultado, sumado a su peso evaluativo, lo convierte en el eje central de la asignatura y en la fuente principal de dificultades reportadas por los estudiantes.

El **Resultado de Aprendizaje 3 (R.A.3)**, con un peso del 5.0 %, tiene un carácter más conceptual, puesto que se enfoca en que los estudiantes comprendan las diferencias entre compilación e interpretación identificando sus beneficios y limitaciones.

La **Competencia Genérica (C.G.2)**, con un peso del 5.0 %, integra varios indicadores del R.A.2 en la construcción de un intérprete completo evaluado en el proyecto final de la asignatura.

3.5. Selección de conceptos base para el prototipo

A partir de las dificultades reportadas por los estudiantes, las percepciones del cuerpo docente y la revisión del microcurrículo, se seleccionaron cinco indicadores de logro como base conceptual para la biblioteca de casos predefinidos del prototipo. El criterio de selección combinó tres factores: el nivel de dificultad reportado por los estudiantes, el peso evaluativo en el microcurrículo y la viabilidad de representación visual e interactiva en una herramienta web.

La Tabla 3.4 presenta los indicadores de logro seleccionados, junto con su relación con los resultados de aprendizaje y una justificación para su inclusión en el prototipo.

Tabla 3.4: Indicadores de logro seleccionados como base para el prototipo

ID	Indicador de logro	R.A.	Justificación
IL 1.1.6	Construye un árbol de sintaxis abstracta a partir de una representación concreta en una gramática BNF	R.A.1	El AST es la estructura central del proceso de interpretación. Los estudiantes reportan dificultades para visualizar cómo una cadena de texto se transforma en una representación jerárquica; su visualización dinámica es la funcionalidad más distintiva del prototipo.
IL 1.2.3	Construye intérpretes básicos para dos conjuntos de valores (valor expresado, valor denotado)	R.A.2	Corresponde al primer nivel de construcción de intérpretes, donde se establece la relación entre la gramática, el AST y la evaluación. Es el punto de entrada natural para comprender el proceso completo de interpretación.
IL 1.2.6	Implementa lenguajes de programación con el concepto de estado asociado al manejo de ambientes para variables en lenguajes imperativos	R.A.2	El concepto de estado y su representación mediante ambientes fue identificado como el de mayor dificultad por los estudiantes (77.8 % reportó dificultades con entornos y estado). La representación visual de cómo evoluciona el ambiente durante la ejecución es uno de los aportes centrales del prototipo.
IL 1.2.7	Comprende la diferencia entre ligadura y asignación de variables	R.A.2	La distinción entre ligadura y asignación es conceptualmente sutil y frecuentemente confundida. Su representación visual mediante ambientes permite hacer explícita la diferencia que el código por sí solo no comunica con claridad.
IL 1.3.1	Identifica los componentes necesarios de un intérprete y un compilador	R.A.3	Provee el marco conceptual global que da sentido al resto de los contenidos. Comprender la arquitectura de un intérprete permite al estudiante situar cada concepto específico dentro de un proceso coherente.

Fuente: elaboración propia a partir del microcurrículo de la asignatura (código 750017C).

Estos cinco indicadores cubren los dos ejes de dificultad identificados en el diagnóstico. Por un lado, se encuentra el eje sintáctico-estructural, representado por IL 1.1.6, correspondiente a la construcción del AST. Por otro lado, se encuentra el eje semántico-evaluativo, representado por IL 1.2.3, 1.2.6 y 1.2.7, referente a la construcción del intérprete y el manejo del ambiente. Ambos ejes se complementan con una dimensión conceptual-arquitectónica de carácter transversal (IL 1.3.1), que sitúa las dificultades específicas dentro de la estructura general de un intérprete y un compilador.

Capítulo 4

Diseño del prototipo FLP Viewer

El presente capítulo documenta los resultados correspondientes al diseño del prototipo web denominado **FLP Viewer**, cuyo nombre combina la sigla de la asignatura con el término en inglés *viewer* (visualizador), que alude al propósito central de la herramienta: permitir al estudiante observar y explorar el comportamiento de un intérprete durante su ejecución.

En primer lugar, se presenta la especificación de requisitos del sistema, derivada del diagnóstico realizado en el capítulo anterior y de los indicadores de logro seleccionados como base conceptual del prototipo. En segundo lugar, se describe la arquitectura adoptada, la selección y justificación de tecnologías, los prototipos de interfaz de usuario y las estrategias de visualización implementadas para apoyar la comprensión de los contenidos de la asignatura.

4.1. Especificación de requisitos del sistema

4.1.1. Enfoque metodológico

La identificación y documentación de los requisitos del sistema se realizó siguiendo las directrices del estándar IEEE 830 [24] para la especificación de requisitos de software, adaptado al contexto educativo del proyecto. Los requisitos fueron derivados a partir de tres fuentes complementarias: en primer lugar, las dificultades conceptuales identificadas en el diagnóstico con estudiantes y docentes (Capítulo 3); en segundo lugar, los indicadores de logro del microcurrículo de la asignatura (Tabla 3.4); y en tercer lugar, las características de calidad de software definidas en la norma ISO/IEC 25010 [25], que orientó la formulación de los requisitos no funcionales.

Cada requisito fue clasificado por prioridad (Alta, Media) en función de su relevancia para los objetivos de aprendizaje identificados y de su viabilidad de implementación en el alcance del proyecto. Los criterios de aceptación asociados a cada requisito fueron formulados de manera verificable, haciendo referencia al comportamiento concreto del sistema implementado. La especificación completa se encuentra en el Anexo E.

4.1.2. Requisitos funcionales

Se definieron ocho requisitos funcionales que cubren las capacidades esenciales del prototipo. La Tabla 4.1 presenta el resumen de los mismos con su prioridad y una descripción concisa.

Tabla 4.1: Resumen de requisitos funcionales del sistema

ID	Nombre	Descripción resumida	Prioridad
RF-01	Generación del AST	Procesar el programa del usuario con la gramática definida y producir el AST correspondiente.	Alta
RF-02	Visualización del AST	Renderizar el AST generado de forma jerárquica e interactiva en un panel dedicado.	Alta
RF-03	Ejecución y evaluación	Invocar el intérprete Racket sobre los archivos del editor y mostrar el resultado en la consola.	Alta
RF-04	Representación del ambiente	Mostrar el ambiente de evaluación como estructura jerárquica de marcos actualizada tras cada ejecución.	Alta
RF-05	Biblioteca de ejemplos	Ofrecer un conjunto de programas predefinidos alineados con los indicadores de logro de la asignatura.	Media
RF-06	Validación de sintaxis Racket	Detectar y reportar errores léxicos y sintácticos en la gramática BNF y en el programa del usuario.	Alta
RF-07	Visualización del intérprete	Presentar los archivos del intérprete generado (gramática, ambiente, evaluador) en un editor organizado por pestañas con secciones protegidas.	Media
RF-08	Plantilla base y exportación	Proveer una plantilla editable del intérprete con secciones bloqueadas y permitir la descarga del proyecto en formato `.zip`.	Alta

Fuente: elaboración propia. Especificación completa en Anexo E.

4.1.3. Requisitos no funcionales

Se definieron cinco requisitos no funcionales que establecen las condiciones de calidad del sistema. La Tabla 4.2 presenta el resumen correspondiente.

Tabla 4.2: Resumen de requisitos no funcionales del sistema

ID	Nombre	Descripción resumida	Prioridad
RNF-01	Usabilidad	La interfaz debe ser operada sin instrucción previa por estudiantes sin experiencia avanzada en compiladores.	Alta
RNF-02	Rendimiento	La generación del AST y la evaluación del programa deben completarse en menos de 5 segundos en condiciones normales.	Alta
RNF-03	Accesibilidad web	El sistema debe ejecutarse en navegadores modernos sin instalación adicional ni autenticación.	Alta
RNF-04	Mantenibilidad	El código debe estar organizado en módulos independientes con separación explícita de tipos, lógica y presentación.	Media
RNF-05	Escalabilidad	La arquitectura debe permitir incorporar nuevos ejemplos, gramáticas o características sin afectar las funcionalidades existentes.	Media

Fuente: elaboración propia. Especificación completa en Anexo E.

4.2. Diseño del prototipo

4.2.1. Arquitectura del sistema

La arquitectura del prototipo se estructuró en tres capas principales: infraestructura de despliegue, servidor y cliente. Esta separación de responsabilidades permite que el procesamiento computacionalmente intensivo (la ejecución de Racket) ocurra en el servidor, mientras que la interfaz de usuario opera íntegramente en el navegador sin necesidad de recargas de página [26].

Las decisiones que dieron lugar a esta arquitectura estuvieron guiadas por tres criterios principales: simplicidad operativa, aislamiento de la ejecución de código arbitrario y minimización de la latencia percibida por el usuario. A partir de estos criterios se definió la distribución de responsabilidades entre los componentes y el modelo de despliegue del sistema.

Con respecto a la simplicidad operativa, se optó por una arquitectura monolítica de servidor único en lugar de una arquitectura de microservicios. Fowler y Lewis señalan que los microservicios introducen una complejidad operativa significativa, relacionada con la coordinación entre servicios, la trazabilidad distribuida y la gestión de fallos, que solo se justifica cuando la escala o la independencia de despliegue de los componentes lo requieren [27]. Dado que el prototipo cuenta con un equipo de desarrollo reducido y un conjunto acotado de funcionalidades, la arquitectura monolítica resulta más adecuada para el alcance del proyecto.

Respecto al aislamiento de la ejecución de código arbitrario, la ejecución de Racket se delega a un subproceso del servidor de aplicación en lugar de separarse como un servicio independiente. Cada invocación se ejecuta en un proceso con un tiempo límite estricto y un directorio temporal propio. De esta manera, un programa con un bucle infinito o un comportamiento inesperado no compromete la estabilidad del servidor principal [28].

Finalmente, para reducir la latencia percibida por el usuario, el pipeline de transformación de gramática se ubicó íntegramente en el cliente. Esta decisión acerca el cómputo al punto de generación de los datos y elimina la latencia de red asociada a las validaciones inmediatas [26]. Además, esta estrategia es viable porque la carga computacional del pipeline es baja y no requiere acceso a recursos

del servidor.

4.2.1.1. Infraestructura de despliegue

La infraestructura de despliegue se compone de dos elementos. El primero es un *reverse proxy*[1], encargado de recibir las solicitudes entrantes y redirigirlas al servidor de aplicación, centralizando así el punto de entrada al sistema. El segundo es el propio servidor de aplicación, que incorpora el intérprete de Racket como dependencia del entorno de ejecución. Esta configuración garantiza que el prototipo sea reproducible independientemente del sistema operativo del servidor anfitrión. La selección de las tecnologías concretas que materializan esta estructura se detalla en la sección 4.2.2.

4.2.1.2. Capa de servidor

La capa de servidor asume dos responsabilidades principales: el renderizado inicial de la interfaz y la ejecución del intérprete. Durante el renderizado inicial, los datos estáticos del sistema, como los ejemplos predefinidos y los contenidos de ayuda, se cargan desde el sistema de archivos y se transmiten directamente a la capa cliente. De esta manera, se evita la necesidad de realizar peticiones adicionales una vez cargada la página.

La segunda responsabilidad corresponde a la ejecución de programas en Racket. Para ello, el servidor expone un punto de acceso interno que recibe los archivos generados por el editor y la expresión a evaluar. A continuación, lanza el proceso de Racket de forma aislada, aplicando un tiempo límite configurable para su ejecución, y devuelve al cliente el resultado estructurado como una secuencia de pasos.

4.2.1.3. Capa de cliente

La capa cliente se diseñó alrededor de un componente orquestador central encargado de mantener el estado global de la sesión de trabajo y coordinar la comunicación entre los distintos paneles de la interfaz. Este componente centraliza las transiciones de estado, incluyendo el inicio y fin de sesión, la recepción de resultados y la navegación paso a paso por las evaluaciones. Gracias a ello, cada panel de visualización puede implementarse como un componente *stateless*[2] que recibe únicamente la información necesaria para su renderizado. La Figura 4.1 resume la distribución de estas tres capas y la comunicación entre sus componentes.

[1] Un *reverse proxy* es un servidor interpuesto entre el cliente y el servidor de aplicación que recibe las solicitudes entrantes y las reenvía al servicio interno correspondiente, ocultando los detalles de la infraestructura *backend* y permitiendo aplicar reglas de seguridad, balanceo de carga o compresión de manera centralizada.

[2] Un componente *stateless* (sin estado) es aquel que no almacena información propia entre *renders*; su salida depende exclusivamente de los datos que recibe como entrada, lo que facilita su prueba y reutilización.

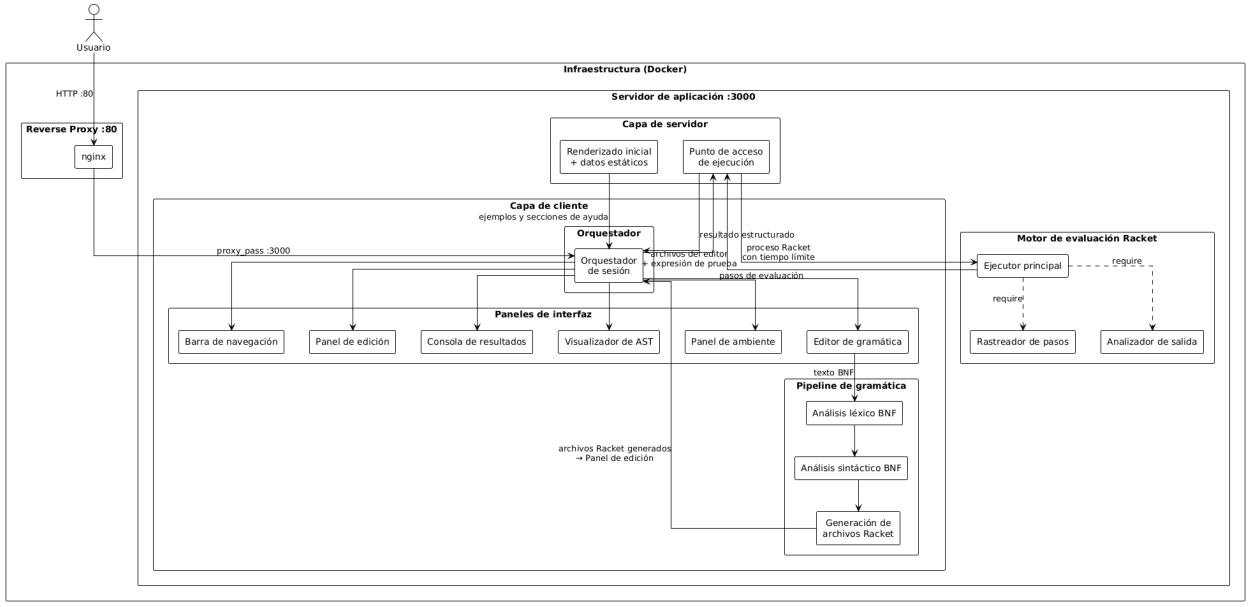


Figura 4.1: Diagrama de arquitectura del sistema
Fuente: Elaboración propia

4.2.1.4. Pipeline de gramática

Dentro de la arquitectura cliente, el pipeline de transformación de gramática constituye un subsistema encargado de procesar las especificaciones definidas por el estudiante sin intervención del servidor. Se diseñó como una cadena de etapas independientes que comprende el análisis léxico de la notación BNF, el análisis sintáctico para la construcción de una representación intermedia de la gramática y la generación de los archivos del intérprete a partir de dicha representación.

La separación en etapas independientes permite identificar y reportar errores de forma precisa en el nivel donde se originan (léxico o sintáctico) y favorece una organización modular del proceso de transformación [8]. Además, esta estructura facilita la incorporación de nuevas etapas o transformaciones sin alterar significativamente los componentes existentes.

Al ejecutarse íntegramente en el cliente, el pipeline proporciona retroalimentación inmediata sobre errores en la gramática sin depender de intercambios con el servidor. Esta característica reduce el tiempo de espera entre la acción del estudiante y la respuesta del sistema, favorece la continuidad de la tarea y contribuye a disminuir la carga cognitiva extrínseca [9].

4.2.2. Selección de tecnologías

La selección de tecnologías se realizó evaluando cuatro criterios. Se consideraron la curva de aprendizaje del equipo de desarrollo, la disponibilidad de documentación, la viabilidad de implementación de las funcionalidades requeridas y la compatibilidad entre las tecnologías seleccionadas. La Tabla 4.3 presenta la justificación de cada elección.

Tabla 4.3: Tecnologías seleccionadas y justificación

Tecnología	Versión	Justificación
Next.js [29]	16.2.6	Framework basado en React que integra renderizado en servidor y rutas de API en una única plataforma. Su adopción permite implementar la arquitectura propuesta sin requerir servicios *backend* adicionales.
React [30]	19.2	Biblioteca para la construcción de interfaces de usuario basada en componentes. Su modelo declarativo facilita la implementación de la interfaz interactiva y la separación de responsabilidades entre los distintos paneles del sistema.
TypeScript [31]	5.9	Superset tipado de JavaScript que permite definir contratos explícitos entre módulos mediante interfaces y tipos. Su utilización contribuye a reducir errores durante el desarrollo y a mejorar la mantenibilidad del código.
Monaco Editor [32]	0.55	Componente de edición utilizado por Visual Studio Code y disponible como librería independiente. Proporciona las capacidades necesarias para implementar el editor interactivo, incluyendo la protección de regiones de código generadas automáticamente.
Tailwind CSS [33]	4	Framework de estilos utilitarios que facilita la construcción de interfaces consistentes mediante clases reutilizables. Su enfoque permite mantener los estilos próximos a los componentes que los utilizan y simplifica el desarrollo de la capa de presentación.
Racket [12]	9.x	Lenguaje utilizado en la asignatura FLP y plataforma de ejecución de la librería EOPL. Su adopción garantiza la compatibilidad entre el código generado por el prototipo y el entorno académico empleado por los estudiantes.
Docker [34]	—	Plataforma de contenedores utilizada para encapsular el entorno de ejecución. Permite garantizar la reproducibilidad del despliegue e incorporar Racket como dependencia sin requerir su instalación en el sistema anfitrión.
pnpm [35]	10	Gestor de paquetes para el ecosistema JavaScript con soporte de almacenamiento eficiente de dependencias. Su utilización reduce los tiempos de instalación y construcción del proyecto en el entorno de despliegue.

Fuente: elaboración propia.

La selección realizada refleja las decisiones arquitectónicas descritas en la sección anterior. Next.js, React y TypeScript conforman la base de la capa cliente y del servidor de aplicación, mientras que Monaco Editor proporciona las capacidades de edición requeridas por el entorno interactivo. Por su parte, Racket constituye el motor de evaluación del prototipo y Docker garantiza la reproducibilidad de su entorno de ejecución, encapsulando sus dependencias de manera independiente de la plataforma de despliegue empleada.

4.2.3. Prototipos de interfaz de usuario

La distribución de la interfaz se diseñó con base en los principios de economía de la atención [16] y en los hallazgos del diagnóstico, que evidenciaron que la fricción visual de las herramientas actuales desplaza la atención del estudiante desde la semántica del lenguaje hacia la resolución de errores sintácticos. Para mitigar este efecto, la interfaz se organizó de manera que las tareas principales del estudiante, que incluyen la definición de la gramática, la edición del intérprete, la ejecución de

programas y el análisis de resultados, siguieran un recorrido visual coherente dentro de una única pantalla, minimizando los cambios de contexto.

La literatura sobre herramientas educativas para la enseñanza de compiladores e intérpretes destaca la importancia de presentar simultáneamente múltiples representaciones del proceso de ejecución, permitiendo que el estudiante relacione el código fuente con las estructuras internas generadas durante el procesamiento [3,4,6]. Asimismo, los principios de carga cognitiva sugieren que la integración de información relevante dentro de un mismo espacio de trabajo reduce el esfuerzo necesario para coordinar fuentes dispersas de información [9]. Por esta razón, se optó por una interfaz multipanel en la que las representaciones principales del proceso de interpretación permanecen visibles de manera simultánea.

La interfaz principal se estructura en cinco zonas funcionales, cuya distribución se presenta en el *wireframe*[3] de la Figura 4.2. La organización propuesta define una configuración inicial de los paneles, permitiendo al estudiante modificar dinámicamente el espacio asignado a cada zona durante la interacción. De esta manera, es posible redistribuir el área disponible entre el editor, las visualizaciones del AST y del ambiente, y la consola de salida según las necesidades de la tarea en curso. Esta flexibilidad permite priorizar las representaciones más relevantes en cada momento, manteniendo una visión integrada del proceso de interpretación y contribuyendo a la usabilidad del sistema [5].

1. **Barra de navegación** (zona superior): concentra el acceso a las funciones globales del sistema, incluyendo la biblioteca de ejemplos, el editor de gramática y los controles asociados a la sesión de ejecución.
2. **Panel de edición** (zona izquierda): alberga el editor de código organizado en múltiples pestañas, desde donde el estudiante interactúa con los distintos archivos que conforman el intérprete.
3. **Panel del AST** (zona derecha superior): presenta la representación jerárquica del árbol de sintaxis abstracta generado durante la evaluación de expresiones.
4. **Panel de ambiente** (zona derecha inferior): visualiza el estado del ambiente de ejecución mediante una estructura de marcos anidados que muestra las ligaduras activas en cada nivel de alcance.
5. **Panel de consola** (zona inferior): concentra la interacción textual con el intérprete, permitiendo introducir expresiones y consultar los resultados producidos durante la ejecución.

[3] Un *wireframe* es un prototipo de baja fidelidad utilizado en el diseño de interfaces para representar la estructura, distribución espacial y flujo de interacción de una aplicación, sin incorporar elementos visuales finales ni detalles de implementación.

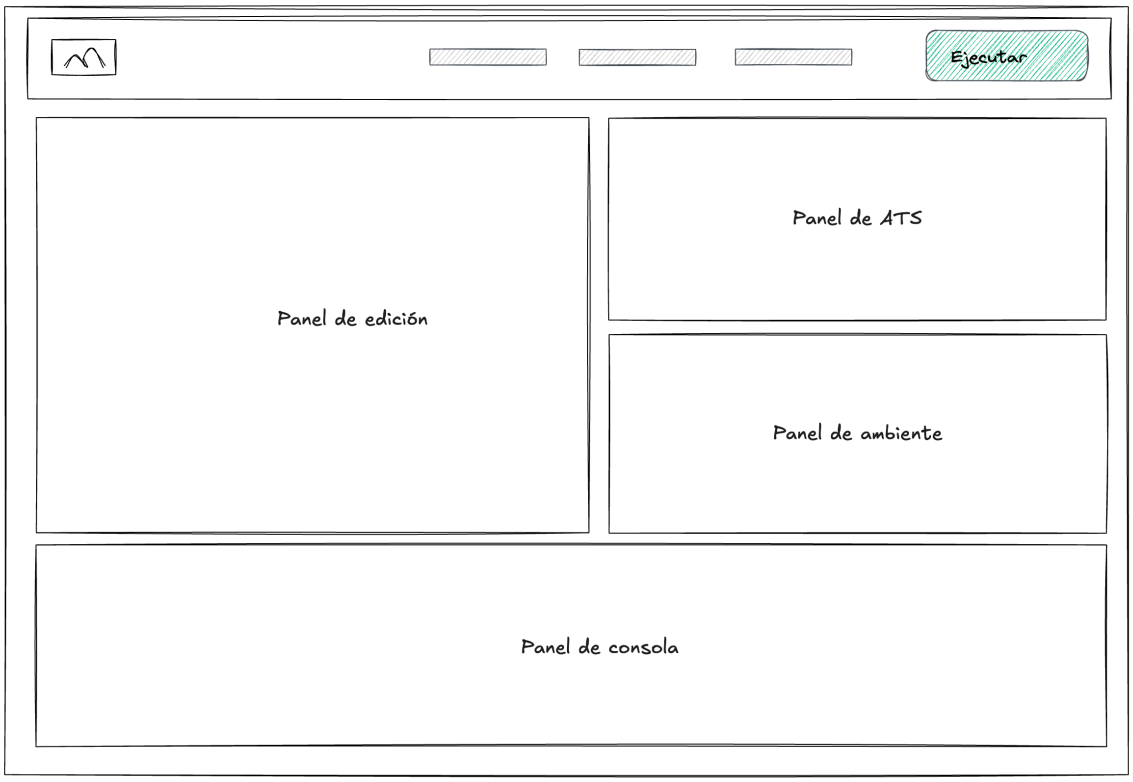


Figura 4.2: Wireframe de la interfaz principal del prototipo FLP Viewer
Fuente: Elaboración propia

Adicionalmente, se diseñó un modal de gramática (Figura 4.3) que permite al estudiante ingresar la especificación léxica y la gramática en notación BNF en dos áreas de texto diferenciadas. Al confirmar la definición, el sistema genera los archivos del intérprete y los carga en el panel de edición. Este flujo de interacción mantiene la gramática y el intérprete en un vínculo explícito y visible para el estudiante, de forma que la relación entre la especificación formal y el código generado pueda observarse directamente. Esta decisión responde a la necesidad de hacer visibles procesos que normalmente permanecen ocultos en los entornos de desarrollo convencionales, una estrategia identificada como beneficiosa para la enseñanza de compiladores e intérpretes [3, 6].



Figura 4.3: Wireframe del modal de definición de gramática BNF
Fuente: Elaboración propia

4.2.4. Estrategias de visualización

El diseño de las estrategias de visualización responde directamente a los dos ejes de dificultad identificados en el diagnóstico: el eje sintáctico-estructural, asociado a la comprensión del AST, y el eje semántico-evaluativo, asociado al seguimiento del estado durante la ejecución [4]. En cada caso, las decisiones de diseño se orientaron a maximizar la correspondencia entre las representaciones visuales y los modelos conceptuales presentados en la asignatura.

El AST se renderiza como un árbol jerárquico interactivo en el que cada nodo muestra el nombre del tipo de producción que lo originó y, cuando corresponde, el valor léxico del token asociado. Esta representación hace visible la correspondencia entre la notación BNF definida por el estudiante y la estructura interna que el parser produce. Sorva et al. señalan que la efectividad de este tipo de herramientas está directamente relacionada con el grado en que la representación visual se corresponde con el modelo conceptual que el estudiante debe construir [13]. Para apoyar esa lectura, los nodos se diferencian visualmente según su categoría semántica, lo que es consistente con los hallazgos de Stamenkovic et al. [6] y Jovanovic et al. [4]. Asimismo, los nodos admiten expansión y colapso interactivo y, por defecto, solo el nivel raíz se expande al cargar un nuevo árbol, de modo que el estudiante parte de una vista de alto nivel y profundiza progresivamente según su interés, en coherencia con los principios de carga cognitiva [9].

El ambiente de ejecución se representa como una cadena de marcos enlazados, donde cada marco expone las ligaduras activas en ese nivel de alcance con su nombre, valor y tipo. La disposición hace visible la relación de extensión entre los ambientes, de modo que el estudiante puede seguir la cadena desde el ambiente inicial hasta el más reciente. Esta representación busca hacer observable

la diferencia entre ligadura y asignación (IL 1.2.7) y la evolución del ambiente con cada llamada a función (IL 1.2.6), abordando directamente la dificultad reportada por el 77.8 % de los estudiantes encuestados. La disposición en cadena sigue el modelo conceptual descrito por Friedman y Wand [7]. Cai et al. argumentan que los modelos de visualización basados en pila no capturan correctamente la estructura de los ambientes léxicos en lenguajes modernos y que se requieren representaciones que reflejen el encadenamiento real de los marcos [18]; la representación adoptada en FLP Viewer responde a esta misma consideración. Esta disposición favorece también la correspondencia entre la teoría presentada en clase y la visualización ofrecida por la herramienta, lo cual es relevante porque las representaciones que no coinciden con el modelo enseñado pueden inducir concepciones erróneas difíciles de corregir [14].

El modo de evaluación paso a paso permite que, cuando el estudiante envía varias expresiones de nivel superior al intérprete, pueda navegar por el resultado de cada una de forma individual en lugar de recibir únicamente el resultado de la última, como ocurre en el modo normal. Cada expresión se evalúa de manera aislada, sin conservar las ligaduras generadas por las expresiones anteriores, de modo que el AST y el ambiente mostrados en cada paso corresponden únicamente a la expresión activa. Este mecanismo retoma el principio de exposición incremental del estado de ejecución presente en herramientas como el *stepper* algebraico de Clements, Flatt y Felleisen [15], Python Tutor [14] y la máquina nocional CSE de Abad y Henz [20], aunque con una diferencia importante: dichas herramientas trazan la ejecución continua de un único programa, en la que el ambiente se acumula de una instrucción a la siguiente, mientras que FLP Viewer navega entre expresiones de nivel superior independientes entre sí. Desde la perspectiva de la carga cognitiva, la presentación controlada de un resultado a la vez reduce la presión sobre la memoria de trabajo [9].

Finalmente, la consola de salida diferencia mediante color y prefijo textual cinco categorías de mensaje: entrada del usuario, valor de retorno, información, advertencia y error. Esta distinción permite al estudiante identificar la naturaleza de cada mensaje sin necesidad de leer su contenido, reduciendo el esfuerzo de interpretación de la retroalimentación. La codificación cromática está respaldada por Hannebauer et al. [17], y la decisión de no exponer directamente la salida nativa de Racket responde a los principios de usabilidad en tecnología educativa que recomiendan facilitar la interpretación de la información relevante durante la interacción [5].

Capítulo 5

Implementación del prototipo FLP Viewer

El presente capítulo documenta los resultados correspondientes al desarrollo e implementación del prototipo FLP Viewer, en cumplimiento del objetivo específico 4 del proyecto. El capítulo se organiza en seis secciones. En primer lugar, se describe el repositorio de código fuente y su estructura de directorios. En segundo lugar, se detallan los componentes implementados, abordando el pipeline de gramática, la arquitectura del cliente, la biblioteca de ejemplos predefinidos, el punto de acceso de ejecución y la instrumentación en tiempo de ejecución. En tercer lugar, se presenta el flujo principal de ejecución a través de un diagrama de secuencia. En cuarto lugar, se describe la configuración de despliegue en el entorno de pruebas. En quinto lugar, se verifica el cumplimiento de los requisitos funcionales definidos en el diseño. Finalmente, se presenta la evaluación de la calidad del software mediante análisis estático del código fuente.

5.1. Repositorio de código fuente

El código fuente del prototipo se encuentra alojado en el repositorio público de GitHub disponible en `https://github.com/Esmeralda-RG/flp-viewer`. El repositorio registra el historial completo de *commits* del proceso de desarrollo, organizado en torno a las funcionalidades incrementales del sistema. El archivo `README.md` incluye la descripción del proyecto, el *stack* tecnológico utilizado, las instrucciones de instalación y ejecución local, la estrategia de integración con Racket y la descripción de la estructura de directorios.

La estructura del proyecto sigue la convención del *App Router*[1] de Next.js [29], con los siguientes directorios principales:

- `app/api/run/`: contiene la ruta de API encargada de gestionar la ejecución del intérprete Racket, junto con los archivos `.rkt` de instrumentación en tiempo de ejecución (`_runner.rkt`, `_tracking.rkt` y `_stream-parser.rkt`).

[1] El *App Router* es el sistema de enrutamiento de Next.js basado en el sistema de archivos introducido en la versión 13. Organiza las rutas, los componentes de servidor y la lógica de la API dentro del directorio `app/`, permitiendo mezclar componentes de servidor y de cliente en la misma aplicación.

- `app/components/`: agrupa los componentes de la interfaz de usuario, entre ellos el orquestador `PlaygroundLayout`, el visualizador de árbol `ASTViewer`, el panel de ambiente `Environment Panel`, la consola `ConsoleOutput`, el editor `EditorPanel` y los modales auxiliares.
- `app/lib/`: contiene los módulos del pipeline de gramática, los generadores de código Racket y las utilidades del playground.
- `app/services/`: incluye el cliente HTTP que comunica la capa cliente con la ruta de API.
- `app/types/`: centraliza las definiciones de tipos TypeScript compartidas entre módulos.
- `examples/`: almacena las plantillas de programas predefinidos cargadas en el renderizado inicial del servidor.

5.2. Implementación de los componentes

5.2.1. Pipeline de gramática BNF

El pipeline de gramática se implementó íntegramente en TypeScript [31] como una cadena de módulos independientes que opera en el cliente sin intervención del servidor. Su punto de entrada único es la función `runPipeline`, definida en `grammar-pipeline.ts`, que recibe la especificación léxica y la gramática en notación BNF y retorna los archivos del intérprete generados o una lista de errores.

La cadena de transformación sigue el modelo clásico de compilación [8] y comprende tres etapas secuenciales, ilustradas en la figura 5.1. La primera es el análisis léxico, implementado en `bnf-lexer.ts`: la función `tokenize` recorre el texto de entrada carácter a carácter y produce una secuencia de tokens tipados. La segunda es el análisis sintáctico, implementado en `bnf-parser.ts`: la función `parse` consume la secuencia de tokens y construye una representación intermedia de la gramática (`GrammarAST`) compuesta por reglas de producción. Los errores de ambas etapas se reportan mediante las clases `LexError` y `ParseError`, respectivamente, ambas con la línea y columna exactas del fallo. La tercera etapa es la generación de código, distribuida en tres módulos especializados que consumen el `GrammarAST` de forma independiente. `eopl-generator.ts` produce el archivo `grammar.rkt` con la especificación léxica y gramatical en formato SLLGEN. `env-generator.ts` genera `environment.rkt` con el ambiente inicial y las funciones de extensión. `main-generator.ts` produce `main.rkt` con los *stubs*[2] de las funciones de evaluación pendientes de implementación por parte del estudiante, registrando además las líneas bloqueadas para protección en el editor.

La especificación léxica se define en un área de texto independiente del modal de gramática, escribiendo un nombre de token por línea. Para cada nombre reconocido (`number`, `float`, `identifier`, `binary`, `octal`, `hex` y `text`) la función `expandLexRule` de `eopl-generator.ts` produce las reglas SLLGEN correspondientes. Las reglas de espaciado (`whitespace`) y de comentarios de línea (`comment`) se incluyen siempre de forma automática. Si el área léxica se deja vacía, el pipeline incorpora el conjunto completo de tokens por defecto. Para tipos léxicos no contemplados en los nombres predefinidos, el estudiante puede escribir directamente la regla en notación SLLGEN entre paréntesis, que el pipeline incorpora sin transformaciones.

[2] Un *stub* es una función con cuerpo incompleto que declara la firma esperada y sirve como marcador del trabajo pendiente de implementación. En este contexto, los *stubs* definen las funciones de evaluación del intérprete que el estudiante debe completar.

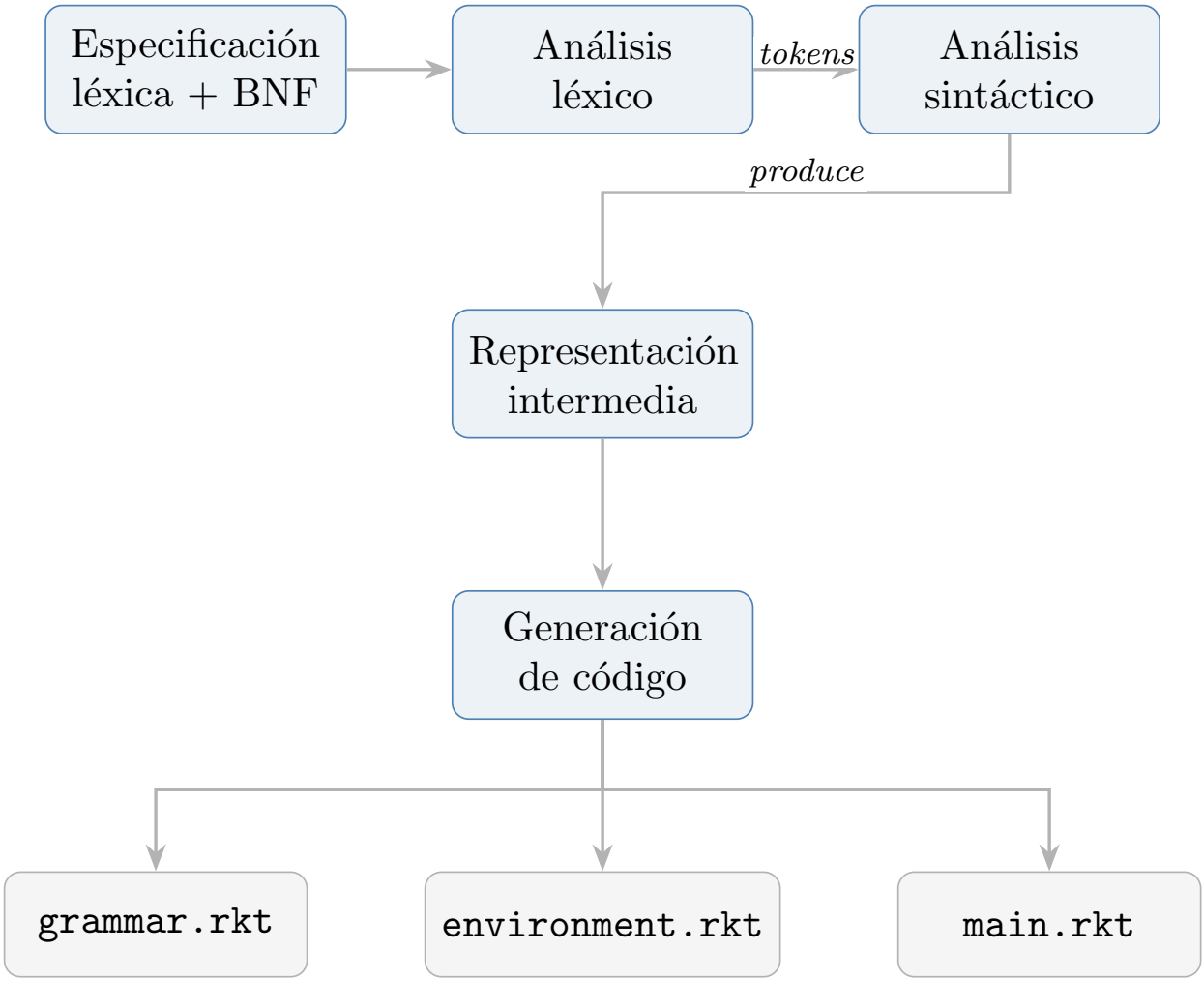


Figura 5.1: Pipeline de gramática BNF: de la especificación de entrada a los archivos Racket generados, pasando por el análisis léxico, el análisis sintáctico y la generación de código.
Fuente: Elaboración propia

5.2.2. Componente orquestador y paneles de la interfaz

La Figura 5.2 muestra el modal de bienvenida que se presenta al estudiante en el primer acceso. El modal ofrece un resumen de las acciones básicas y carga automáticamente el ejemplo *Hola Mundo* como punto de partida.

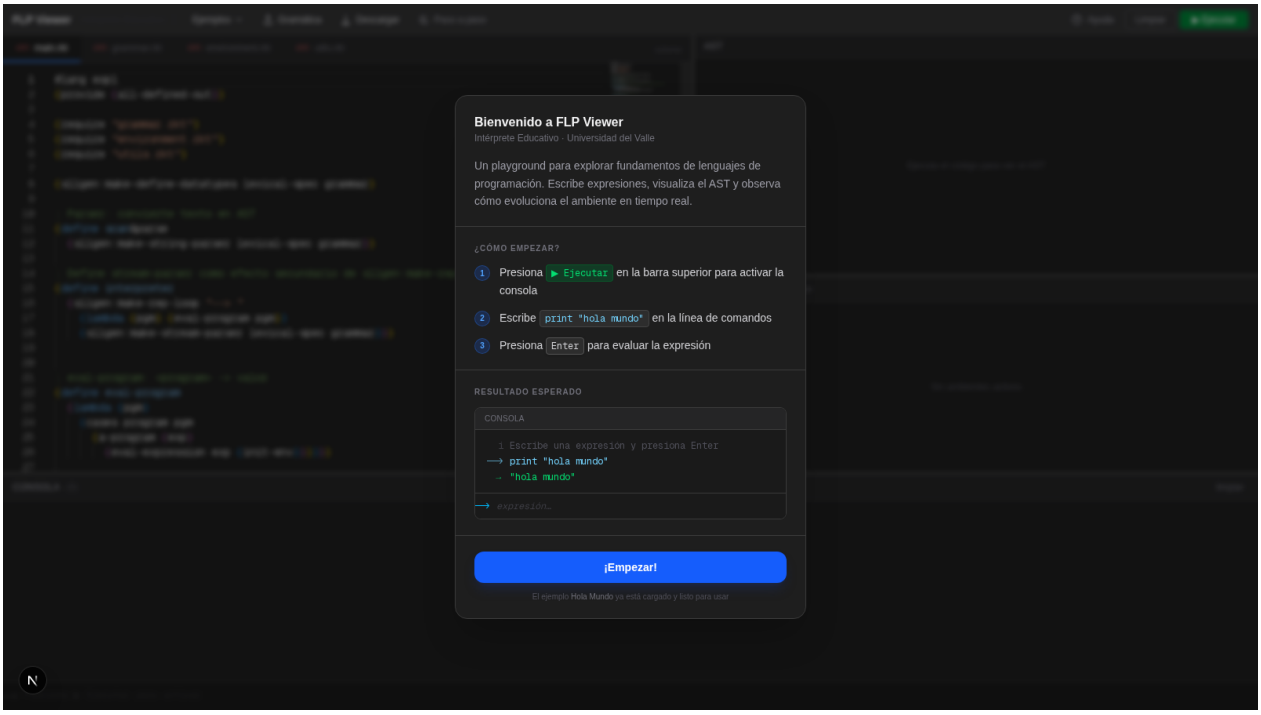


Figura 5.2: Modal de bienvenida del prototipo FLP Viewer
Fuente: Elaboración propia

El componente `PlaygroundLayout` actúa como orquestador central de la interfaz. Mantiene el estado global de la sesión de trabajo e implementa las transiciones de estado definidas en el diseño: inicio y fin de ejecución, recepción de resultados, avance paso a paso y apertura de modales auxiliares.

El estado que gestiona el orquestador comprende los archivos del editor (`files`), el árbol de sintaxis abstracta activo (`ast`), los marcos del ambiente de ejecución (`frames`), el historial de mensajes de la consola (`logs`), los indicadores de sesión activa y ejecución en curso, el modo de evaluación paso a paso (`stepMode`) y la cola de pasos pendientes (`pendingSteps`). Los cuatro paneles de visualización (`ASTViewer`, `EnvironmentPanel`, `ConsoleOutput` y `EditorPanel`) son componentes React [30] sin estado propio: reciben exclusivamente los datos necesarios para su renderizado a través de las propiedades que el orquestador les suministra.

El modo de evaluación paso a paso opera mediante una cola de pasos (`pendingSteps`) que se inicializa con el resultado retornado por la API para cada expresión de nivel superior enviada por el estudiante. Cada vez que el usuario activa el control *Next step*, el orquestador extrae el siguiente paso de la cola y actualiza de forma sincronizada el árbol AST, los marcos del ambiente y la salida de la consola, mostrando el resultado de la expresión correspondiente a ese paso de forma aislada, sin conservar las ligaduras de las expresiones anteriores, con la granularidad descrita en el Capítulo 4.

La Figura 5.3 ilustra la interfaz en modo paso a paso, con el botón de avance visible en la consola y el estado del AST y el ambiente correspondiente al paso activo.

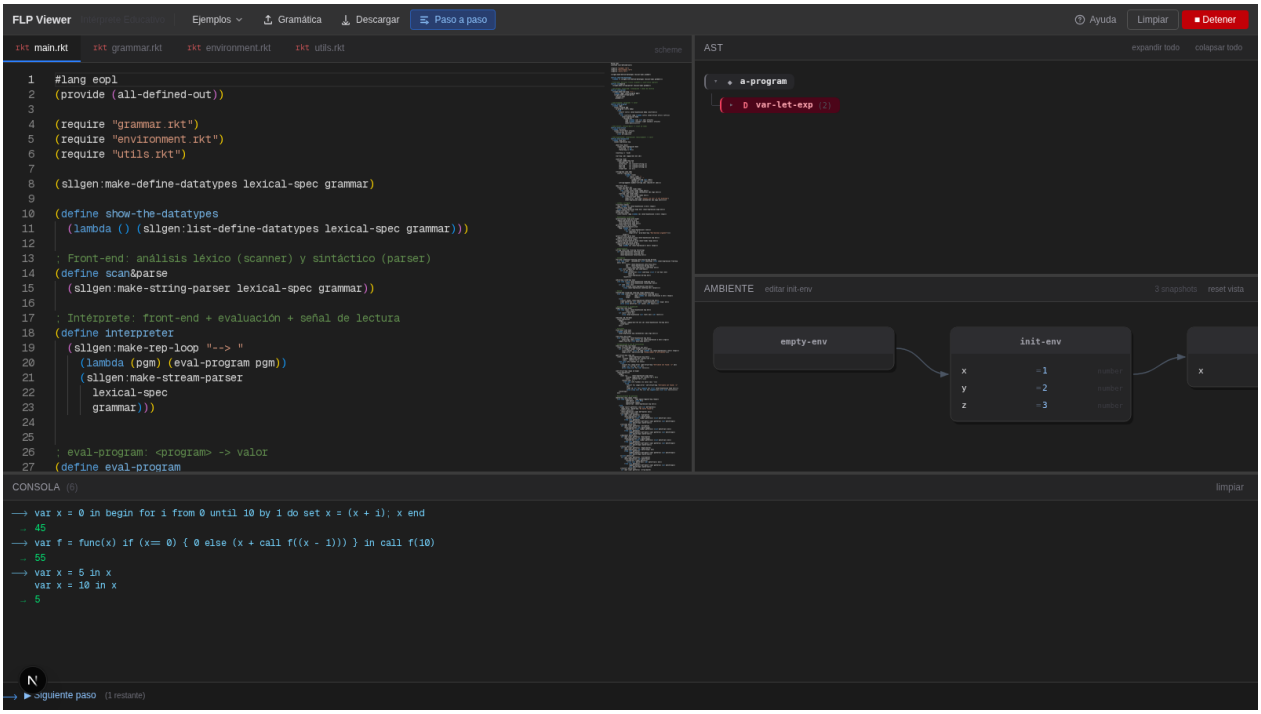


Figura 5.3: Modo de evaluación paso a paso con pasos pendientes de navegación
Fuente: Elaboración propia

La Figura 5.4 ilustra el modal de definición de gramática con una especificación BNF y la vista previa del archivo `grammar.rkt` generado.

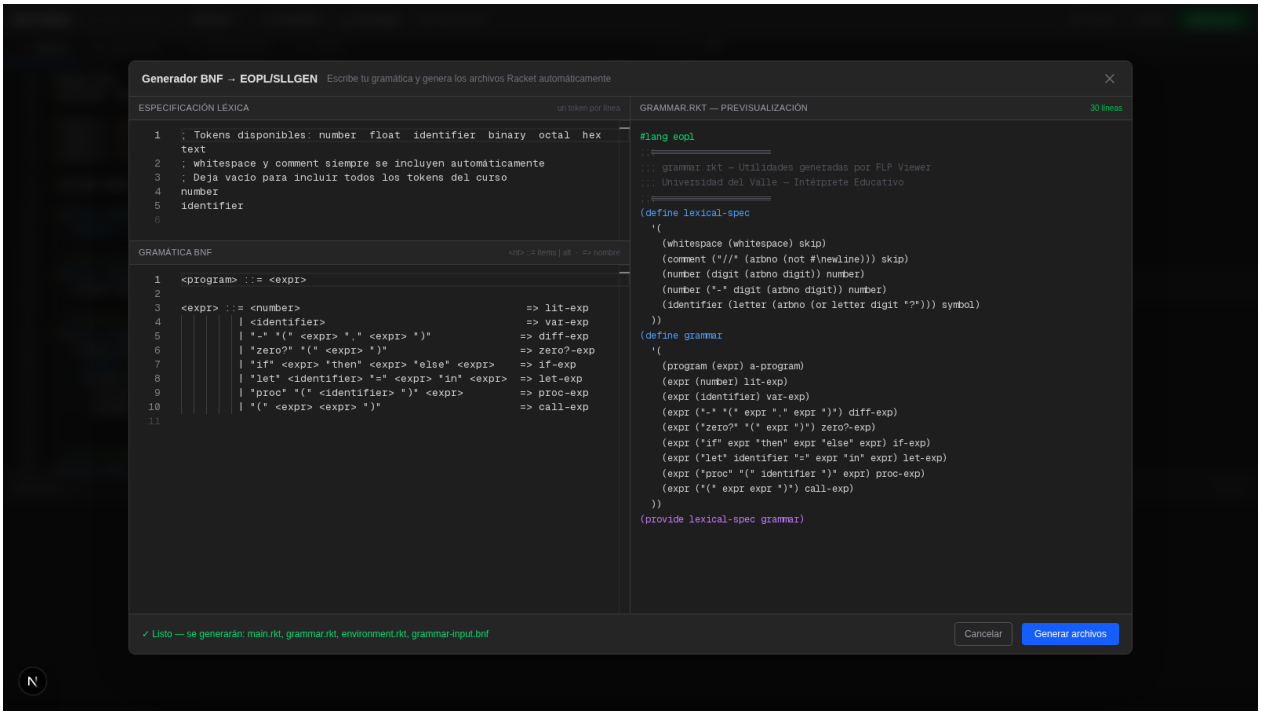


Figura 5.4: Modal de definición de gramática BNF con vista previa del archivo generado
Fuente: Elaboración propia

El modal de gramática (`GrammarModal`) recibe las entradas del estudiante y, al confirmar, invoca `runPipeline` para obtener los archivos generados, que son cargados en el panel de edición a través del mecanismo `upsertFile`. Si el pipeline reporta errores léxicos o sintácticos, estos se presentan en el modal antes de cerrar, sin modificar los archivos activos en el editor.

El componente `EditorPanel` utiliza Monaco Editor [32] para la edición de código. Las líneas generadas automáticamente por el pipeline se marcan como de solo lectura mediante la API de rangos de Monaco, de modo que el estudiante solo puede editar las secciones correspondientes a la lógica de evaluación.

Las Figuras 5.5 y 5.6 documentan dos evaluaciones representativas sobre el ejemplo *Intérprete EOPL (avanzado)*, seleccionadas por su correspondencia directa con los indicadores de logro IL 1.2.6 e IL 1.2.7, identificados en el Capítulo 3 como las fuentes de mayor dificultad conceptual del R.A.2 (Tabla 3.4). Dicho resultado concentra el 50.0 % del peso evaluativo de la asignatura, y el 77.8 % de los estudiantes encuestados reportó haber tenido dificultades con los conceptos asociados (entornos, estado y secuenciación) mientras cursaban la asignatura (P4, Tabla 3.1). Las evaluaciones cubren, de forma complementaria, los dos ejes semánticos que el diagnóstico identificó como prioritarios: el estado mutable con iteración, cuya comprensión exige distinguir entre ligadura y asignación destructiva (IL 1.2.7), y las funciones de orden superior con clausuras, cuya representación hace visible cómo el ambiente registra procedimientos como valores de primera clase almacenados junto a cualquier otra ligadura (IL 1.2.6) [7].

La Figura 5.5 evalúa la expresión `var x = 0 in begin for i from 0 until 10 by 1 do set x = (x + i); x end`, que acumula la suma de 0 a 9 mediante un bucle con asignación destructiva.

El panel de ambiente muestra la cadena `empty-env` → `init-env` → `x`, donde el marco final refleja el valor de `x` tras la última iteración del bucle. Este caso documenta que la instrumentación captura correctamente la evolución del estado mutable a lo largo de la ejecución.

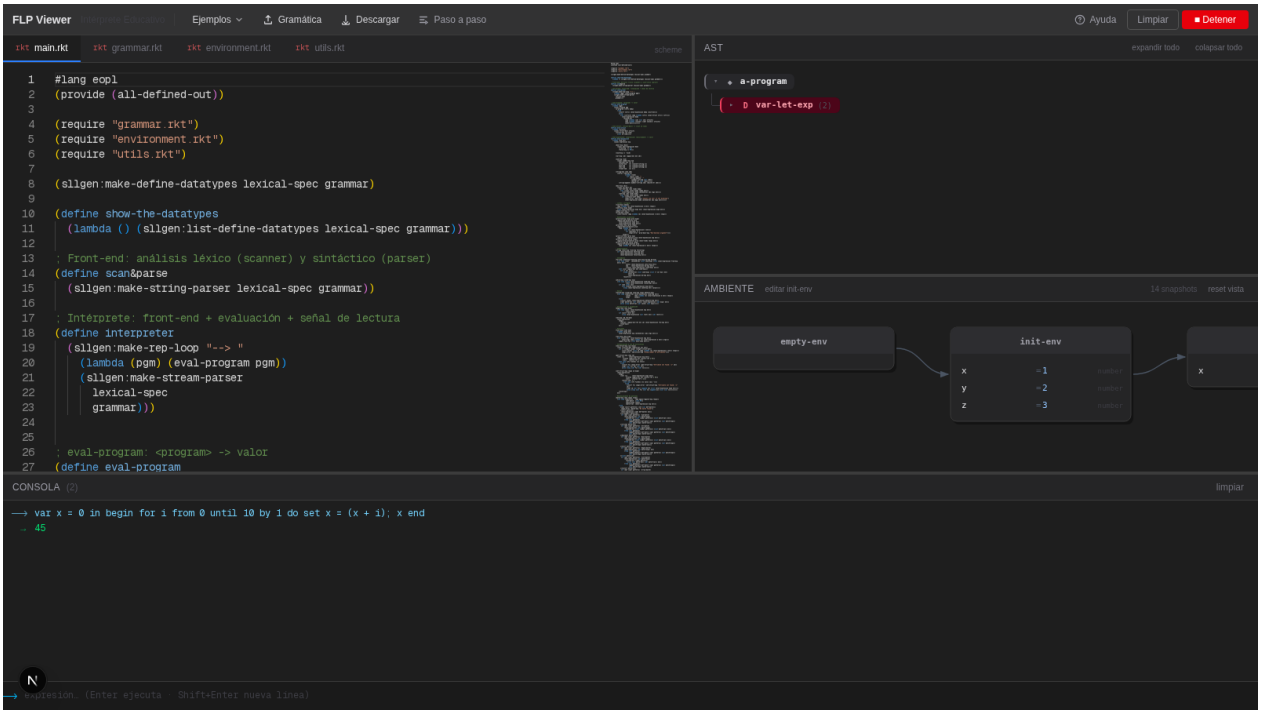


Figura 5.5: Evaluación de una expresión con iteración y estado mutable: suma acumulada mediante bucle `for`

Fuente: Elaboración propia

La Figura 5.6 evalúa la expresión `var f = func(x) if (x == 0) { 0 else (x + call f((x - 1))) } in call f(10)`, que define una función recursiva `f` y la aplica sobre 10, obteniendo la suma triangular 55. A diferencia del caso anterior, el marco final del ambiente contiene la ligadura `f` cuyo valor es una clausura, lo que permite al estudiante observar que en el modelo de EOPL las funciones son valores de primera clase almacenados en el ambiente como cualquier otra ligadura [7]. La presencia de la clausura en el panel de ambiente distingue esta visualización de la anterior y justifica su inclusión como caso complementario.

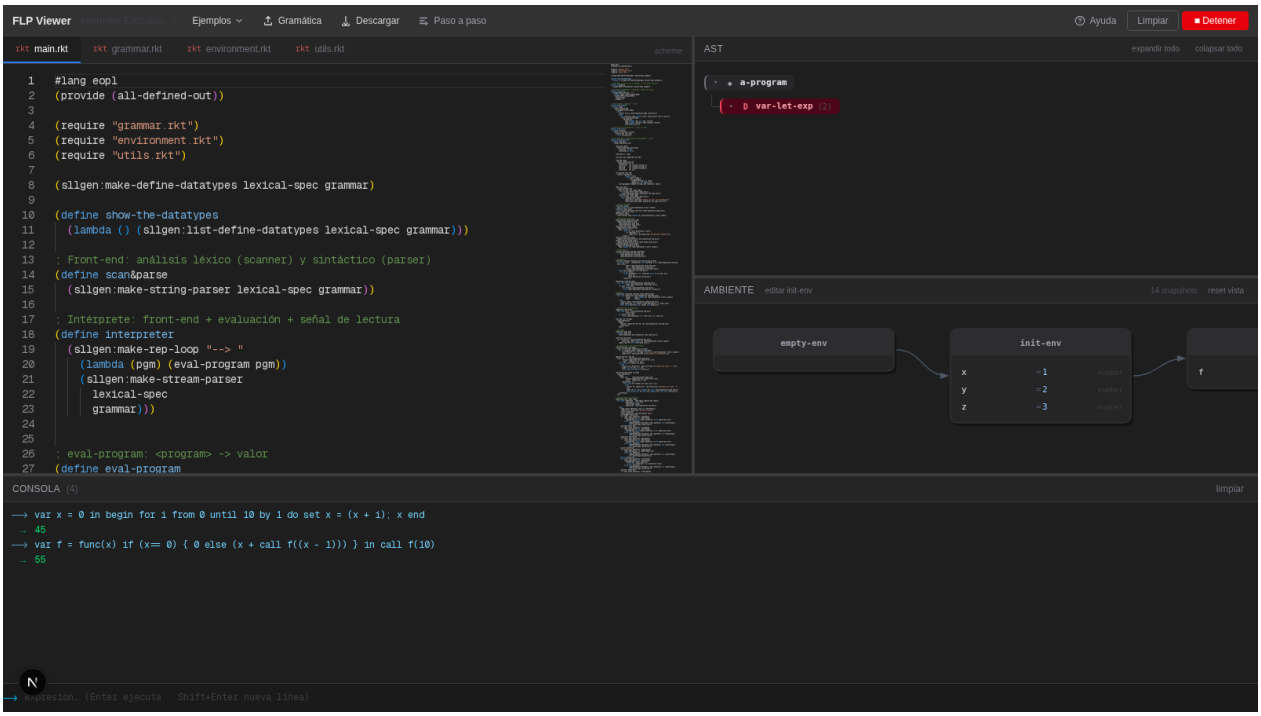


Figura 5.6: Evaluación de una función recursiva con clausura: la ligadura `f` en el panel de ambiente refleja una función como valor de primera clase

Fuente: Elaboración propia

5.2.3. Biblioteca de ejemplos predefinidos

La biblioteca de ejemplos predefinidos (RF-05) comprende seis programas cargados durante el renderizado inicial del servidor mediante `load-examples.ts` y presentados al estudiante desde el modal de bienvenida. Cada entrada está compuesta por el conjunto completo de archivos del intérprete (`grammar.rkt`, `environment.rkt`, `main.rkt` y `utils.rkt`), de modo que cualquier ejemplo puede ejecutarse y explorarse de forma inmediata sin modificaciones previas. La Tabla 5.1 presenta la correspondencia entre los ejemplos implementados y los indicadores de logro identificados en el Capítulo 3.

Tabla 5.1: Biblioteca de ejemplos predefinidos y correspondencia con indicadores de logro

#	Ejemplo	IL abordados
1	Hola Mundo	IL 1.3.1
2	Lenguaje LET	IL 1.1.6, IL 1.2.3
3	Ambientes	IL 1.2.6, IL 1.2.7
4	Procedimientos y clausuras	IL 1.2.6
5	Estado y asignación	IL 1.2.6, IL 1.2.7
6	Intérprete EOPL (avanzado)	IL 1.1.6, IL 1.2.6, IL 1.2.7

Fuente: elaboración propia a partir del microcurrículo de la asignatura (código 750017C).

La secuencia responde a dos ejes de variación simultáneos: la complejidad del lenguaje interpretado y la arquitectura del ambiente de ejecución. Los tres primeros ejemplos comparten un ambiente

funcional con valores almacenados en listas inmutables; a partir del cuarto, el ambiente introduce celdas mutables mediante el tipo `a-ref` sobre vectores, cambio arquitectónico en `environment.rkt` que hace observable la diferencia entre ligadura y asignación [7].

Hola Mundo define dos reglas gramaticales (`print-exp` y `string-exp`) y dos casos de evaluación, sin ligaduras activas ni operaciones sobre el ambiente, exponiendo el pipeline completo de interpretación en su forma más reducida [6]. *Lenguaje LET* implementa el lenguaje homónimo del Capítulo 3 de EOPL [7] con siete reglas de producción y seis casos de evaluación; `let-exp` introduce la primera llamada a `extend-env`, apilando cada ligadura en un nuevo marco funcional inmutable. *Ambientes* reutiliza la gramática y el evaluador del ejemplo anterior, modificando únicamente `init-env` para precargar tres ligaduras (`x=1`, `y=2`, `z=3`), lo que permite observar de inmediato la búsqueda por la cadena de marcos y el *shadowing* entre ligaduras homónimas [13].

Procedimientos y clausuras extiende el Lenguaje LET con `proc` y `call` (lenguaje PROC, Capítulo 3 de EOPL [7]); la clausura se implementa como una función Racket que captura el ambiente léxico vigente al evaluar `proc-exp`, de modo que la invocación posterior extiende ese ambiente capturado y no el activo en el punto de llamada. *Estado y asignación* introduce el lenguaje IMPLICIT-REFS (Capítulo 4 de EOPL [7]) mediante un cambio concentrado en `environment.rkt`: los valores pasan de listas a celdas mutables (`a-ref` sobre vectores), lo que habilita en `main.rkt` la operación `set` vía `setref!` sin crear un nuevo marco, a diferencia de `let`. *Intérprete EOPL (avanzado)* combina el modelo de referencias mutables con un lenguaje propio que incorpora estructuras de datos, arreglos, iteradores (`for`, `while`), `switch` y reconocimiento de patrones; la función `contains-set?` hace cumplir en tiempo de ejecución que las ligaduras `let` no admitan asignación, y este intérprete sirve como base de las evaluaciones representativas documentadas en las Figuras 5.5 y 5.6.

5.2.4. Punto de acceso de ejecución

La ruta de API `/api/run` se implementó como un manejador HTTP POST en `app/api/run/route.ts`, siguiendo el modelo de rutas de Next.js. Su flujo de operación comprende cuatro fases.

En la primera fase, el servidor crea un directorio temporal exclusivo para la solicitud mediante `mkdtemp`[3] y escribe en él todos los archivos del editor recibidos en el cuerpo de la petición. Antes de escribir `environment.rkt` y `main.rkt`, la función `injectRuntime` inyecta en su contenido los módulos de instrumentación correspondientes (`_tracking.rkt` y `_stream-parser.rkt` respectivamente). El archivo `_runner.rkt` se copia también al directorio temporal como punto de entrada del proceso Racket.

En la segunda fase, el servidor lanza el proceso Racket sobre `_runner.rkt` mediante `execFile`, pasando la expresión de prueba como argumento y aplicando un tiempo límite de 15 segundos. La señal de cancelación del `AbortController`[4] asociado a la petición permite interrumpir el proceso si el cliente cancela la solicitud antes de que finalice la ejecución.

En la tercera fase, el servidor deserializa la salida estándar del proceso. Si la salida es un arreglo JSON válido, se interpreta como una secuencia de pasos de evaluación. De lo contrario, se trata

[3] `mkdtemp` es una llamada del sistema POSIX que crea un directorio temporal con nombre único, garantizando que no colisione con directorios preexistentes aunque varias solicitudes se procesen de forma concurrente.

[4] `AbortController` es una API web estándar que permite cancelar operaciones asíncronas mediante una señal compartida entre el código que inicia la operación y el que la ejecuta. Cuando el cliente cierra la conexión, la señal se activa y el proceso Racket es terminado antes de consumir el tiempo límite completo.

como texto plano. Los mensajes de error presentes en `stderr` son normalizados por la función `cleanStderr`, que elimina las secciones de contexto interno del intérprete Racket que resultan irrelevantes para el estudiante.

En la cuarta fase, el servidor elimina el directorio temporal y retorna la respuesta HTTP con los campos `steps`, `stderr` y `error`. La eliminación del directorio se realiza en el bloque `finally` del manejador para garantizar la limpieza del sistema de archivos independientemente del resultado de la ejecución.

5.2.5. Instrumentación en tiempo de ejecución

La instrumentación en tiempo de ejecución se implementó mediante seis archivos Racket que la función `injectRuntime` de `route.ts` inyecta en la copia temporal descrita en la sección anterior, de forma transparente para el estudiante. Dos de ellos, `_json-value.rkt` y `_runner.rkt`, se copian sin modificaciones al directorio temporal; los cuatro restantes se anexan condicionalmente al final de `environment.rkt` o de `main.rkt`.

- `_json-value.rkt` define la función `json-value`, que convierte cualquier valor producido por el intérprete del estudiante, incluyendo los *structs* generados por `define-datatype`, clausuras y celdas mutables, en una estructura serializable mediante `write-json`. La función rastrea por identidad los valores ya visitados para evitar ciclos infinitos ante clausuras que se capturan a sí mismas.
- `_tracking.rkt` se anexa siempre a `environment.rkt` y redefine `empty-env` y `extend-env` para registrar en el log interno `_env-log` un *snapshot*[5] de cada marco creado, junto con la posición del marco padre que extiende, de modo que la cadena de ambientes pueda reconstruirse en el panel de visualización.
- `_tracking-assign.rkt` se anexa a `environment.rkt` únicamente cuando este define `apply-env-ref` y `setref!`, detectado mediante una expresión regular sobre su contenido, es decir, en los intérpretes construidos sobre referencias implícitas. Redefine ambas funciones para registrar las asignaciones como eventos independientes de las ligaduras, asociando cada una al marco cuya celda fue mutada.
- `_tracking-init-env.rkt` se anexa a `environment.rkt` únicamente cuando `init-env` está definido como un procedimiento. Rotula como `init-env` todos los marcos creados durante su ejecución, de modo que un ambiente inicial compuesto por varios marcos se identifique en su totalidad como tal y no solo el primero.
- `_stream-parser.rkt` se anexa siempre al final de `main.rkt` y expone la función `stream-parser`, construida a partir de la especificación léxica y gramatical del estudiante mediante `sllgen:make-stream-parser`.
- `_runner.rkt` actúa como punto de entrada del proceso Racket: carga los módulos del proyecto en el orden correcto, recibe la expresión de prueba como argumento de línea de comandos y, mediante la función `run-trace`, itera sobre cada expresión de nivel superior producida por `stream-parser`, reinicia el log de ambientes antes de evaluarla y serializa como JSON el

[5] Un *snapshot* (instantánea) es una copia del estado de una estructura de datos en un instante determinado, tomada para preservar su contenido en ese momento aunque la estructura original siga modificándose después.

AST, el ambiente resultante y el valor producido en cada paso, dentro de un manejador de excepciones que captura los errores del intérprete.

Dado que la inyección ocurre exclusivamente sobre la copia temporal empleada para la ejecución, los archivos que conforman el estado del editor en el cliente nunca contienen código de instrumentación. En consecuencia, la función `downloadZip` de `playground-utils.ts` exporta directamente esos archivos en un ZIP compatible con el entorno DrRacket [12], sin requerir ningún paso adicional de limpieza.

5.3. Flujo de ejecución principal

La Figura 5.7 ilustra el flujo de ejecución que se produce cuando el estudiante envía una expresión desde la consola del prototipo, desde la interacción inicial hasta la obtención del resultado en el servidor. El flujo inicia en el componente `ConsoleOutput`, que delega la acción al orquestador `PlaygroundLayout`. Este registra la expresión en el historial, inicializa el mecanismo de cancelación y deshabilita nuevas ejecuciones concurrentes. A continuación, invoca el servicio `racket.ts`, que empaqueta los archivos del editor y la expresión en una petición HTTP POST hacia `/api/run`. El servidor de aplicación recibe la solicitud, prepara el entorno de ejecución aislado, lanza el proceso Racket, deserializa su salida y devuelve la secuencia de pasos al cliente.

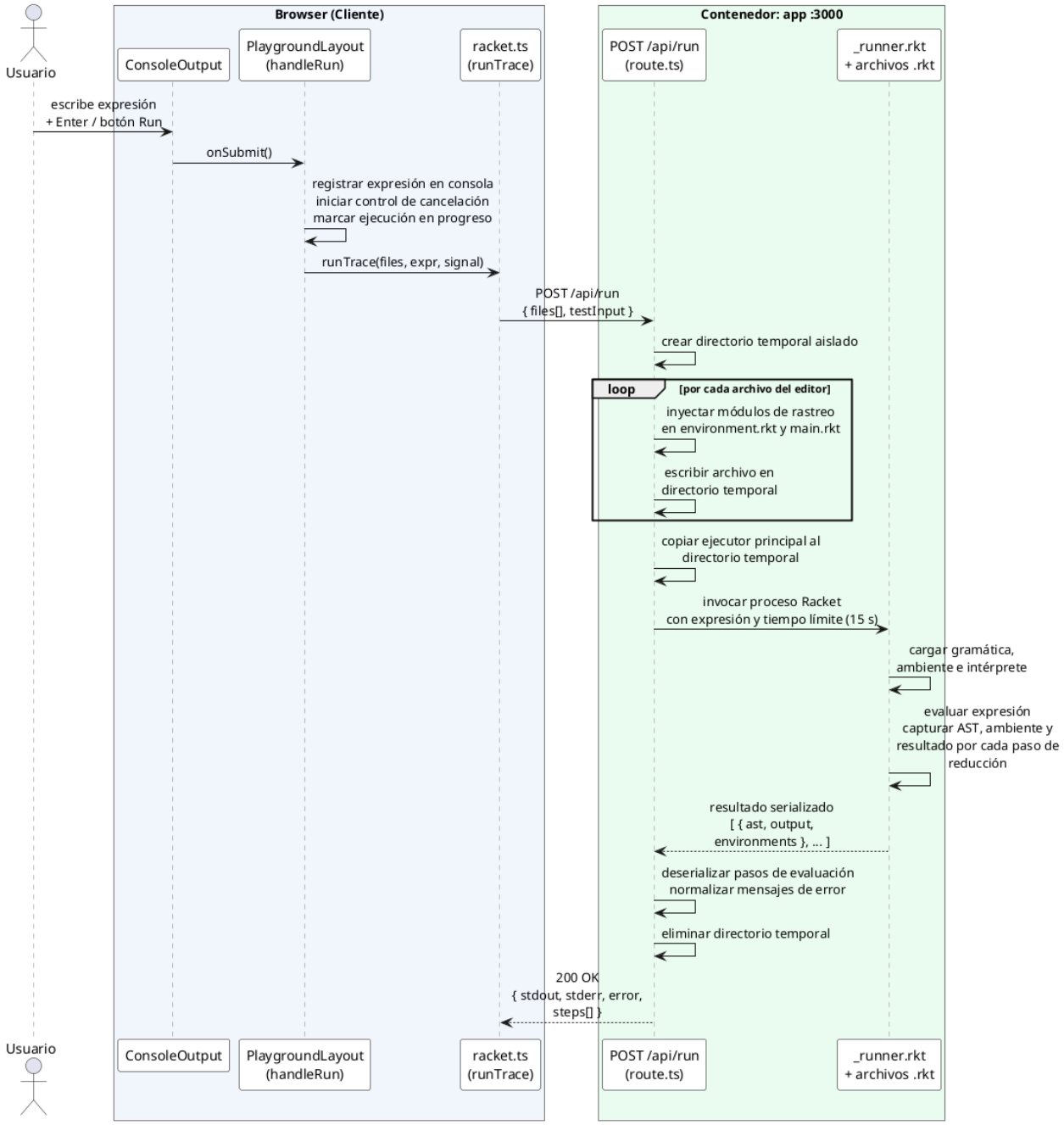


Figura 5.7: Diagrama de secuencia de la solicitud y evaluación en el servidor del prototipo FLP Viewer

Fuente: Elaboración propia

Una vez que el cliente recibe la respuesta, `PlaygroundLayout` distribuye los resultados entre los paneles de visualización según el modo activo, tal como se muestra en la Figura 5.8: en modo normal actualiza el AST, el ambiente y la consola con el estado final, y en modo paso a paso encola los pasos y presenta el primero de forma inmediata, habilitando la navegación manual.

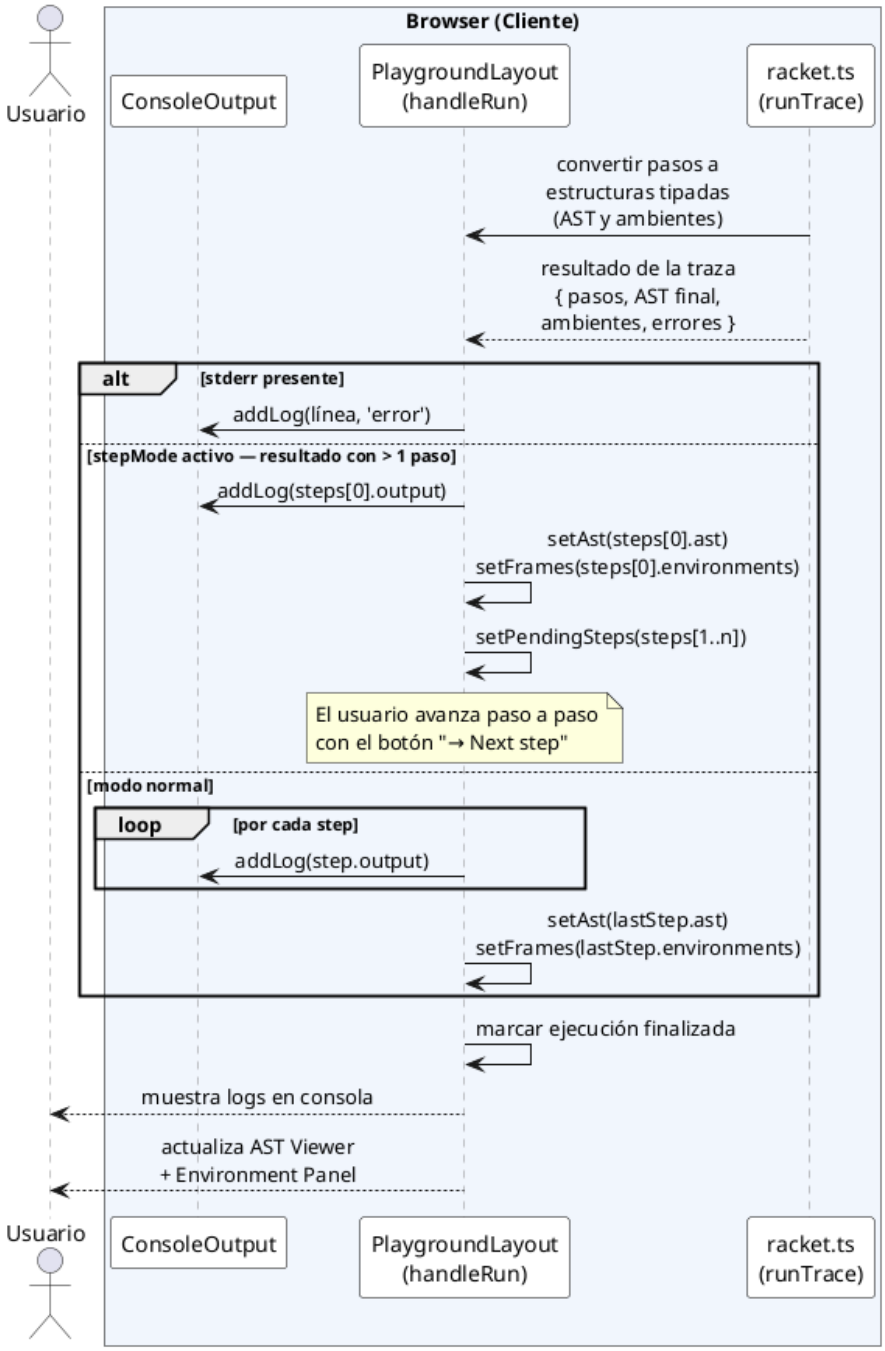


Figura 5.8: Diagrama de secuencia de la distribución de resultados en la interfaz del prototipo FLP Viewer

Fuente: Elaboración propia

5.4. Despliegue en entorno de pruebas

El prototipo se despliega mediante Coolify [36], una plataforma de despliegue autoalojada que permite gestionar aplicaciones contenerizadas sobre infraestructura propia y construir la imagen de la aplicación directamente a partir del *Dockerfile* del proyecto. Su elección responde a la necesidad de disponer de un mecanismo de despliegue reproducible y centralizado sobre el servidor utilizado para el entorno de pruebas, delegando en la plataforma tareas de infraestructura como el enrutamiento de solicitudes y la gestión de certificados TLS. La Figura 5.9 presenta el diagrama de la infraestructura de despliegue.

La imagen del contenedor de aplicación se construye mediante un *Dockerfile* multi-etapa con tres etapas diferenciadas. La primera etapa (`deps`) instala las dependencias del proyecto utilizando `pnpm` sobre una imagen base de Node.js 25 Alpine. La segunda etapa (`builder`) ejecuta la construcción de Next.js en modo *standalone*[6], que produce un artefacto autocontenido con todas las dependencias necesarias para ejecutar el servidor en producción. La tercera etapa (`runner`) utiliza la imagen `node:25-bookworm-slim` e instala el intérprete de Racket como dependencia del sistema operativo mediante `apt-get`[7]. Esta etapa copia únicamente el artefacto *standalone*, el directorio de archivos estáticos y los tres archivos de instrumentación Racket, minimizando el tamaño de la imagen final. La variable de entorno `RACKET_BIN` se configura en `/usr/bin/racket` para que la ruta de API localice el ejecutable sin depender de la configuración del sistema anfitrión.

El contenedor de la aplicación expone el puerto interno 3000, el cual no se publica directamente hacia el host. Coolify enruta las solicitudes externas hacia este puerto mediante su proxy interno, basado en Traefik por defecto, que dirige cada solicitud hacia el contenedor correspondiente según el dominio configurado y gestiona de forma automática los certificados TLS mediante Let's Encrypt [37], sin requerir la definición manual de reglas en el proyecto. Esta arquitectura garantiza que el acceso externo al prototipo se realice exclusivamente a través del proxy, centralizando el control de las solicitudes entrantes.

[6] El modo *standalone* de Next.js genera un servidor Node.js autocontenido que incluye únicamente las dependencias estrictamente necesarias para producción, sin requerir la carpeta `node_modules` completa en el entorno de despliegue. Esto reduce significativamente el tamaño de la imagen Docker resultante.

[7] `apt-get` es el gestor de paquetes de las distribuciones basadas en Debian, utilizado para instalar, actualizar y eliminar software del sistema operativo a partir de repositorios remotos.

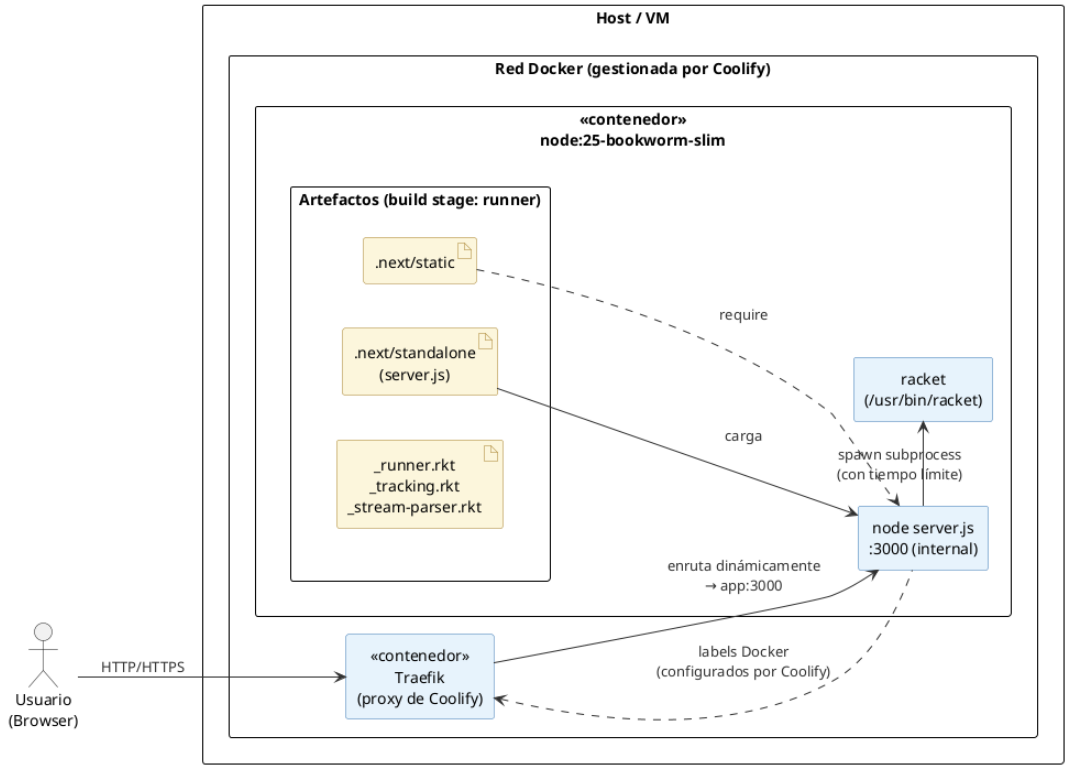


Figura 5.9: Diagrama de despliegue del prototipo FLP Viewer en el entorno de pruebas
Fuente: Elaboración propia

5.5. Verificación de requisitos funcionales

La Tabla 5.2 presenta la correspondencia entre los requisitos funcionales definidos en el diseño y los módulos que materializan cada uno en la implementación. Todos los requisitos de prioridad alta y media definidos en la especificación fueron implementados en el prototipo entregado.

Tabla 5.2: Verificación del cumplimiento de los requisitos funcionales

ID	Nombre	Módulos de implementación	Estado
RF-01	Generación del AST	`_runner.rkt`, `_stream-parser.rkt`, `racket.ts`	Implementado
RF-02	Visualización del AST	`ASTViewer.tsx`, `PlaygroundLayout.tsx`	Implementado
RF-03	Ejecución y evaluación	`app/api/run/route.ts` (*endpoint* `/api/run`)	Implementado
RF-04	Representación del ambiente	`_tracking.rkt`, `EnvironmentPanel.tsx`	Implementado
RF-05	Biblioteca de ejemplos	`examples/`, `load-examples.ts`, `Welcome Modal.tsx`	Implementado
RF-06	Validación de sintaxis Racket	`bnf-lexer.ts`, `bnf-parser.ts`, `cleanStderr()` en `route.ts`	Implementado
RF-07	Visualización del intérprete	`EditorPanel.tsx`, `CodeEditor.tsx` (líneas bloqueadas con Monaco Editor)	Implementado
RF-08	Plantilla base y exportación	`INITIAL_FILES` y `downloadZip()` en `playground-utils.ts`	Implementado

Fuente: elaboración propia.

5.6. Evaluación de la calidad del software

Con el propósito de complementar la verificación funcional presentada en la sección anterior con una valoración de la calidad técnica del código fuente, se realizó un análisis estático del prototipo FLP Viewer mediante SonarCloud. Esta evaluación permite identificar problemas relacionados con confiabilidad, seguridad, mantenibilidad, duplicación de código y cobertura de las pruebas automatizadas, aspectos que no son observables a través de la verificación de requisitos funcionales descrita previamente.

SonarCloud es una plataforma de análisis continuo de código fuente que examina un proyecto en busca de errores (*bugs*), vulnerabilidades, *code smells*[8] y fragmentos duplicados, además de integrar los resultados de cobertura generados por las pruebas automatizadas del proyecto. Sobre estas métricas, la plataforma aplica *Quality Gates*, definidos como un conjunto de condiciones que se evalúan sobre los resultados de cada análisis para determinar si el código cumple un nivel mínimo de calidad. El *Quality Gate* se reporta como aprobado cuando los resultados satisfacen dichas condiciones, y como no aprobado en caso contrario [38].

Para la evaluación del prototipo, el repositorio de FLP Viewer se integró con SonarCloud y se ejecutó el análisis sobre la rama principal (`main`) una vez finalizado el desarrollo de las funcionalidades contempladas, con el propósito de obtener una medición del estado final del código. La Figura 5.10 presenta el panel de resultados generado por la plataforma, y la Tabla 5.3 resume los valores obtenidos para cada métrica.

[8] Patrones de código que, sin constituir errores funcionales, incrementan la deuda técnica y dificultan el mantenimiento del software.

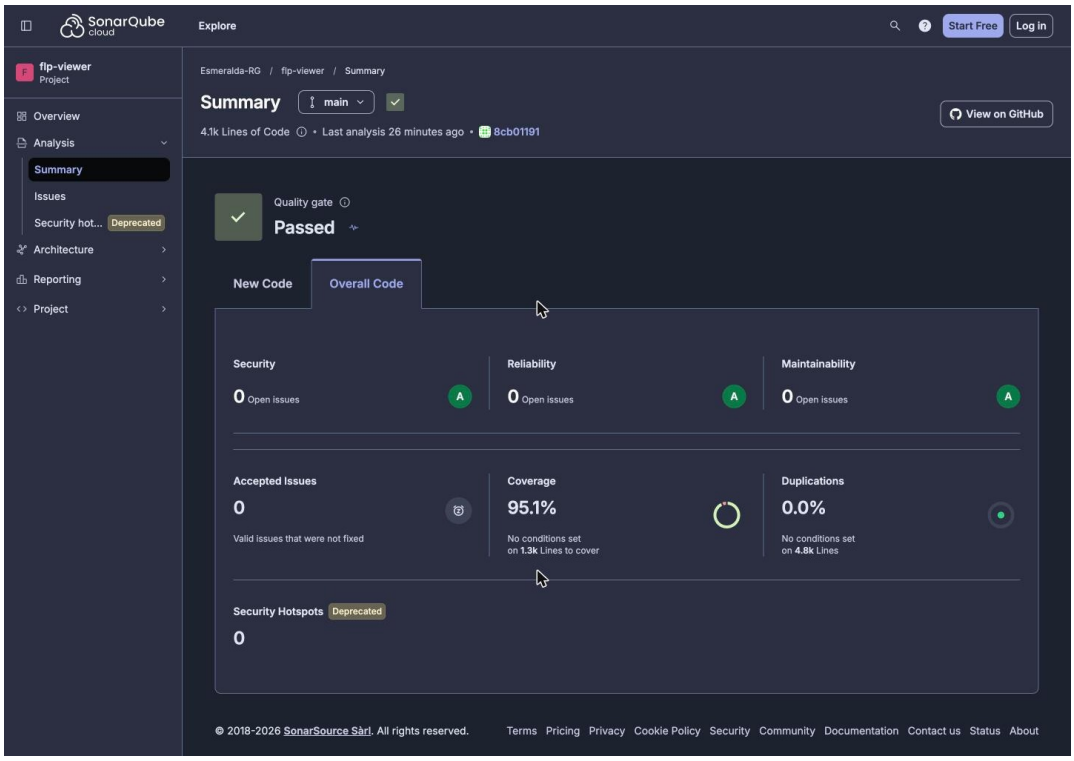


Figura 5.10: Panel de resultados del análisis de calidad del prototipo FLP Viewer en SonarCloud
Fuente: SonarCloud, rama `main` del repositorio del proyecto

Tabla 5.3: Resultados del análisis de calidad del prototipo FLP Viewer mediante SonarCloud

Métrica	**Resultado**
Quality Gate	Aprobado
Bugs	0
Vulnerabilidades	0
Code Smells	0
Security Hotspots	0
Duplicación de código	0.0 %
Cobertura de código	95.1 %
Reliability Rating	A
Security Rating	A
Maintainability Rating	A

Fuente: elaboración propia, a partir del análisis de SonarCloud sobre la rama `main` *del repositorio*[9].

La cobertura obtenida, del 95.1 %, indica la proporción del código ejecutable del prototipo que fue ejercitada mediante las pruebas automatizadas del proyecto. Esta métrica permite identificar qué partes del código fueron efectivamente recorridas durante las pruebas y, en consecuencia, detectar zonas que podrían requerir casos de prueba adicionales [39]. Las calificaciones de confiabilidad, seguridad y mantenibilidad obtenidas corresponden en los tres casos al nivel A, el más favorable de la escala empleada por SonarCloud, en la cual el nivel A de confiabilidad y de seguridad se asigna cuando el análisis no reporta errores ni vulnerabilidades, respectivamente, y el nivel A de mante-

nibilidad se asigna cuando la deuda técnica estimada no supera el 5 % del esfuerzo de desarrollo del código analizado [40]. De manera consistente con estas calificaciones, el análisis no reportó *code smells*, *security hotspots* pendientes de revisión ni fragmentos de código duplicados, y el proyecto satisfizo las condiciones del *Quality Gate Sonar way* aplicado, por lo que el resultado global del análisis se reportó como aprobado.

Estos resultados deben interpretarse dentro del alcance del análisis realizado. El hecho de que SonarCloud no reporte errores ni vulnerabilidades en el código analizado no implica que el prototipo esté libre de defectos, sino que el código no presentó, al momento de la evaluación, incidencias frente a las reglas de análisis estático aplicadas por la herramienta. De manera análoga, la cobertura reportada refleja qué proporción del código fue ejercitada por las pruebas existentes, sin constituir por sí sola una medida de la corrección funcional del sistema. En conjunto, los resultados obtenidos constituyen evidencia favorable respecto a la calidad técnica del código del prototipo, particularmente en lo referente a la ausencia de duplicación, la cobertura de pruebas alcanzada y las calificaciones de confiabilidad, seguridad y mantenibilidad, en el contexto de un prototipo de alcance acotado de aproximadamente 4 100 líneas de código.

Capítulo 6

Evaluación del prototipo FLP Viewer

Una vez implementado el prototipo, se planificó su evaluación con estudiantes de la asignatura FLP con el propósito de determinar en qué medida la herramienta resulta utilizable y si los estudiantes perciben que contribuye a la comprensión de los conceptos que motivaron su desarrollo. La evaluación se orientó a dos dimensiones complementarias: la usabilidad, entendida como la facilidad con la que el estudiante interactúa con el sistema, y la utilidad pedagógica percibida, entendida como la valoración que hace el estudiante del aporte del prototipo frente a los indicadores de logro seleccionados en el Capítulo 3.

Ambas dimensiones deben analizarse de forma conjunta, ya que una herramienta con una utilidad conceptual alta puede resultar poco aprovechable si su interfaz introduce una carga adicional al estudiante, mientras que una interfaz de uso sencillo no garantiza, por sí sola, un aporte al aprendizaje. Esta distinción cobra especial relevancia en el contexto de la asignatura, donde el diagnóstico identificó que la sintaxis de Racket actúa como una fuente de carga cognitiva extrínseca que desplaza la atención de los estudiantes de la semántica de los lenguajes hacia la resolución de errores sintácticos [9].

6.1. Metodología

La evaluación se planteó como un estudio descriptivo con enfoque mixto. El componente cuantitativo se apoyó en cuestionarios con escala Likert [22], mientras que el componente cualitativo se desarrolló mediante preguntas abiertas que permiten recoger, en las propias palabras del estudiante, los aspectos que no alcanzan a capturar los ítems cerrados. Este planteamiento mantiene la lógica del diagnóstico presentado en el Capítulo 3, lo que facilita contrastar las dificultades reportadas inicialmente por los estudiantes con su valoración del prototipo.

La evaluación se dirigió exclusivamente a los estudiantes y no incluyó una valoración por parte del cuerpo docente. En la sede Tuluá, la asignatura FLP es dictada por un único profesor en todos los períodos académicos, quien a su vez se desempeña como director del presente trabajo de grado. En estas condiciones, un instrumento dirigido al docente contaría con un solo respondiente, lo que impide cualquier tratamiento estadístico y reduce su aporte a una opinión individual. Adicionalmente, su doble rol como director y evaluador compromete la independencia de la valoración, dado que quien juzga la pertinencia del prototipo participó en la orientación de su diseño. Por estas razones,

la perspectiva docente se reemplazó por un contraste con las limitaciones en el aprendizaje de estos conceptos documentadas en la literatura, el cual se presenta en la Sección 6.2 y ofrece un marco de referencia externo e independiente para interpretar las valoraciones de los estudiantes.

6.1.1. Diseño del instrumento

El instrumento se estructuró en cuatro secciones aplicadas de forma consecutiva en un único formulario, cuyo contenido completo se documenta en el Anexo F. La primera sección recoge datos de contexto del participante, la segunda corresponde al cuestionario de usabilidad, la tercera al cuestionario de utilidad pedagógica y la cuarta a las preguntas abiertas.

La sección de datos de contexto se compone de tres preguntas orientadas a caracterizar a los participantes sin comprometer su anonimato. Estas indagan por el semestre que cursa el estudiante, su experiencia previa con Racket antes de la asignatura y si había utilizado el prototipo con anterioridad a la sesión de evaluación. Las dos últimas preguntas permiten identificar si la familiaridad previa con el lenguaje o con la herramienta incide en la valoración que el estudiante hace de esta.

Para medir la usabilidad se seleccionó el cuestionario System Usability Scale (SUS), un instrumento estandarizado de diez ítems con escala Likert de cinco niveles, ampliamente utilizado para medir la usabilidad percibida de un sistema interactivo con independencia de su dominio de aplicación [41]. Se optó por este instrumento en lugar de uno de elaboración propia porque su carácter estandarizado permite interpretar el puntaje obtenido a partir de un umbral de referencia y una escala de adjetivos respaldados empíricamente, en lugar de depender de una interpretación arbitraria [42]. Los ítems del cuestionario alternan afirmaciones en sentido positivo y negativo, lo que reduce el sesgo de respuesta automática y obliga al participante a leer cada enunciado antes de responder.

Para la dimensión pedagógica se elaboró un cuestionario propio, dado que no existe un instrumento estandarizado que mida la utilidad percibida de una herramienta de visualización frente a los indicadores de logro específicos de FLP. El cuestionario está compuesto por ocho ítems con la misma escala Likert de cinco niveles utilizada en el SUS (1: Totalmente en desacuerdo, 5: Totalmente de acuerdo), de modo que el participante no deba adaptarse a un formato de respuesta distinto dentro del mismo formulario. A diferencia de una medición genérica de satisfacción, cada ítem se formuló a partir de los indicadores de logro seleccionados como base conceptual del prototipo (Tabla 3.4), de manera que las respuestas pudieran relacionarse con un contenido concreto de la asignatura. Así, el ítem P1 se vincula con la construcción del AST a partir de una gramática BNF (IL 1.1.6), los ítems P2 y P5 con el manejo del estado y los ambientes de ejecución durante la evaluación de un programa (IL 1.2.3 e IL 1.2.6) y el ítem P3 con la distinción entre ligadura y asignación de variables (IL 1.2.7).

Los ítems P4, P6 y P7 se orientaron a la valoración general del prototipo como recurso de apoyo, considerando su relación con las herramientas empleadas actualmente en la asignatura, la utilidad de la biblioteca de ejemplos predefinidos y su pertinencia para el estudio de los contenidos de FLP. Su inclusión responde directamente a los hallazgos del diagnóstico, en el que solo el 19.4 % de los estudiantes consideró que Racket y EOPL facilitaron su comprensión y el 94.5 % manifestó que recursos adicionales habrían mejorado su aprendizaje. Por su parte, el ítem P8 se redactó en sentido inverso respecto a los demás ítems del bloque, como mecanismo de control frente al sesgo de aquiescencia, es decir, la tendencia de algunos participantes a mostrarse de acuerdo con las afirmaciones independientemente de su contenido. La correspondencia completa entre ítems e

indicadores de logro se presenta en la Tabla F.4 del Anexo F.

Finalmente, el instrumento cierra con tres preguntas abiertas que indagan por la funcionalidad del prototipo que el estudiante considera más útil para comprender los contenidos de la asignatura, los aspectos que mejoraría o que echó en falta, y los conceptos que el prototipo no le ayudó a comprender o que le generaron confusión adicional. Esta última pregunta se incluyó de forma deliberada para recoger evidencia contraria a la hipótesis de utilidad del prototipo, evitando que el instrumento se limitara a registrar valoraciones favorables.

6.1.2. Selección de la población

La población objetivo corresponde a los estudiantes matriculados en la asignatura FLP en la Universidad del Valle sede Tuluá durante el período de desarrollo del trabajo de grado. A diferencia de la encuesta diagnóstica del Capítulo 3, que se aplicó de forma retrospectiva a estudiantes de distintas cohortes, la evaluación exige que el participante interactúe directamente con el prototipo antes de responder el instrumento, por lo que la convocatoria se restringió a quienes cursan actualmente la asignatura y tienen presentes los contenidos evaluados.

La selección de los participantes fue por conveniencia y la participación fue voluntaria. El tamaño de muestra previsto se ubica entre 10 y 20 estudiantes, conforme al alcance definido para el trabajo de grado. Si bien este tamaño no permite generalizar los resultados a la población de estudiantes de la asignatura, resulta adecuado para un estudio descriptivo de usabilidad, en el que el objetivo es caracterizar la experiencia de uso de un prototipo y no estimar parámetros poblacionales.

Con el fin de garantizar un tratamiento ético de la información, el formulario inicia con un consentimiento informado en el que se explica el propósito del estudio, el carácter voluntario y anónimo de la participación y la ausencia de efectos sobre la calificación de la asignatura. La información recolectada se utiliza exclusivamente con fines académicos, conforme a lo dispuesto en la Ley 1581 de 2012 sobre protección de datos personales [43]. El texto completo del consentimiento se presenta en el Anexo F.2.

6.1.3. Procedimiento de aplicación

La evaluación se desarrolla en una sesión en la que cada estudiante interactúa con las tres funcionalidades núcleo del prototipo. En primer lugar, el estudiante define o selecciona una gramática en notación BNF, ya sea construyéndola desde cero o partiendo de uno de los casos de la biblioteca de ejemplos predefinidos. A continuación, ingresa un programa y visualiza el AST generado a partir de la gramática, y finalmente ejecuta el programa paso a paso mientras observa la evolución del ambiente de ejecución.

El instrumento se responde inmediatamente después de esta interacción, de modo que la valoración del estudiante se base en una experiencia reciente con el prototipo y no en una impresión general de la herramienta. Aplicar el instrumento al final de la sesión, y no durante ella, evita interrumpir la exploración del prototipo e impide que las preguntas orienten la forma en que el estudiante interactúa con este.

6.1.4. Plan de análisis

El puntaje SUS de cada participante se calcula conforme al procedimiento definido por el instrumento. Para los ítems impares se resta uno a la respuesta registrada, para los ítems pares se resta la respuesta registrada de cinco, y la suma de los diez valores resultantes se multiplica por 2.5, obteniendo un puntaje entre 0 y 100 [41]. A partir de los puntajes individuales se obtienen el promedio, la desviación estándar y los valores mínimo y máximo, y el promedio se interpreta con base en la escala de adjetivos de referencia y el umbral de 68 puntos reportado en la literatura [42].

Las respuestas del cuestionario de utilidad pedagógica se analizan mediante la distribución porcentual de cada nivel de la escala por ítem, agrupando los niveles de acuerdo y de desacuerdo, siguiendo el mismo formato empleado en el análisis de la encuesta diagnóstica. Antes del análisis, el puntaje del ítem P8 se invierte restando la respuesta registrada de seis, y la consistencia interna del bloque se estima mediante el coeficiente alfa de Cronbach [44]. Este cálculo permite verificar si los ítems de elaboración propia miden de forma coherente una misma dimensión antes de interpretar sus resultados de manera conjunta.

Las respuestas a las preguntas abiertas se someten a un análisis temático, en el que se agrupan los comentarios recurrentes en categorías, del mismo modo que en el análisis cualitativo de la encuesta diagnóstica. Estas categorías se contrastan con los resultados cuantitativos para identificar coincidencias o tensiones entre la valoración numérica de los estudiantes y sus observaciones escritas.

Por último, los resultados de ambos cuestionarios y de las preguntas abiertas se contrastan con las limitaciones de aprendizaje documentadas en la literatura para cada concepto evaluado. Este contraste no pretende establecer una relación causal entre el uso del prototipo y el aprendizaje, sino determinar si las valoraciones de los estudiantes son coherentes con las dificultades que la literatura reporta y con las estrategias que esta misma literatura propone para atenderlas.

6.2. Limitaciones de aprendizaje documentadas en la literatura

Con el fin de disponer de un marco de referencia independiente para interpretar los resultados, se revisaron las dificultades de aprendizaje que la literatura reporta sobre los conceptos representados en el prototipo. La revisión se organizó en torno a los mismos indicadores de logro que orientaron la formulación del cuestionario de utilidad pedagógica, de modo que cada limitación identificada pudiera relacionarse con uno o más ítems del instrumento.

6.2.1. De la gramática a la estructura jerárquica

La relación entre una gramática formal y la estructura jerárquica que esta produce constituye una de las primeras barreras en los cursos de compiladores e intérpretes. Las herramientas orientadas a este contenido parten de la observación de que los estudiantes tienen dificultades para relacionar una especificación textual, como una gramática, con el árbol que resulta de aplicarla sobre un programa concreto, y reportan mejoras en la comprensión cuando ambas representaciones se presentan de forma dinámica y sincronizada [4, 6]. En la misma línea, la experiencia de del Vado Vírseda muestra que la visualización de las estructuras que produce el analizador sintáctico permite al estudiante conectar la implementación con la teoría que la sustenta [19]. Estas limitaciones sirven como referencia para interpretar el ítem P1, asociado al indicador IL 1.1.6.

6.2.2. Ambientes de ejecución y estado

La literatura sobre enseñanza de la programación identifica la comprensión de la dinámica de ejecución como una de las principales dificultades de los estudiantes. Sorva describe esta dificultad a partir del concepto de máquina nocional, entendida como la abstracción del computador que ejecuta los programas de un lenguaje determinado, y señala que los estudiantes suelen construir modelos mentales incompletos o erróneos de dicha máquina porque su funcionamiento permanece oculto durante la ejecución [45]. La revisión de Qian y Lehman confirma que buena parte de las concepciones erróneas en programación se relaciona con lo que ocurre internamente al ejecutar una instrucción, como una asignación, y no con la sintaxis del lenguaje [46].

En el caso particular de los ambientes léxicos, Cai et al. argumentan que los modelos basados en pila no capturan adecuadamente la estructura de ambientes y clausuras, lo que motivó el desarrollo de visualizaciones específicas para representarlos [18]. Abad y Henz extendieron este enfoque a una máquina nocional completa para Scheme, que muestra paso a paso la evolución de los ambientes durante la evaluación [20]. Estas limitaciones sirven como referencia para interpretar los ítems P2 y P5, asociados a los indicadores IL 1.2.3 e IL 1.2.6.

6.2.3. Ligadura, asignación y alcance

La distinción entre ligadura y asignación, junto con los conceptos de alcance y mutación, presenta dificultades que no se resuelven necesariamente con la experiencia acumulada en programación. Fisler et al. evaluaron estudiantes de tercer y cuarto año en un curso de lenguajes de programación y encontraron que una proporción importante de ellos mantenía concepciones erróneas sobre alcance, mutación y paso de parámetros al inicio del curso, que los avances fueron parciales y dependientes del lenguaje utilizado, y que la transferencia de estos conceptos entre lenguajes no ocurre de forma espontánea [47]. Este hallazgo resulta especialmente pertinente para FLP, ya que se trata de un curso de nivel avanzado en el que se asume que los estudiantes dominan estos conceptos a partir de su formación previa. Estas limitaciones sirven como referencia para interpretar el ítem P3, asociado al indicador IL 1.2.7.

6.2.4. Herramientas de trabajo como fuente de carga adicional

La literatura también reconoce que las herramientas empleadas durante el aprendizaje pueden constituir una fuente de dificultad independiente del contenido. Cuando el estudiante debe coordinar información dispersa entre varios archivos y la consola, o descifrar una sintaxis poco familiar, los recursos cognitivos disponibles para la comprensión del contenido disminuyen [9, 11]. En el caso de los lenguajes con notación basada en paréntesis, la estructura visual plana del código puede obstaculizar la comprensión de su semántica [10]. Asimismo, la efectividad de una herramienta de visualización depende del grado en que su representación corresponde al modelo conceptual que el estudiante debe construir [13]. Estas limitaciones sirven como referencia para interpretar los ítems P4, P6 y P7, que valoran el prototipo como recurso de apoyo frente a las herramientas empleadas actualmente en la asignatura.

Capítulo 7

Conclusiones

Bibliografía

[1] Q. Liu, W. Zhao, D. Liu, and J. Ji, "Evaluation of situational case-based teaching technique in engineering education," in *Proceedings of the ACM Turing 50th Celebration Conference - China*, ACM TURC '17, pp. 1–7, Association for Computing Machinery, May 2017.

[2] E. A. Navas-López, "Modular and didactic compiler design with xml inter-phases communication," *International Journal of Computer Science, Engineering and Information Technology (IJCSEIT)*, vol. 12, pp. 1–15, February 2022.

[3] L. Spigariol, J. Bono, and F. S. Guijarro, "Aprendiendo a desarrollar un intérprete de un lenguaje de programación funcional," in *Actas del XXVIII Congreso Argentino de Ciencias de la Computación (CACIC)*, (La Rioja, Argentina), pp. 166–175, Universidad Tecnológica Nacional; Universidad Nacional de General San Martín; Universidad de Buenos Aires; Universidad Nacional de Hurlingham, Red de Universidades con Carreras en Informática, Oct. 2023.

[4] N. Jovanović, S. Stamenković, and D. Miljković, "Comvis — interactive simulation environment for compiler learning," *Computer Applications in Engineering Education*, vol. 30, no. 2, pp. 275–291, 2021.

[5] J. Lu, M. Schmidt, M. Lee, and R. Huang, "Usability research in educational technology: A state-of-the-art systematic review," *Educational Technology Research and Development*, vol. 70, no. 6, pp. 1951–1992, 2022.

[6] S. Stamenković and N. Jovanović, "A web-based educational system for teaching compilers," *IEEE Transactions on Learning Technologies*, vol. 17, pp. 143–156, 2024.

[7] D. P. Friedman and M. Wand, *Essentials of Programming Languages, 3rd Edition*. The MIT Press, 3 ed., 2008.

[8] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*. Pearson Addison-Wesley, 2 ed., 2006.

[9] J. Sweller, "Cognitive load theory and individual differences," *Learning and Individual Differences*, vol. 110, p. 102423, 2024.

[10] C. S. Gordon, "The linguistics of programming," in *Proceedings of the 2024 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! '24)*, (New York, NY, USA), pp. 1–21, ACM, 2024.

[11] J. Zagami, *Cognitive Influences on Learning Programming*, pp. 389–399. Springer, 02 2023.

[12] M. Felleisen, R. B. Findler, M. Flatt, S. Krishnamurthi, E. Barzilay, J. McCarthy, and S. Tobin-Hochstadt, "A programmable programming language," *Communications of the ACM*, vol. 61, no. 3, pp. 62–71, 2018.

[13] J. Sorva, A. Karavirta, and L. Malmi, "A review of generic program visualization systems for introductory programming education," *ACM Transactions on Computing Education*, vol. 13, no. 4, pp. 1–64, 2013.

[14] P. J. Guo, "Online Python Tutor: Embeddable web-based program visualization for CS education," in *Proceedings of the 44th ACM Technical Symposium on Computer Science Education*, pp. 579–584, ACM, 2013.

[15] J. Clements, M. Flatt, and M. Felleisen, "Modeling an algebraic stepper," in *Programming Languages and Systems (ESOP 2001)*, vol. 2028 of *Lecture Notes in Computer Science*, pp. 320–334, Springer, 2001.

[16] D. A. Norman, *The Design of Everyday Things: Revised and Expanded Edition.* New York: Basic Books, 2013.

[17] C. Hannebauer, M. Hesenius, and V. Gruhn, "Does syntax highlighting help programming novices?," *Empirical Software Engineering*, vol. 23, no. 4, pp. 1987–2013, 2018.

[18] K. Cai, M. Henz, K.-L. Low, X. Y. Ng, J. R. Soh, K.-H. Tang, and K. W. Toh, "Visualizing environments of modern scripting languages," in *Proceedings of the 15th International Conference on Computer Supported Education (CSEDU 2023)*, pp. 146–153, SCITEPRESS, 2023.

[19] R. del Vado Vírseda, "Visualizing compiler design theory from implementation through an interactive tutoring tool: Experiences and results," in *Proceedings of the 15th International Conference on Computer Supported Education (CSEDU 2023)*, pp. 333–340, SCITEPRESS, 2023.

[20] K. M. Abad and M. Henz, "Beyond SICP — design and implementation of a notional machine for Scheme," 2024.

[21] J. C. Henao Gallego, "Sistema de recomendación de materiales didácticos para el curso de Fundamentos de Compilación e Interpretación de Lenguajes de Programación." Trabajo de grado, Programa Académico de Ingeniería de Sistemas, Universidad del Valle, sede Tuluá, 2026.

[22] A. Joshi, S. Kale, S. Chandel, and D. K. Pal, "Likert scale: Explored and explained," *British Journal of Applied Science & Technology*, vol. 7, no. 4, pp. 396–403, 2015.

[23] B. Németh, E. Choi, E. Makihara, and H. Iida, "Investigating compilation errors of students learning haskell," *Trends in Functional Programming in Education*, vol. 295, pp. 52–64, 2019.

[24] "IEEE recommended practice for software requirements specifications," Tech. Rep. IEEE Std 830-1998, Institute of Electrical and Electronics Engineers, New York, NY, USA, 1998.

[25] "Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models," Tech. Rep. ISO/IEC 25010:2011, International Organization for Standardization, Geneva, Switzerland, 2011.

[26] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.

[27] M. Fowler and J. Lewis, "Microservices: a definition of this new architectural term," *martinfowler.com*, March 2014. Consultado en 2025.

[28] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*. Pearson, 4 ed., 2015.

[29] Vercel, "Next.js — the react framework for the web." `https://nextjs.org`, 2024. Versión 16.2.6.

[30] Meta Open Source, "React — a javascript library for building user interfaces." `https://react.dev`, 2024. Versión 19.

[31] Microsoft, "Typescript — javascript with syntax for types." `https://www.typescriptlang.org`, 2024. Versión 5.9.

[32] Microsoft, "Monaco editor." `https://microsoft.github.io/monaco-editor`, 2024. Versión 0.55.

[33] Tailwind Labs, "Tailwind CSS — a utility-first CSS framework." `https://tailwindcss.com`, 2024. Versión 4.

[34] D. Merkel, "Docker: Lightweight linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, March 2014.

[35] pnpm Contributors, "pnpm — fast, disk space efficient package manager." `https://pnpm.io`, 2024. Versión 10.

[36] Coolify, "Coolify — an open-source & self-hostable heroku / netlify / vercel alternative." `https://coolify.io`, 2024.

[37] Coolify, "Traefik overview." `https://coolify.io/docs/knowledge-base/proxy/traefik/overview`, 2024. Documentación oficial de Coolify.

[38] SonarSource, "Quality gates." `https://docs.sonarsource.com/sonarqube-cloud/standards/quality-gates`, 2024. Documentación oficial de SonarQube Cloud (SonarCloud).

[39] SonarSource, "Test coverage." `https://docs.sonarsource.com/sonarqube-cloud/analyzing-source-code/test-coverage/overview`, 2024. Documentación oficial de SonarQube Cloud (SonarCloud).

[40] SonarSource, "Metric definitions." `https://docs.sonarsource.com/sonarqube-cloud/managing-your-projects/metric-definitions`, 2024. Documentación oficial de SonarQube Cloud (SonarCloud).

[41] J. Brooke, "SUS: A quick and dirty usability scale," in *Usability Evaluation in Industry* (P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, eds.), pp. 189–194, London: Taylor & Francis, 1996.

[42] A. Bangor, P. Kortum, and J. Miller, "Determining what individual SUS scores mean: Adding an adjective rating scale," *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009.

[43] Congreso de Colombia, "Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales." Diario Oficial No. 48.587, 2012. 17 de octubre de 2012.

[44] L. J. Cronbach, "Coefficient alpha and the internal structure of tests," *Psychometrika*, vol. 16, no. 3, pp. 297–334, 1951.

[45] J. Sorva, "Notional machines and introductory programming education," *ACM Transactions on Computing Education*, vol. 13, no. 2, pp. 1–31, 2013.

[46] Y. Qian and J. Lehman, "Students' misconceptions and other difficulties in introductory programming: A literature review," *ACM Transactions on Computing Education*, vol. 18, no. 1, pp. 1–24, 2017.

[47] K. Fisler, S. Krishnamurthi, and P. Tunnell Wilson, "Assessing and teaching scope, mutation, and aliasing in upper-level undergraduates," in *Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education*, pp. 213–218, ACM, 2017.

Anexo A

Microcurrículo de FLP

https://docs.google.com/spreadsheets/d/1duTwpkge6Ln9R7umk3b2TuNqEoKkBEqN/edit?usp=drive_link&ouid=117991041727654766098&rtpof=true&sd=true

Anexo B

Encuesta a estudiantes

https://docs.google.com/spreadsheets/d/1iAudDGoibOyQa7UZzohLvkP29nifeoau/edit?usp=sharing&ouid=117991041727654766098&rtpof=true&sd=true

Anexo C

Encuesta a docentes

https://docs.google.com/spreadsheets/d/
1qeH1EeaJMBcqJZjNWuMYajdB9eqsvoa4XCK5dhJFenU/edit?gid=1616479297#gid=1616479297

Anexo D

Protocolo de búsqueda sistemática de literatura

Este anexo documenta el protocolo de búsqueda sistemática utilizado para la identificación de los antecedentes presentados en la Sección 2.3 del Capítulo 2 (Tabla 2.1), siguiendo la estructura de reporte propuesta por PRISMA 2020 (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) adaptada a un protocolo de identificación de trabajos relacionados. Se documentan las preguntas de búsqueda, las fuentes de información, las cadenas booleanas empleadas en cada base de datos académica, los criterios de elegibilidad, el proceso de selección y la correspondencia entre los estudios incluidos y las referencias del documento.

D.1. Preguntas de búsqueda

- **PB1:** ¿Qué herramientas o prototipos con componente de visualización o interactividad se han propuesto para apoyar la enseñanza de la interpretación o compilación de lenguajes de programación entre 2021 y 2026?
- **PB2:** De esas herramientas, ¿cuáles abordan específicamente la visualización del ambiente de ejecución o la evaluación de expresiones en lenguajes funcionales, y qué evidencia de uso o evaluación reportan?

D.2. Fuentes de información

D.2.1. Bases de datos primarias

La búsqueda sistemática se ejecutó en tres bases de datos académicas, acotada al período 2021–2026:

- **IEEE Xplore Digital Library**
- **ACM Digital Library**
- **Scopus**

D.2.2. Fuentes complementarias

Dado que parte de la producción relevante en el ámbito latinoamericano y los preprints no están indexados en las tres bases anteriores, se complementó la búsqueda sistemática con:

- **Snowballing** (rastreo hacia adelante y hacia atrás de citas) a partir de los estudios ya incluidos, en particular desde el ecosistema Source Academy [18], que permitió identificar la máquina nocional CSE Machine [20] publicada como preprint en arXiv.
- **Repositorios regionales y actas de congresos no indexadas en las bases anteriores**: SEDICI (Repositorio Institucional de la Universidad Nacional de La Plata) y actas del Congreso Argentino de Ciencias de la Computación (CACIC), que permitieron identificar el intérprete pedagógico funcional de Spigariol et al. [3].

Esta decisión metodológica se documenta explícitamente porque ninguna de las tres bases primarias indexa estas dos fuentes (arXiv no es indexado como texto completo por IEEE Xplore, ACM DL ni Scopus; SEDICI/CACIC es un repositorio regional fuera del alcance editorial de las tres bases), por lo que su inclusión exclusivamente a través de la búsqueda sistemática habría sido imposible.

D.3. Estrategia de búsqueda

D.3.1. Conceptos y términos

La estrategia de búsqueda se construyó combinando tres bloques conceptuales mediante el operador `AND`, con sinónimos y variantes combinados mediante `OR` dentro de cada bloque, tal como se resume en la Tabla D.1.

Bloque	**Descripción**	**Términos (`OR`)**
C1 — Objeto	Sistema o artefacto de lenguaje sobre el que trabaja la herramienta	`interpreter`, `compiler`, `"programming language"`
C2 — Foco	Mecanismo de representación o visualización empleado	`visualiz*`, `"notional machine"`, `"environment diagram"`, `"environment model"`, `"execution environment"`, `"abstract syntax tree"`
C3 — Contexto	Ámbito pedagógico de aplicación	`teach*`, `educat*`, `learn*`, `course`, `pedagog*`

Tabla D.1: Bloques conceptuales de la estrategia de búsqueda (C1 `AND` C2 `AND` C3).

D.3.2. Cadenas de búsqueda por base de datos

Las cadenas siguientes instancian los bloques de la Tabla D.1 en la sintaxis propia de cada base, filtrando por el período 2021–2026.

IEEE Xplore (Command Search)

```
("Full Text & Metadata":interpreter OR
 "Full Text & Metadata":compiler OR
 "Full Text & Metadata":"programming language")
```

```
AND
("Full Text & Metadata":visualiz* OR
 "Full Text & Metadata":"notional machine" OR
 "Full Text & Metadata":"environment diagram" OR
 "Full Text & Metadata":"environment model" OR
 "Full Text & Metadata":"execution environment" OR
 "Full Text & Metadata":"abstract syntax tree")
AND
("Full Text & Metadata":teach* OR
 "Full Text & Metadata":educat* OR
 "Full Text & Metadata":learn* OR
 "Full Text & Metadata":course OR
 "Full Text & Metadata":pedagog*)
```

Filtros aplicados: *Content Type* = Conferences, Journals; *Year*: 2021–2026.

ACM Digital Library (Advanced Search)

```
[[Title: interpreter] OR [Title: compiler] OR [Title: "programming language"]
 OR [Abstract: interpreter] OR [Abstract: compiler]
 OR [Abstract: "programming language"]]
AND
[[Title: visualization] OR [Title: "notional machine"]
 OR [Abstract: visualization] OR [Abstract: "notional machine"]
 OR [Abstract: "environment diagram"] OR [Abstract: "environment model"]
 OR [Abstract: "execution environment"] OR [Abstract: "abstract syntax tree"]]
AND
[[Title: teaching] OR [Title: education] OR [Title: course]
 OR [Abstract: teaching] OR [Abstract: education]
 OR [Abstract: learning] OR [Abstract: pedagogy]]
```

Filtros aplicados: *Publication Date*: 01/2021 – 12/2026.

Scopus (Document Search)

```
TITLE-ABS-KEY (
  ( interpreter OR compiler OR "programming language" )
  AND
  ( visualiz* OR "notional machine" OR "environment diagram"
    OR "environment model" OR "execution environment"
    OR "abstract syntax tree" )
  AND
  ( teach* OR educat* OR learn* OR course OR pedagog* )
)
AND PUBYEAR > 2020 AND PUBYEAR < 2027
AND ( LIMIT-TO ( DOCTYPE , "cp" ) OR LIMIT-TO ( DOCTYPE , "ar" ) )
```

Filtros aplicados: *Document type*: Conference Paper, Article; *Subject area*: Computer Science.

D.4. Criterios de elegibilidad

D.4.1. Criterios de inclusión

- **CI1:** Presenta una herramienta, prototipo o sistema con componente de visualización o interactividad propio (no un estudio puramente teórico o de opinión).
- **CI2:** Está orientado a la enseñanza de al menos una fase del proceso de interpretación o compilación de lenguajes de programación (análisis léxico, análisis sintáctico, construcción del AST, ambiente de ejecución o evaluación de expresiones).
- **CI3:** Reporta al menos una experiencia de uso, evaluación o validación con estudiantes, aunque sea preliminar.
- **CI4:** Fue publicado entre 2021 y 2026, o fue identificado mediante búsqueda complementaria (snowballing o repositorios regionales) por su relevancia directa para PB1/PB2.
- **CI5:** Texto completo disponible en español o inglés.

D.4.2. Criterios de exclusión

- **CE1:** Aborda visualización de algoritmos o estructuras de datos generales sin relación específica con interpretación o compilación de lenguajes.
- **CE2:** Es una revisión de literatura, editorial, panel o resumen extendido que no presenta una herramienta propia.
- **CE3:** No reporta ningún tipo de evaluación, validación o experiencia de uso.
- **CE4:** Es un duplicado o una versión preliminar (*workshop paper*) de un estudio ya incluido en una versión extendida.
- **CE5:** El texto completo no está disponible pese a la solicitud a través del acceso institucional.

D.5. Proceso de selección

La selección de estudios siguió las cuatro etapas del modelo PRISMA 2020: identificación, cribado, elegibilidad e inclusión. Los registros recuperados de las tres bases de datos se cribaron por título y resumen aplicando los criterios CI1–CI5 y CE1–CE5; los registros que superaron el cribado se evaluaron a texto completo bajo los mismos criterios. En paralelo, los registros identificados mediante snowballing y repositorios regionales se incorporaron directamente a la etapa de evaluación a texto completo, dado que su relevancia ya había sido establecida por su relación directa con los estudios incluidos desde las bases primarias. La Figura D.1 resume el proceso; el conteo exacto de registros brutos y duplicados por base se obtiene al ejecutar las cadenas de la Sección D.3 con acceso institucional, dado que las interfaces de búsqueda avanzada de las tres bases exigen sesión autenticada y bloquean la consulta automatizada.

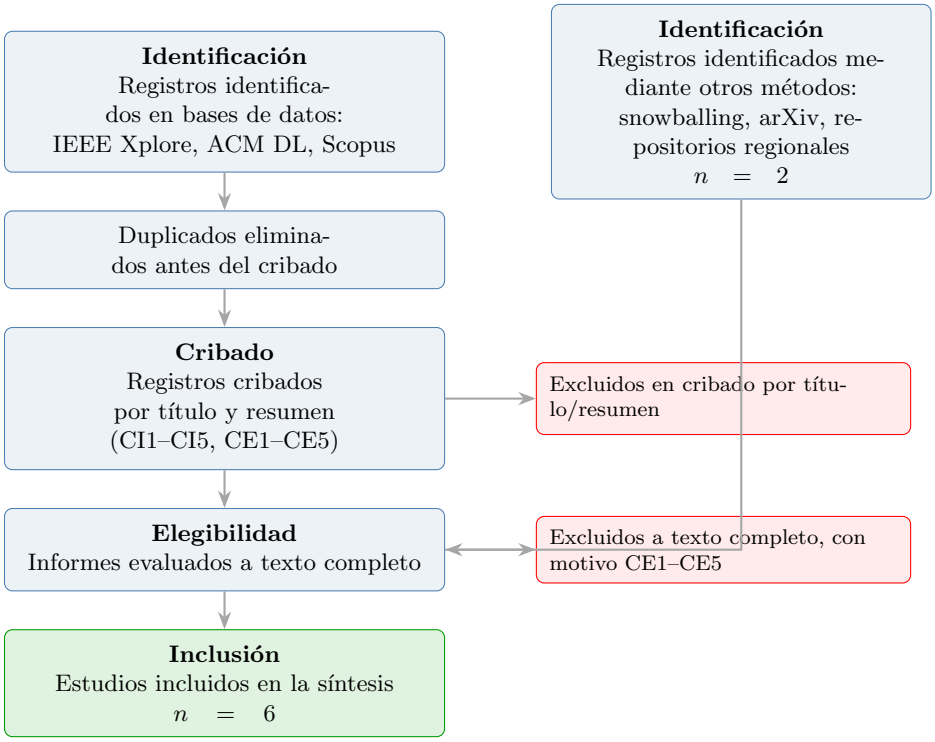


Figura D.1: Diagrama de flujo PRISMA del proceso de selección de antecedentes.

D.6. Estudios incluidos

La Tabla D.2 presenta la correspondencia entre los seis estudios incluidos en la síntesis de la Tabla 2.1 y la vía por la cual fueron identificados.

Estudio	Referencia	Vía de identificación
Jovanović et al. (2021)	[4]	Búsqueda sistemática (Scopus)
Cai et al. (2023)	[18]	Búsqueda sistemática (Scopus)
del Vado Vírseda (2023)	[19]	Búsqueda sistemática (Scopus)
Spigariol et al. (2023)	[3]	Búsqueda complementaria (repositorio SEDICI / actas CACIC)
Abad y Henz (2024)	[20]	Búsqueda complementaria (snowballing desde [18]; preprint arXiv)
Stamenkovic y Jovanovic (2024)	[6]	Búsqueda sistemática (IEEE Xplore, Scopus)

Tabla D.2: Estudios incluidos en la síntesis de antecedentes y su vía de identificación.

Anexo E

Requisitos del sistema

E.1. Requisitos Funcionales

Código	RF-01
Título	Generación del Árbol de Sintaxis Abstracta (AST)
Descripción	El sistema debe procesar el programa ingresado por el usuario utilizando la gramática previamente definida para generar su correspondiente Árbol de Sintaxis Abstracta (AST).
Prioridad	Alta
Criterios de aceptación	▪ El sistema invoca el intérprete Racket con los archivos del editor y la expresión de entrada; el AST resultante es serializado en formato JSON y transmitido al cliente. ▪ El AST generado es renderizado en el panel ASTViewer reflejando la estructura jerárquica del programa evaluado. ▪ Cuando el programa contiene errores sintácticos o de evaluación, el sistema muestra un mensaje de error en la consola con el prefijo × y nivel visual diferenciado en rojo; el panel ASTViewer conserva el último AST válido. ▪ El AST corresponde fielmente a la última expresión evaluada exitosamente.

Tabla E.1: RF-01: Generación del AST

Código	RF-02
Título	Visualización gráfica e interactiva del AST
Descripción	El sistema debe mostrar el árbol de sintaxis abstracta generado de forma gráfica e interactiva, permitiendo la exploración visual de su estructura.
Prioridad	Alta

Criterios de aceptación	▪ El panel ASTViewer renderiza la representación jerárquica del AST con nodos que indican el tipo de dato retornado por el intérprete. ▪ La visualización se actualiza automáticamente tras cada ejecución exitosa, sin necesidad de acción adicional por parte del usuario. ▪ El panel es responsivo y se adapta al tamaño del contenedor asignado dentro del layout redimensionable. ▪ En modo paso a paso, el panel refleja el AST correspondiente al paso de evaluación actualmente visualizado. ▪ No se presentan errores de renderizado ni contenido huérfano durante el ciclo de vida de la sesión.

Tabla E.2: RF-02: Visualización gráfica e interactiva del AST

Código	RF-03
Título	Ejecución y evaluación del programa
Descripción	El sistema debe permitir ejecutar el programa del usuario e interpretar su resultado, siguiendo el proceso de evaluación definido por la gramática y el intérprete generado.
Prioridad	Alta
Criterios de aceptación	▪ El sistema transmite los archivos del editor y la expresión de entrada a la ruta `POST /api/run` y recibe el arreglo de pasos de evaluación. ▪ El resultado de cada paso es presentado en la consola con nivel de salida correspondiente: verde (`output`) para valores, rojo (`error`) para excepciones capturadas, gris (`info`) para resultados `void`. ▪ Los errores producidos por `eopl:error` son capturados por el runtime, limpiados de trazas internas de Racket, y presentados con el prefijo `×` en color rojo. ▪ La ejecución puede ser cancelada por el usuario durante el proceso; el servidor aborta el subproceso Racket correspondiente. ▪ El tiempo máximo de ejecución del subproceso Racket es de 15 segundos; transcurrido ese tiempo, el sistema reporta el error de tiempo límite en la consola. ▪ El sistema soporta un modo de evaluación paso a paso que permite avanzar la ejecución de forma controlada mostrando el estado del AST y del ambiente en cada paso.

Tabla E.3: RF-03: Ejecución y evaluación del programa

Código	RF-04
Título	Representación del ambiente de ejecución
Descripción	El sistema debe representar el ambiente de evaluación como una estructura jerárquica de marcos que se actualiza en cada ejecución, permitiendo al estudiante observar la evolución del estado del programa.
Prioridad	Alta
Criterios de aceptación	▪ El panel EnvironmentPanel muestra el ambiente como una pila de marcos jerárquicos, donde cada marco lista las ligaduras activas con su nombre, valor serializado y tipo de dato inferido. ▪ El ambiente mostrado corresponde al último paso de evaluación completado; en modo paso a paso, se actualiza con cada avance del usuario. ▪ La estructura de marcos refleja fielmente la cadena de ambientes construida durante la evaluación, permitiendo al estudiante identificar el alcance léxico de cada variable. ▪ El panel permanece visible y no colapsa ante la ausencia de ligaduras en el ambiente actual.

Tabla E.4: RF-04: Representación del ambiente de ejecución

Código	RF-05
Título	Biblioteca de ejemplos
Descripción	El sistema debe ofrecer una biblioteca de ejemplos predefinidos orientados a los indicadores de logro identificados como base conceptual del prototipo, con el propósito de facilitar el aprendizaje mediante casos de uso concretos.
Prioridad	Media
Criterios de aceptación	▪ Los ejemplos son cargados desde el sistema de archivos del servidor durante el renderizado inicial y transmitidos al cliente como propiedades del componente orquestador. ▪ El sistema presenta la lista de ejemplos disponibles en el modal de bienvenida con nombre y descripción breve de cada uno. ▪ Al seleccionar un ejemplo, los archivos del editor son reemplazados por los archivos correspondientes sin pérdida de estado de la interfaz. ▪ Los ejemplos cubren los cinco indicadores de logro seleccionados en el diagnóstico: IL 1.1.6, IL 1.2.3, IL 1.2.6, IL 1.2.7 e IL 1.3.1. ▪ Cada ejemplo es compatible con el pipeline de gramática y puede ser ejecutado sin modificaciones adicionales.

Tabla E.5: RF-05: Biblioteca de ejemplos

Código	RF-06
Título	Soporte y validación de sintaxis en Racket
Descripción	El sistema debe ser capaz de reconocer la sintaxis del lenguaje Racket en el editor y validar la gramática BNF ingresada por el usuario, reportando errores de forma clara y oportuna.
Prioridad	Alta
Criterios de aceptación	▪ El editor Monaco provee resaltado de sintaxis para archivos con extensión `.rkt`. ▪ El pipeline de gramática detecta y reporta errores léxicos en la especificación BNF durante el procesamiento; los mensajes son presentados en el modal de gramática antes de generar los archivos. ▪ El sistema no genera los archivos del intérprete ni habilita la ejecución cuando la gramática BNF contiene errores léxicos o sintácticos. ▪ Los errores de evaluación producidos por el intérprete Racket son presentados en la consola sin exponer trazas internas del runtime al usuario.

Tabla E.6: RF-06: Soporte y validación de sintaxis en Racket

Código	RF-07
Título	Visualización de los componentes internos del intérprete
Descripción	El sistema debe permitir al estudiante visualizar los archivos que componen el intérprete generado, mostrando su estructura con distinción entre las secciones técnicas y las secciones destinadas a la implementación del estudiante.
Prioridad	Media
Criterios de aceptación	▪ El panel de edición organiza los archivos del intérprete en pestañas independientes, accesibles mediante navegación por clic. ▪ Las líneas generadas automáticamente por el pipeline son marcadas como de solo lectura y presentadas con distinción visual respecto a las líneas editables. ▪ Las líneas destinadas a la implementación del estudiante (stubs de `eval-program`, `eval-expression`, `apply-primitive`) permanecen activas y editables. ▪ No se permite la modificación de las líneas protegidas durante la sesión de trabajo.

Tabla E.7: RF-07: Visualización de los componentes internos del intérprete

Código	RF-08
Título	Plantilla base editable y exportación del entorno
Descripción	El sistema debe proporcionar una plantilla base del intérprete donde el estudiante pueda completar la implementación, y permitir la descarga del proyecto completo en formato `.zip` compatible con la librería EOPL.
Prioridad	Alta
Criterios de aceptación	▪ Al confirmar la gramática, el pipeline genera automáticamente los cuatro archivos del intérprete con los stubs de las funciones a implementar, cargándolos en el editor sin intervención adicional del usuario. ▪ Las secciones de infraestructura técnica del intérprete están protegidas y no son modificables por el usuario. ▪ Las secciones de implementación están identificadas con comentarios `;; TODO` que orientan al estudiante sobre qué debe completar. ▪ El sistema genera un archivo `.zip` con todos los archivos del editor en su estado actual, incluyendo las modificaciones realizadas por el usuario. ▪ El archivo descargado incluye una estructura de directorios compatible con la librería EOPL y puede ser abierto directamente en DrRacket.

Tabla E.8: RF-08: Plantilla base editable y exportación del entorno

E.2. Requisitos No Funcionales

Código	RNF-01
Título	Usabilidad
Descripción	El sistema debe ser intuitivo y operable por estudiantes sin experiencia avanzada en compiladores, minimizando la carga cognitiva extrínseca impuesta por la interfaz.
Prioridad	Alta

Criterios de aceptación	▪ Los usuarios pueden iniciar una sesión de trabajo, definir una gramática y ejecutar un programa sin consultar documentación externa. ▪ Los mensajes de error presentados en la consola son comprensibles sin conocimiento del funcionamiento interno de Racket. ▪ La distribución de paneles sigue un orden visual coherente con el flujo de trabajo del estudiante: definición de gramática, edición del intérprete, ejecución y observación de resultados. ▪ Las secciones del editor que no deben ser modificadas están diferenciadas visualmente de forma que el estudiante pueda identificarlas sin instrucción explícita.

Tabla E.9: RNF-01: Usabilidad

Código	RNF-02
Título	Rendimiento
Descripción	El sistema debe completar la generación del AST y la evaluación del programa en un tiempo que no interrumpa el flujo cognitivo del estudiante durante la exploración.
Prioridad	Alta
Criterios de aceptación	▪ La transformación BNF a archivos Racket se completa en el cliente en menos de 1 segundo para gramáticas de hasta 20 producciones. ▪ La respuesta de la ruta `/api/run` para programas de complejidad habitual en la asignatura FLP se recibe en menos de 5 segundos bajo condiciones normales de red. ▪ El tiempo máximo de ejecución del subproceso Racket está limitado a 15 segundos; transcurrido ese tiempo, el sistema notifica al usuario y libera los recursos asociados. ▪ No se producen bloqueos del hilo principal del navegador durante la ejecución de la petición al servidor.

Tabla E.10: RNF-02: Rendimiento

Código	RNF-03
Título	Accesibilidad web
Descripción	El sistema debe ejecutarse en navegadores web modernos sin requerir instalación de software adicional, plugins ni autenticación por parte del usuario.
Prioridad	Alta

Criterios de aceptación	▪ La aplicación carga y opera correctamente en las versiones actuales de Google Chrome, Mozilla Firefox y Microsoft Edge. ▪ No se requiere instalación de extensiones, plugins ni entornos de ejecución adicionales en el equipo del usuario. ▪ El acceso al sistema no requiere registro ni autenticación de ningún tipo. ▪ La interfaz es funcional en resoluciones de pantalla iguales o superiores a 1280 × 720 píxeles.

Tabla E.11: RNF-03: Accesibilidad web

Código	RNF-04
Título	Mantenibilidad
Descripción	El código fuente debe estar organizado de forma modular, con separación explícita de responsabilidades, para facilitar la incorporación de mejoras futuras.
Prioridad	Media
Criterios de aceptación	▪ Las definiciones de tipos e interfaces TypeScript residen exclusivamente en el directorio `app/types/`, con importaciones explícitas desde los módulos que las consumen. ▪ Las funciones puras y utilitarios que no dependen del estado React residen en `app/lib/` y no en los archivos de componentes. ▪ El pipeline de gramática está segmentado en módulos independientes (lexer, parser, generadores) con responsabilidades no solapadas. ▪ Es posible añadir un nuevo generador de archivo Racket incorporando un módulo en `app/lib/` sin modificar los componentes de interfaz.

Tabla E.12: RNF-04: Mantenibilidad

Código	RNF-05
Título	Escalabilidad
Descripción	La arquitectura del sistema debe permitir incorporar nuevos ejemplos, gramáticas de mayor complejidad o características adicionales sin afectar la estabilidad de las funcionalidades existentes.
Prioridad	Media

Criterios de aceptación	▪ La incorporación de nuevos ejemplos se realiza añadiendo archivos al directorio de ejemplos del servidor, sin modificar código de componentes ni de la API. ▪ El pipeline de gramática procesa gramáticas con hasta 20 producciones sin degradación observable del rendimiento en el cliente. ▪ La adición de un nuevo panel de visualización puede realizarse como un componente React independiente integrable en el layout sin alterar el comportamiento de los paneles existentes. ▪ El sistema de contenedores Docker permite escalar el servicio de ejecución de forma independiente al servidor de la interfaz.

Tabla E.13: RNF-05: Escalabilidad

Anexo F

Instrumento de evaluación del prototipo

Este anexo documenta el instrumento aplicado durante la evaluación del prototipo FLP Viewer reportada en el Capítulo 6, incluyendo la ficha técnica y los ítems de cada sección.

F.1. Ficha técnica

Instrumento dirigido a	Estudiantes matriculados en FLP que participaron en la sesión de evaluación del prototipo
Tamaño de muestra esperado	POR DEFINIR estudiantes, conforme al alcance definido para el trabajo de grado
Duración estimada	POR DEFINIR
Momento de aplicación	Inmediatamente después de que el estudiante interactuara con las tres funcionalidades núcleo del prototipo: edición de la gramática, visualización del AST y visualización del ambiente de ejecución

Tabla F.1: Ficha técnica del instrumento de evaluación

F.2. Consentimiento informado

Esta encuesta hace parte de la evaluación del prototipo FLP Viewer, desarrollado como trabajo de grado del programa de Ingeniería de Sistemas por los estudiantes Jota Emilio López Ramírez y Esmeralda Rivas Guzmán. Su participación es voluntaria y anónima, no se solicitan datos que permitan identificarlo y sus respuestas no tienen ningún efecto sobre la calificación de la asignatura. La información recolectada será analizada y almacenada en hojas de cálculo que harán parte de los anexos del capitulo de Evaluación, y se utilizará exclusivamente con fines académicos, conforme a lo dispuesto en la Ley 1581 de 2012 sobre protección de datos personales [43]. Al continuar y responder el formulario, usted manifiesta su consentimiento para participar en el estudio.

F.3. Sección A. Datos de contexto

1. Semestre que cursa actualmente.
2. ¿Ha usado Racket antes de esta asignatura? Sí / No.
3. ¿Había usado el prototipo antes de participar en la evaluación? Sí / No.

F.4. Sección B. Escala de Usabilidad del Sistema (SUS)

Para cada afirmación de la Tabla F.2, el estudiante indicó su grado de acuerdo en una escala de cinco puntos, donde 1 corresponde a "Totalmente en desacuerdo" y 5 a "Totalmente de acuerdo" [41].

#	**Afirmación**
1	Creo que me gustaría usar este sistema con frecuencia.
2	Encontré el sistema innecesariamente complejo.
3	Pensé que el sistema era fácil de usar.
4	Creo que necesitaría el apoyo de una persona con conocimientos técnicos para poder utilizar este sistema.
5	Encontré que las distintas funciones de este sistema estaban bien integradas.
6	Pensé que había demasiada inconsistencia en este sistema.
7	Me imagino que la mayoría de las personas aprenderían a usar este sistema muy rápidamente.
8	Encontré el sistema muy incómodo/engorroso de usar.
9	Me sentí muy seguro/a utilizando el sistema.
10	Necesité aprender muchas cosas antes de poder manejarme con este sistema.

Tabla F.2: Ítems del cuestionario SUS aplicado

El puntaje SUS se calculó de la siguiente manera: para los ítems impares (1, 3, 5, 7, 9), puntaje = respuesta − 1; para los ítems pares (2, 4, 6, 8, 10), puntaje = 5 − respuesta. Los diez puntajes se sumaron y el resultado se multiplicó por 2.5, obteniendo un valor entre 0 y 100. La interpretación de dicho valor siguió la escala de adjetivos de referencia (menor a 51: pobre; 51 a 67.9: aceptable; 68 a 80.3: bueno; mayor a 80.3: excelente) [42].

F.5. Sección C. Cuestionario de utilidad pedagógica

Para cada afirmación de la Tabla F.3, el estudiante indicó su grado de acuerdo en la misma escala de cinco puntos utilizada en la Sección B.

#	**Afirmación**
P1	La visualización del AST ofrece una forma directa de representar la relación entre una gramática BNF y una estructura jerárquica.
P2	La visualización del ambiente de ejecución resulta útil para explorar los conceptos de estado y ligaduras de variables.

P3	La representación utilizada en el prototipo permite diferenciar visualmente entre ligadura y asignación de variables.
P4	El prototipo constituye un complemento de utilidad para el trabajo realizado con Racket y EOPL.
P5	La ejecución paso a paso ofrece una forma adecuada de observar el orden de evaluación de los programas ingresados.
P6	La biblioteca de ejemplos predefinidos ofrece un punto de partida adecuado para explorar los conceptos antes de construir una gramática propia.
P7	Considero que el prototipo tiene valor como recurso de apoyo para el estudio de los contenidos de la asignatura Fundamentos de interpretación y compilación de lenguajes de programación (FLP).
P8	Las funcionalidades disponibles en el prototipo tienen poca utilidad como apoyo para el estudio de los conceptos abordados.

Tabla F.3: Ítems del cuestionario de utilidad pedagógica

El ítem P8 está redactado en sentido inverso respecto de los demás ítems del bloque, como control de respuestas por aquiescencia, su puntaje se invierte (6 menos la respuesta registrada) antes del análisis. La fiabilidad interna del bloque se estimó mediante el coeficiente alfa de Cronbach [44].

La Tabla F.4 presenta la correspondencia entre cada ítem y el indicador de logro del microcurrículo de FLP (Anexo A) que orientó su formulación.

Ítem	**Indicador de logro relacionado**
P1	IL 1.1.6 – Construcción del AST a partir de una gramática BNF
P2, P5	IL 1.2.6 – Estado y ambientes de ejecución; IL 1.2.3 – Construcción de intérpretes básicos
P3	IL 1.2.7 – Distinción entre ligadura y asignación de variables
P4, P6, P7	Valoración general de la utilidad percibida del prototipo como recurso de apoyo
P8	Ítem de control, redactado en sentido inverso; no se asocia a un indicador de logro específico

Tabla F.4: Relación entre los ítems del cuestionario de utilidad pedagógica y los indicadores de logro

F.6. Sección D. Preguntas abiertas

1. ¿Qué parte del prototipo le resultó más útil para comprender los contenidos de la asignatura? ¿Por qué?
2. ¿Qué mejoraría o qué le faltó al prototipo?
3. ¿Hay algún concepto de la asignatura que el prototipo no le ayudó a comprender, o que le generó confusión adicional?